DESIGN AND CONSTRUCTION OF A FARM INTRUSION DETECTION SYSTEM USING COMPUTER VISION & CLOUD TECHNOLOGY

GODFREY IFEOLUWA AWOJOBI

(20/30GC024)

Thesis submitted to the Department of Electrical and Electronics Engineering, University of Ilorin, Ilorin, in partial fulfilment of the requirements for the award of B.Eng. (Hons) in Electrical and Electronics Engineering.

July 2026

DECLARATION

I, **AWOJOBI, Godfrey Ifeoluwa,** hereby declare that I carried out the work reported in this thesis in the Department of Electrical and Electronics Engineering, University of Ilorin, Ilorin, under the supervision of Dr. A. S. Afolabi. I also declare that to the best of my knowledge, no part of this thesis has been submitted elsewhere in a previous application for the award of a degree. All sources of knowledge used have been duly acknowledged.

……..……..

AWOJOBI Godfrey Ifeoluwa

Matric No: 20/30GC024

CERTIFICATION

This is to certify that the thesis titled **"Design and Construction of a Farm Intrusion Detection System Using Computer Vision & Cloud Technology",** carried out by **AWOJOBI, Godfrey Ifeoluwa (20/30GC024),** meets the requirements and regulations governing the award of the Bachelor of Engineering degree (B.Eng.) in Electrical and Electronics Engineering of the University of Ilorin, Ilorin, Nigeria.

_____ _____

Dr. A. S. Afolabi **Date**

(Supervisor)



_____ _____

Engr. (Dr.) S.A Olayanju **Date**

(Project Coordinator)



_____ _____

Dr. A.O Otuoze **Date**

(Head of Department)



_____ _____

Prof. F.M Dahunsi **Date**

(External Examiner)

DEDICATION

This thesis is dedicated to my parents, whose endless love, sacrifice, and encouragement have been my greatest pillar of strength throughout this journey. This work is also dedicated to my siblings for their constant patience and cheer.

ACKNOWLEDGEMENTS

First, all glory goes to God Almighty for life, strength, and the grace to see this project through, there were nights this build did not want to cooperate, and somehow it always came together.

My sincere appreciation goes to my supervisor, Dr. A. S. Afolabi, for the patience to sit through my many drafts, for his invaluable guidance, expert advice, and constant encouragement throughout the development of this research.

I am also grateful to my Level Adviser and Acting Head of Department, Dr. A. O. Otuoze, and the entire academic staff of the Department of Electrical and Electronics Engineering at the University of Ilorin for the five years of grounding that made a project like this possible in the first place.

To my parents, Apostle Dr. and Pst. Mrs. Awojobi, thank you for the unwavering support, prayers, love and care. To my siblings, Greatness, Gloria and Goodness, thank you for putting up with me during the stressful weeks, you know which ones.

To my friends and course mates, this project carries a piece of all of you in it. Finally, to everyone, who lent a hand, an idea, or just moral support along the way, thank you.

ABSTRACT

Farm produce theft, livestock rustling, and rural banditry are a persistent problem for Nigerian agriculture, and conventional sensor-based security only makes it worse: PIR alarms fire on wind and stray animals until farmers stop trusting them, and intruders have adapted by wearing masks or balaclavas specifically to defeat CCTV identification. In this project, an Intelligent Farm Intrusion Detection System (FAIDS) that pairs low-power IoT hardware at the edge with cloud-based inference to get around both problems at once was built. A Raspberry Pi 3B, wired to dual HC-SR501 PIR sensors and dual USB cameras and run off a 3S Li-ion pack with a BMS and buck converter, handles motion sensing and image capture. When something moves, the image is transmitted over an asynchronous FastAPI backend to a Microsoft Azure Custom Vision model trained to classify faces as either masked or unmasked, keeping the heavy inference off a device with no GPU to spare. A GPIO-triggered LED indicator gives an immediate local signal independent of the cloud round trip, while a React/Tailwind frontend dashboard and email pipeline handle the actual alerting. In field testing, the power system ran on battery for over 15 hours to simulate grid outage, the PIR sensors caught human targets up to a distance of 5–6 m, and the vision model hit 99.6% mAP, with precision and recall both above 97%. Average end-to-end latency, from motion to alert in the farmer's email inbox, came out to 860 ms despite the flakiness of rural cellular coverage. The results back up the core idea: cheap local sensors triggering enterprise-grade cloud inference is a workable, affordable way to secure off-grid farmland that neither pure edge nor pure cloud architectures handle well on their own.

TABLE OF CONTENTS

DECLARATION.....**ii**
CERTIFICATION.....**iii**
DEDICATION.....**iv**
ACKNOWLEDGEMENTS.....**v**
ABSTRACT.....**vi**
TABLE OF CONTENTS.....**vii**
LIST OF FIGURES.....**x**
LIST OF TABLES.....**xi**
LIST OF SYMBOLS AND ABBREVIATIONS.....**xii**
CHAPTER 1.....**1**
INTRODUCTION.....**1**
1.1 Background of the Study.....1
1.2 Statement of the Problem.....3
1.3 Aim and Objectives of the Study.....6
1.4 Significance of the Study.....7
1.5 Scope of the Study.....9
1.6 Areas of Application.....9
1.7 Thesis Outline.....11
CHAPTER 2.....**13**
LITERATURE REVIEW.....**13**
2.1 Introduction.....13
2.1.1 Hardware Context of the Proposed System.....13
2.2 IoT-Based Intrusion Detection Systems for Agricultural Environments.....15
2.3 Motion Detection Technologies: Passive Infrared (PIR) Sensors.....17
2.4 Deep Learning for Threat Classification and Personnel Authorization.....19
2.4.1 Evolution of Detection Algorithms (YOLO vs. R-CNN).....19
2.4.2 Authorized Personnel Detection Strategies.....20
2.5 Face Mask Detection Technologies.....21
2.5.1 Approaches to Face Mask Detection.....21
2.5.2 Challenges in Mask Detection Under Real-World Conditions.....22
2.6 Cloud-Based Cognitive AI Services: Microsoft Azure Face API.....23
2.6.1 Architecture and Key Features.....24

2.6.2 Accuracy, Limitations, and Responsible AI Considerations.....25
2.6.3 Prior Research Employing Azure Face API.....25
2.7 Edge Computing, Cloud Computing, and Hybrid Architectures for IoT Security. .26
2.7.1 Cloud Computing in IoT Contexts.....26
2.7.2 Edge Computing in IoT Contexts.....27
2.7.3 The Hybrid Cloud-Edge Architecture: The Optimal Paradigm.....27
2.8 The Raspberry Pi as an IoT Edge Computing Node.....29
2.9 Active Deterrence: Psychological and Technical Frameworks.....30
2.9.1 The Psychology of Deterrence and Habituation.....30
2.9.2 Strobe Lights and Audible Sirens as Deterrent Mechanisms.....31
2.10 Real-Time Notification Systems: The Telegram Bot API.....31
2.11 Review of Related Systems: Comparative Analysis and Research Gaps.....33
2.12 Theoretical Framework.....34
CHAPTER 3.....**37**
METHODOLOGY AND MATERIALS USED.....**37**
3.1 INTRODUCTION.....37
3.2 SYSTEM OVERVIEW & ARCHITECTURE.....37
3.2.1 The Edge Device Layer.....38
3.2.2 The Backend Layer.....39
3.2.3 The Frontend Layer.....40
3.3 HARDWARE DESIGN.....40
3.3.1 Circuit Diagram.....40
3.3.2 Power Unit.....42
3.3.3 Central Processing & Storage Unit.....45
3.3.4 Sensory & Visual Unit.....46
3.4 SOFTWARE DEVELOPMENT.....47
3.4.1 Microsoft Azure Services.....47
3.4.2 AI Model Development & Training.....48
3.4.3 User Interface Design.....50
3.5 SYSTEM INTEGRATION.....51
3.6 EXPERIMENTAL TEST/ DESIGN (KPI TESTS).....52
CHAPTER 4.....**55**
SYSTEM TESTING, RESULTS AND DISCUSSION.....**55**
4.1 System Testing Environment and Methodology.....55

4.1.1 Experimental Setup and Bill of Materials (BOM).....55
4.1.2 Testing Parameters and Key Performance Indicators (KPIs).....57
4.2 Hardware Architecture Evaluation.....57
4.2.1 Power Autonomy and CPU Load Profiling.....57
4.2.2 PIR Sensor Range Matrices and False Alarm Rate (FAR).....59
4.2.3 Visual Acquisition Latency Limits.....61
4.3 Software, Cloud, and Network Evaluation.....61
4.3.1 Computer Vision Detection Accuracy and Resource Utilization.....61
4.3.2 Network Scalability and Throughput Performance.....67
4.3.3 End-to-End Latency and System Availability.....68
4.4 Discussion of Results and System Limitations.....70
4.4.1 KPI Target vs. Actual Performance.....70
4.4.2 Identified Limitations.....71
CHAPTER 5.....72
SUMMARY, CONCLUSION, AND RECOMMENDATIONS.....72
5.1 Summary of the Study.....72
5.2 Summary of Key Findings.....74
5.3 Conclusion.....75
5.4 Limitations of the Study.....76
5.5 Recommendations.....77
REFERENCES.....80
APPENDICES:.....84
APPENDIX A: BEME.....84
APPENDIX B: FRONTEND AND BACKEND CODE.....86
APPENDIX C: RASPBERRY PI CODE.....115

LIST OF FIGURES

Figure 2. 1: Labeled view of the Raspberry Pi 3B highlighting major component interfaces.....14
Figure 2. 2: Labeled view of the Passive InfraRed (PIR) Sensor Module (HC-SR501).....18
Figure 3. 1: Block Diagram of a Farm Intrusion Detection System Using Computer Vision & Cloud Technology.....38
Figure 3. 2: Schematic Circuit Diagram of the Edge Device Hardware.....42
Figure 3. 3: Transfer Learning Pipeline for the Object Detection Model.....48
Figure 4. 1: Physical Prototype and Experimental Setup of the Farm Intrusion Detection System.....56
Figure 4. 2: Precision-recall curves of the Azure Custom Vision model on experimental data.....64
Figure 4. 3: Training and Validation Loss and Accuracy Over 20 Epochs.....65

LIST OF TABLES

Table 2. 1: Comparative Analysis of Deep Learning Object Detection Models.....19
Table 2. 2: Comparative Analysis of Authorized Personnel Detection Methods.....20
Table 3. 1: Power Consumption Budget of the Edge Device Layer at a 5V Logic Rail.....43
Table 3. 2: Consolidated KPI Test Plan & Target Benchmarks.....52
Table 4. 1: HC-SR501 Sensor Distance and Alert Transmission Testing.....59
Table 5. 1: Summary of Study Objectives and Achievement Status.....73

LIST OF SYMBOLS AND ABBREVIATIONS

- **API** – Application Programming Interface
- **CPU** – Central Processing Unit
- **CV** – Computer Vision
- **DC** – Direct Current
- **GPIO** – General-Purpose Input/Output
- **IaaS** – Infrastructure as a Service
- **IDS** – Intrusion Detection System
- **IoT** – Internet of Things
- **IP** – Internet Protocol
- **LED** – Light Emitting Diode
- **OS** – Operating System
- **PaaS** – Platform as a Service
- **PIR** – Passive Infrared
- **RAM** – Random Access Memory
- **RPi** – Raspberry Pi
- **SaaS** – Software as a Service
- **USB** – Universal Serial Bus
- **VM** – Virtual Machine

CHAPTER 1

INTRODUCTION

1.1 Background to the Study

The global agricultural sector is universally acknowledged to be at the heart of economic stability, food security and national development. In the developing countries, especially in Sub-Saharan Africa, agriculture is the major economic activity. For instance, Nigeria is home to a huge proportion of the rural population who have adopted subsistence farming, mechanised farming and rearing of livestock for their livelihood and the sector employs a considerable percentage of the workforce in the country (Majekodunmi et al., 2017). The agricultural landscape has undergone several industrializations in the last few decades around the world. It has moved from manual agriculture (Agriculture 1.0) and highly mechanized agriculture (Agriculture 2.0) to automated, sensor-driven agriculture (Agriculture 3.0). The current world is operating in the paradigm of "Agriculture 4.0", which is based on the Internet of Things (IoT), Cloud Computing, Edge Artificial Intelligence (AI), and autonomous robotics, with the aim of increasing crop yields, tracking supply chains and optimizing resource use (Zhai et al., 2021; ResearchGate, 2025).

The security of agricultural assets, however, is a challenge that has not been fully met in the history of agriculture, although significant technology and capital has been invested in precision farming, crop health monitoring and automated irrigation. Unauthorized man intrusions, rural banditry and organized livestock theft are serious and growing hazards to

agricultural perimeters. These security lapses have a significant socioeconomic effect and risk compromising food security and livelihoods. Even with the vast amounts spent on the nation's security, farmland in the countryside is still extremely prone to vulnerability because of the vastness and inaccessibility of its borders.

A field survey in 2025, conducted by EAS Publisher in Nigeria, showed that farmers' insecurity is mostly in the form of theft of farm produce and crops in North Central Nigeria, especially during the harvesting season when farm produces are temporarily left on the fields, affecting an alarming 88.9% of the farmers in the respondent’s areas. In recent years, Community of Agricultural Stakeholders of Nigeria (CASON) has conducted a series of studies on agriculture security, and it estimates that the loss of farm produce in Nigeria to organized criminals reaches up to 40% of the total harvest of the farmers, costing the agricultural sector over N 15 billion annually (The Nation, 2024). From a micro-economic perspective, individual farmers have often reported losses of over ₦2 million on their harvests in a single night, while the malicious attacks on farmlands have reported damages of up to ₦4 million per attack (Leadership News, 2024). Moreover, the high level of violence against farmers' personal security in the process of armed crop raiding and rural banditry has compelled thousands of farmers to abandon their farms, thereby causing a localized shortage of food and inflation rates (2019 WANEP, 2024 Oyelade et al.).

Traditional security measures for farmers include passive elements like brick walls, scarecrows, and physical fence, as well as manual guards. These techniques are very labour

intensive, too costly to be used on large farms in rural areas and inherently unsuccessful. These are modern-day intruders, not doing things by the seat of their pants. They increasingly are using tricks to hide their identity, mostly face masks or balaclavas or bandanas. The passive Closed-Circuit Television (CCTV) cameras are rendered useless through this specific tactic, and it makes law enforcement's post incident investigation and prosecution all but impossible.

The combination of IoT and Cloud Computing provides them a proactive and intelligent solution in this crisis. Edge computing nodes, like the Raspberry Pi, have been combined with state-of-the-art cloud-based cognitive services including Microsoft Azure AI Vision, which can now facilitate a self-learning system to detect intrusions and alert the system to specific individuals trying to cover their tracks by wearing a face mask (Microsoft Azure, 2024; NCBI, 2005; Vivek, 2023). In addition, the latest technical good practice requires not only being able to notice the presence of a masked intruder, but also to actively prevent it. The system must immediately produce unpredictable, but effective, active deterrence, such as flashing strobe lights and warbling sirens, only when an intrusion is detected by the system using AI, in order to effectively deter the intruder before farm assets are lost (Gilsdorf et al., 2003; CCTV Security Pros, 2025).

1.2 Statement of the Problem

Despite the critical necessity of securing agricultural assets, existing electronic security frameworks are fundamentally ill-suited for the unique, harsh realities of rural farm

environments. A critical analysis of the current landscape of farm security reveals three major technical and operational problems:

1. The Inadequacy of Passive Motion Sensors and Alert Fatigue:

The traditional motion-based security systems are based almost exclusively on Passive InfraRed (PIR) sensors or low-level microwave detectors. In an outdoor agricultural application, these sensors can be considered as ‘blind’. They produce vast numbers of false alarms caused by wind-blown leaves, a thermal gradient (sun coming through cloud), or benign wildlife (dogs or birds straying in the vicinity). The false-positive rate is very high and can cause farm owners to become alert fatigue, meaning they tune out the mobile alerts, turn off the alarms or disable the alarms altogether, leaving the farm completely exposed to real threats.

2. The Identity Concealment Dilemma and Edge Computing Limitations:

In the modern era, agricultural thieves, rural bandits and cattle rustlers use identity concealment techniques to avoid future prosecution. A standard localized surveillance system simply captures footage on a local hard drive, but will not "think" to identify a masked person as a critical threat in real time. An immediate challenge here is the deployment of complex Deep Convolutional Neural Networks (CNNs) for facial attribute extraction and mask detection directly onto low-cost, off-grid edge devices (e.g. a standard Raspberry Pi or microcontroller), which causes significant hardware constraints. These edge devices have little to no Random-Access Memory (RAM), no dedicated Graphics

Processing Units (GPUs), and are easily thermally throttled and battery drained when continually using computer vision inferences in the agricultural heat.

3. The Constraints of Cloud-Only Architectures in Rural Settings:

By contrast, fully cloud-based security systems (which upload 24/7 video streams to a remote cloud-based server for analysis) need high bandwidth, constant 4G/5G internet connectivity and stable grid power. These resources are seldom available or are very unpredictable in rural farm lands.

Thus, an urgent need to develop a Hybrid Cloud-Edge computing security architecture exists, which has yet to be met. This proposed system needs to fill the gap, first by relying on local ultra-low-power sensors for hardware wake-up, then by taking an image of a person in a specific manner, and finally by taking advantage of powerful cloud-based APIs – in this case, the Microsoft Azure Face API. This enables the system to automatically detect identity concealment (mask-wearing) in real-time with enterprise-grade accuracy. This way, power and cellular data are only being used when there is a verified human threat event.

4. Poor Human-Computer Interaction (HCI) and Lack of Centralized Control:

Existing agricultural security systems often fail at the user experience level. When an intrusion occurs, farmers receive an email or SMS. They lack a centralized User Interface or web dashboard or app to historically log these events, view/review the captured images of intruders, or manually override the system to trigger or cut off alarms before and after

suspicious activity. This lack of the User Interface affects the operational efficiency of the farm manager.

1.3 Aim and Objectives of the Study

The primary goal of this study is to design, implement and test an Intelligent Farm Intrusion Detection System (FAIDS) that uses a hybrid architecture of a Raspberry Pi edge node with a Microsoft Azure AI Vision service to automatically detect masked intruders and a dashboard that the user can view intruders.

To achieve this comprehensive aim, the specific objectives of the study are:

1. to develop an Edge IoT hardware node: Design a power efficient hybrid sensor design for an Edge IoT hardware node that is capable of efficiently detecting motion and capturing images of perimeter breaches using a Passive InfraRed (PIR) sensor (HC-SR501) and a high-resolution Pi Camera module attached to a Raspberry Pi 3 Model B;
2. to implement Cloud-based mask recognition: Use the Python SDK to integrate the Microsoft Azure Face API to analyse the captured images in real-time. This includes securely sending the payload over HTTPS, identifying the face's bounding boxes, and assessing the facial features to classify the intruder's face as either "masked" or "unmasked";
3. to engineer a power-optimized wake-up strategy: Utilize the PIR sensor as a low

power hardware interrupt to wake the camera and start the Azure API call to initiate video monitoring, thus helping to save the bandwidth of the cellular data and to reduce the power consumption required to continuously monitor video;

4. to deploy automated active deterrence: Develop an active deterrence backend code that will generate a ‘mask detected’ signal if the cloud server detects a payload on the mask, and according to this payload, energize GPIO pins to activate relay modules for physical countermeasures to avoid habituation to the mask.
5. to develop a remote notification subsystem: Develop a custom API integration module that interfaces directly with Twilio and the Telegram Bot API to push real-time visual evidence and send high-priority threat alerts to the farm owner's smartphone.
6. to design and integrate an interactive user interface: Develop a centralized web dashboard that provides the farm administrator with a secure portal to access real time logs, see a history of intruder images, manually trigger deterrence systems and monitor system health.

1.4 Significance of the Study

The complete development, deployment and documentation of this hybrid Cloud-Edge security system has a great impact in several academic and industrial areas:

1. Economic and Security Impact:

The system actively defends against intruders by explicitly addressing the problem of identity concealment (mask wearing) and by enforcing immediate active deterrence. This directly saves the farmer's produce, farm assets and livestock, thus helping to address the estimated loss of ₦15 Billion to the Nigerian agricultural sector annually and ensuring the financial sustainability of farmers in the rural communities.

2. Operational Efficiency (Elimination of Alert Fatigue):

The system verifies threat by transferring the image to Microsoft Azure's deep learning algorithms. This almost eliminates all false alarms on traditional PIR based farm alarm. When a masked human is present on the farm, farmers will only be notified in this case, making them more likely to trust automated security measures and ensuring the alarm sounds when an intruder is indeed detected.

3. Technological Innovation in Resource-Constrained Environments:

This project proves the realism of a "Hybrid Cloud-Edge" in a rural, off-grid environment. It is clear that low cost, low latency, on-premises hardware (such as the Raspberry Pi) can efficiently use enterprise-grade cloud AI (Azure Vision) without necessarily needing to continuously process high amounts of data or continuously stream the Internet. This seriously decreases the entry barrier for smart farming security.

1.5 Scope of the Study

This study only deals with the perimeter and access point surveillance for medium scale farmlands, storage silos, and livestock enclosures.

Technologically, the hardware scope is limited to the integration of a Raspberry Pi 3 Model B (localized Edge Gateway), 1080p standard Pi Cameras, standard PIR motion sensors and a relay driven active deterrence circuit.

The software intelligence and cognitive analysis are handed over to the Microsoft Azure AI Vision (Face API). The system is designed to identify human faces and treat the problem as a binary classification problem. Does not cover localized training of custom deep learning object detection models (such as YOLO, SSD or Faster R-CNN).

The project uses Telegram messaging platform public API for alerting and Twilio for SMS alerting. Last but not least, a fundamental premise and limitation of this scope is that the deployment site needs to have access to at least a basic and stable 3G/4G cellular network router, to enable outbound HTTPS requests towards the Azure servers and the Telegram ecosystem. If the entire network fails, the system will revert to a local offline motion alarm.

1.6 Areas of Application

The basic working principles and logic of the proposed hybrid Cloud-Edge security solution can be adapted to various architectures, but the solution is tailored to satisfy the

particular challenges and security risks of agricultural environments. Key areas of application include:

i. **Agricultural Sector:**

Livestock Enclosures and Kraals: Protecting high-value livestock (cattle, sheep, poultry) from targeted rustling, especially during nocturnal hours when thieves typically conceal their faces to avoid recognition by farmhands.

Farm Equipment Sheds & Silos: Securing expensive mechanized farming machinery, fertilizers, and harvested crop storage facilities from masked vandals and organized theft syndicates.

ii. **Commercial and Residential Security:**

Private Estates and Warehouses: Protecting commercial logistics hubs, gated communities, and private residences. Identifying masked individuals approaching a perimeter provides an early warning of premeditated burglaries before forced entry into a building actually occurs.

Bank ATMs and Vaults: Flagging individuals wearing full facial coverings approaching sensitive financial installations outside of standard health-mandated requirements.

iii. **Public Safety and Border Control:**

Rural Border Security: Monitoring vast, unpatrolled rural borders and illegal entry points where smugglers and intruders frequently wear masks to evade traditional facial recognition checkpoints and border patrol cameras.

iv. **Industrial Compliance and Occupational Health (Inverted Logic):**

The Azure mask-detection algorithm can be flipped and applied to industrial processing plants, chemical facilities or healthcare spaces. This time, the system can be used to ensure compliance instead, by sounding an alarm when the worker is not safely wearing the required protective face mask (mask), respirator or safety gear before entering a hazardous area or by denying access to the door to the worker if they are not wearing the mask.

1.7 Thesis Outline

This project report is arranged in five chapters which are designed as per the standard guidelines of undergraduate project report, to make the information logical and to record the process of research:

Chapter One: Introduction – presents the background of the study, outlines the transition towards Agriculture 4.0 and formulates the main statement of the problem on agricultural theft and concealment of identity. It sets the goal and specific objectives, explains the importance of the study, gives an overview of the scope of the design, and examines the general scope of application of the proposed technology.

Chapter Two: Literature Review – This chapter provides a critical examination of previous scholarly works, foundational theories, and existing systems. It reviews the evolution of agricultural security, compares Edge computing versus Cloud

computing paradigms, and explores the theoretical framework of facial attribute detection, specifically focusing on the mechanics of the Microsoft Azure Face API and the psychological impact of active deterrence techniques.

Chapter Three: Methodology and Materials Used – This chapter details the architectural framework used to develop the system. It outlines the proposed Hybrid Cloud-Edge architecture, provides detailed block diagrams and flowcharts illustrating the data flow between the Raspberry Pi and the Azure Cloud and the web dashboard, and details the specifications of the hardware components and software libraries utilized.

Chapter Four: System Testing, Results and Discussions – This chapter focuses on the practical realization of the project. It covers the physical circuit wiring, the Python codebase implementation, API key integration, and the setup of the Telegram Bot, User Interface. Furthermore, it presents empirical testing results, proves that the system correctly recognises masks under different lighting conditions, triggering the deterrence hardware, and evaluating API latency.

Chapter Five: Summary, Conclusion, and Recommendations – The final chapter synthesizes the research findings. It provides a summary of the project's achievements against its initial objectives, offers concluding remarks on the viability of Azure-backed farm security in rural Nigeria, and provides actionable recommendations for future enhancements, commercialization, and scalability.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter offers a complete and critical analysis of the literature, empirical studies and technical frameworks that are directly relevant to the design and implementation of a Farm Intrusion Detection System (FAIDS). The review aims to progressively establish a theoretical and technical background for the proposed system, by addressing the following thematic areas: Prior intrusion detection systems for agricultural environments using IoT; Motion detection technologies with particular interest in Passive Infrared (PIR) sensors; Computer vision and deep learning for object detection; Facial attribute detection and face mask recognition technologies; Capabilities and architecture of the Microsoft Azure Face API; Edge computing, cloud computing and hybrid architectures; The Raspberry Pi as an IoT edge node; The theoretical framework of active deterrence; Real-time mobile notification systems. The chapter ends with an identification of key research gaps that will be filled by the proposed FAIDS.

2.1.1 Hardware Context of the Proposed System

To give some background to the technical frameworks and literature surveyed in this chapter, it is necessary to briefly describe the target hardware architecture of the proposed FAIDS.

The edge computing layer is implemented on a Raspberry Pi 3b microcomputer, chosen for its low cost and on-board processing features. When motion is detected, the primary

sensory node collects visual data using HC-SR501 Passive Infrared (PIR) sensors in combination with standard USB cameras. Moreover, since there is no reliable grid electricity in remote agricultural environments, in the hardware design, an off-grid hybrid power unit is needed, including a serial connection of 3.7 V Li-ion battery cells, a 3S Battery Management System (BMS) for safety, and an LM2596 Buck Converter to reduce the voltage to a stable 5V rail for the logic components. These physical pieces lay the practical foundation for the edge-computing, computer vision and IoT connectivity concepts discussed in subsequent sections. Fig. 2.1 illustrates the physical layout of the Raspberry Pi 3 Model B used as the edge-computing node in this design, highlighting the GPIO header, USB and camera interfaces, and power input that support the sensor and camera integration described above.

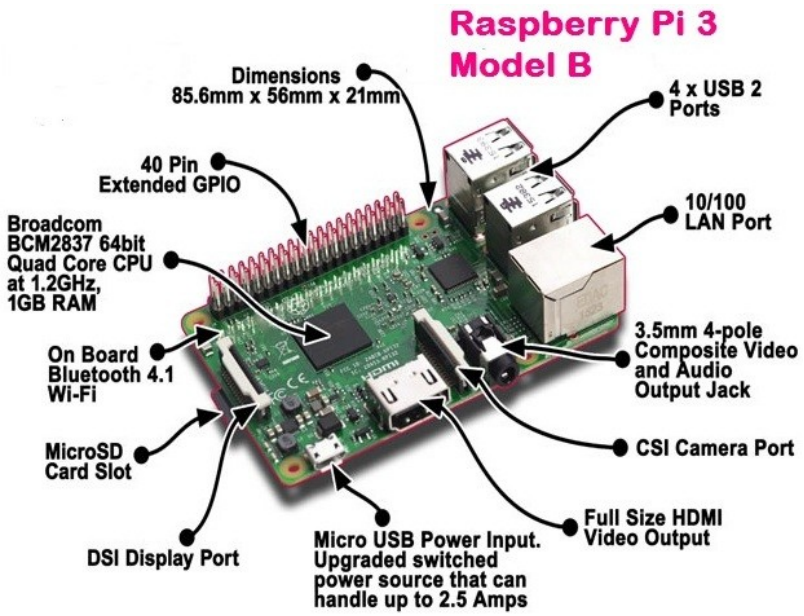


Figure 2.1: Labeled view of the Raspberry Pi 3B highlighting major component interfaces

2.2 IoT-Based Intrusion Detection Systems for Agricultural Environments

Many studies have appeared in recent years concerning the use of IoT concepts to detect intrusions in the agricultural domain, with the main attention being directed towards the detection of animal intrusion in agricultural fields in order to avoid crop damage. It is instructive to do a comparative analysis of this literature in order to understand the capabilities of the technology as well as the gaps that must be filled by an intrusion and identity-concealment focused system.

One of the most relevant work to this research was presented by Oyelade et al. (2024) which proposed intrusion detection model in farmland using IoT and computer vision techniques for the Nigerian agricultural context. They used PIR sensors and cameras strategically located around the farm in their system. When the PIR sensor detected

movement through body heat radiation, it caused the Raspberry Pi to take a photo of the subject, which was then passed along to an improved version of Faster Region-Based Convolutional Neural Network (Faster R-CNN) that was used to detect objects, and a one-shot learning algorithm which was used to identify the face. The result of this work is a confirmation of the basic feasibility of the PIR-trigger, camera capture, AI-analysis pipeline as an architecture for agricultural intrusion detection. One major drawback of their work was the effort to deploy complex deep learning models (Faster R-CNN) locally on edge devices with limited computation resources, which posed serious performance constraints.

Likewise, Boyinbode and Olanipekun (2023) designed an intrusion detection system for farmland based on the Internet of Things (IoT) to solve the false alarm problem of the simple PIR system. Their system implemented another step of image verification to minimize false positives. The two-stage detection architecture (hardware trigger and cognitive verification of image) is shown to be better than the single-stage PIR-only approach in agricultural scenarios, as a result of this work.

A wider class of systems have been directed towards animal intrusion detection, although these attempts to detect other classes of threat actors, they adopt an architecturally similar approach. A hybrid YOLOv5-SSD detection framework was suggested to protect durian plantations from animal intrusion and a Telegram messaging was introduced for farmer notification as a communication method in the proposed FAIDS, as presented in the research published in the Delwar et al. (2025). The study (Delwar et al., 2025) on real-time

farm surveillance by using IoT and YOLOv8 benchmarked the performance of the various deep learning architectures, including Inception, Xception, VGG16, AlexNet, and YoloV8, with YoloV8 delivering the best performance. This proves that computer vision technology has reached a stage where it can be used in agriculture for surveillance, but the high computational power requirements of such models make them unsuitable for low-cost, off-grid edge devices that do not include hardware accelerators.

Notably, the IoT-based Farmland Intrusion Detection and Alert System presented in Rohan and Chandan (2024) showcases the practical application and low cost of implementing the Telegram Bot API as a primary notification system for agricultural security events. The design of the notification subsystem for the proposed FAIDS can be directly informed by this finding. But these studies do not consider the unique situation of a human intruder wearing a mask, which is the most common threat actor in Nigerian agricultural banditry. The proposed FAIDS is a special one because it is specifically targeted at this threat.

2.3 Motion Detection Technologies: Passive Infrared (PIR) Sensors

The Passive Infrared (PIR) sensor, in particular the HC-SR501 sensor module used in this research, is the first line motion detection mechanism in the proposed FAIDS architecture. It is important to have a good knowledge of its working, its benefits and the known limitations in outdoors so as to rationalize the hybrid detection approach which is used.

The working principle of a PIR sensor is to detect any change in infrared radiation (heat) in its field of view. They are "passive" as they receive infrared radiation from warm body

sources, like humans and animals, rather than emitting any themselves. If a warm body passes over the zones of the sensor, there is a difference in the infrared radiation which produces an electrical signal that activates an output. The HC-SR501 PIR sensor also features extremely low power consumption, adjustable sensitivity range (3-7 meters approximately), a time delay which can also be adjusted, and a simple digital output (HIGH/LOW) which makes the sensor a perfect hardware interrupt trigger for a more power consuming computing node, like the Raspberry Pi.

There is a large amount of IoT security research that underpins the PIR sensor's widespread use as the most common motion detection method for low-cost surveillance systems. As part of this study, several works used the PIR sensor as the main motion-causing component of a Raspberry Pi and camera system, such as the Smart Surveillance Monitoring System developed and published by the researchers on ResearchGate. The systems are based on the same architectural pattern which is: PIR detection activates camera capturing, which is sent to the user by e-mail, SMS or API for instant messaging.

But the major drawback PIR sensors have in outdoor agricultural applications is the tendency to trigger falsely. False positive outputs are produced regularly by environmental factors such as wind-blown foliage, cloud cover changes, reflected heat from sunlit surfaces and small non-threatening animals moving. This is a known fact in literature as one of the major reasons for the occurrence of alert fatigue in farm security deployments (Boyinbode & Olanipekun, 2023). To counteract this one element of the threat, this research proposes a design for the PIR sensor that performs a wake-up hardware interrupt

only when a human face is masked, shifting all threat verification and classification tasks to a cloud-based service, the Microsoft Azure Face API.

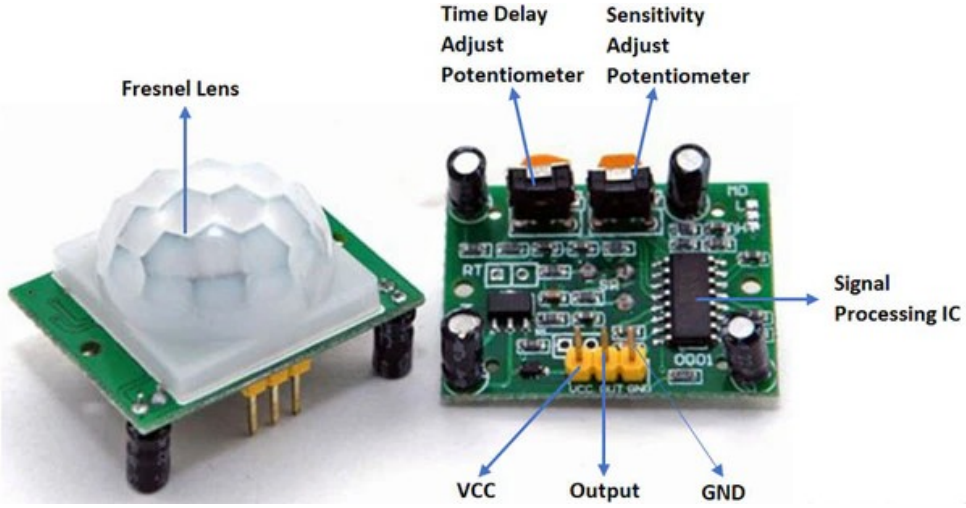


Figure 2.2: Labeled view of the Passive InfraRed (PIR) Sensor Module (HC-SR501)

2.4 Deep Learning for Threat Classification and Personnel Authorization

The intelligence of an FAIDS is the ability to read visual information. It consists of two separate tasks: Object Detection (identifying the target as Human, Animal, or Vehicle) and Authorization (identifying whether a detected human should be allowed to be there).

2.4.1 Evolution of Detection Algorithms (YOLO vs. R-CNN)

Initial applications of agricultural vision systems used two-stage detectors such as Faster R-CNN. Although these models are very accurate with small objects in complicated backgrounds, their computational intensity leads to high inference latency. According to Oyelade et al. (2024), two-stage detectors usually have lower frame rates than active deterrence on edge devices.

This has led to studies on single-stage detectors, namely the YOLO (You Only Look Once) family. In contrast to R-CNN, which first proposes regions and then classifies them, YOLO poses detection as a single regression task, where a bounding box and a class probability are predicted together. Recent benchmarks by Delwar et al. (2025) indicate that YOLOv8 is a better option in an agricultural setting, with mean Average Precision (mAP) scores over 99% on particular animal datasets because of its anchor-free design.

Table 2.1: Comparative Analysis of Deep Learning Object Detection Models

Model Architecture	Type	Pros for Agriculture	Cons for Agriculture	Recommended Use Case
YOLOv8	Single-Stage (Anchor-free)	**Highest Accuracy:** Achieves >99% mAP on agricultural datasets (Delwar et al., 2025).	**Computation:** Slightly higher load than v5; requires modern PyTorch environment.	Precision surveillance on Raspberry Pi 4/5 or Jetson for species classification.
YOLOv5	Single-Stage (Anchor-based)	**Stability:** Vast community support. **Speed:** Extremely fast inference on edge CPUs.	**Tuning:** Requires manual anchor box tuning for animals with unusual aspect ratios (e.g., snakes).	General monitoring on older hardware or battery-critical nodes.
Faster R-CNN	Two-Stage	**Small Objects:** Superior detection of small pests or distant intruders (Mou et al., 2023).	**Latency:** Very slow inference (<1 FPS on Raspberry Pi); unsuited for active deterrence.	Offline analysis or drone imagery processing where real-time response isn't critical.

2.4.2 Authorized Personnel Detection Strategies

Another fatal weakness of early intrusion systems was that many false alarms were caused by friendly fire by farmers and workers. In this regard, the contemporary architectures have adopted the Whitelisting strategies. Although Facial Recognition is not a novel technique of biometric identification, modern literature suggests a variety of different techniques such as token-based authentication (RFID) or geofencing authentication (Location-based).

Table 2.2: Comparative Analysis of Authorized Personnel Detection Methods

Method	Technology / Principle	Advantages in Agriculture	Limitations
Facial Recognition	**Biometric:** Uses CNNs to match live video faces against database.	**Passive:** Hands free; requires no tokens/phones, High security.	**Privacy:** GDPR/data concerns. **Environmental:** Fails in low light or if the farmer wears a mask/hat.
RFID / NFC Tags	**Token-based:** Farmers carry a tag or card to be read by a scanner at the gate.	**Reliability:** 99.9% accuracy; unaffected by weather/light. Low power consumption.	**Active Action:** Requires user to "scan in." **Theft:** Tags can be stolen or lost.
Geofencing (GPS)	**Location-based:** Virtual perimeter defined by GPS coordinates. App triggers "Safe" status when phone is inside.	**Automation:** Hands-free; works for large, undefined perimeters. Can track fleet/machinery.	**Power:** Drains smartphone battery. **Drift:** GPS error (±5m) may cause false alarms near boundaries.
Schedule-Based	**Rule-based:** System automatically disarms during specific "working hours" (e.g., 08:00 - 17:00).	**Simplicity:** No hardware/sensors required. Zero cost implementation.	**Inflexible:** Cannot distinguish between a thief and a farmer during "working hours."

2.5 Face Mask Detection Technologies

The most remarkable research effort has been devoted to the problem of detecting if a human subject is wearing or not wearing a facial mask, after the global COVID-19 pandemic, with a large number of papers dealing with different methodological solutions for binary classification of masks. This research builds on this academic momentum to tackle a different but technologically similar problem; identity concealment by an intruder.

2.5.1 Approaches to Face Mask Detection

According to a systematic review published by NCBI (2025), the existing face mask detection methods can be classified into three main categories: feature-extraction-and-classification-based approaches, object-detection-model-based methods, and multi-sensor-fusion-based methods. The first methods were also feature-based approaches like the Histogram of Oriented Gradients (HOG) with Support Vector Machines (SVM) but are not widely used anymore because they are not very generalizable to different lighting conditions, masks and facial angles.

Mask detection systems based on deep learning have obtained outstanding accuracy results. The use of the two deep learning architectures (DCNN - custom deep CNN, and MobileNetV2 through transfer learning) to achieve an overall accuracy of 98.7% was shown on unseen test data for binary mask/unmasked classification in a study published in Wireless Communications and Mobile Computing (Hussain et al., 2022). Research by

Militante and Dionisio reported in the same study built an automatic mask detection system with an alarm when a mask is not detected using VGG-16 with a high detection accuracy of 96%. They are mostly preserved in the literature, and a survey conducted by Hosny et al. (2024) on the field of "Artificial intelligence-based masked face detection" shows that CNN-based methods have consistently improved over the years.

2.5.2 Challenges in Mask Detection Under Real-World Conditions

Although well-done face mask detection system performance is achieved under controlled test conditions, the literature shows that there are several key challenges to implementing face mask detection systems in practical, uncontrolled environments similar to agricultural environments. According to a comprehensive overview in Delwar et al. (2025), the main challenges to consider are light changes (during the night, out of the light), face occlusion (mask's coverage is partial), diversity of mask types (surgical masks, N95 respirators, cloth masks, balaclava), face pose and face angle (angle of the face and type of pose), and motion blur (with respect to the moving subjects). These are especially challenging in a typical outdoor agricultural surveillance situation, where lighting conditions are unpredictable, the subject can be at different distances and angles, and the camera may be vibrating, potentially affecting image quality.

The review published in Delwar et al. (2025) also reported the recent development of an “Occlusion-Aware Module Network” (OAM-Net) approach, which uses two sub-networks: occlusion-aware sub-network and key region-aware sub-network, for tackling

mask-based occlusion challenges in recognition tasks. Such architectures are important to the academic community, but require significant computing power and are impractical for a Raspberry Pi without cloud offloading. It can be interpreted as a fundamental design decision of the proposed FAIDS to use a cloud-based API for detecting the mask, instead of trying to do it locally.

2.6 Cloud-Based Cognitive AI Services: Microsoft Azure Face API

Microsoft Azure Face API is a part of the Azure AI Vision suite (previously Azure Cognitive Services) and is the cloud intelligence engine for the proposed FAIDS. The architecture, capabilities, evolution and the documented performance characteristics of the system, need to be carefully examined in order to justify its selection over the alternatives.

2.6.1 Architecture and Key Features

Azure Face API is an HTTPS-accessible, RESTful, cloud-based service that allows developers to harness enterprise-level facial analysis technology without having to train or host a complex deep learning model. It currently provides several core functionalities according to Microsoft Azure (2024):Face detection-Identify the locations of faces within an image by returning the coordinate of the bounding box of each face found; Facial attribute analysis-Detect whether there is a mask, presence of glasses, hair colour, estimation of age and head pose etc.; Face verification-Determining whether two detected

faces belong to the same person; and Face identification within a group of enrolled people.

With regards to the proposed FAIDS, Microsoft has updated the API to include "faceMask" attribute in the Detection 03 model, which detects whether a mask is worn on the face, and, if so, whether the mask is covering nose and mouth according to "noseAndMouthCovered" attribute (Microsoft Learn, 2024). This provides binary classification output with a confidence value which is the very attribute needed for the FAIDS threat classification engine. Microsoft Learn also confirmed that the current (as of the review) Recognition 04 model includes "improved recognition for users wearing face covers (surgical masks, N95 masks, cloth masks)" (Microsoft Learn, 2024), indicating that this has been a deliberate addition.

2.6.2 Accuracy, Limitations, and Responsible AI Considerations

When it comes to accuracy, Microsoft's documentations on the characteristics and limitations of the Azure Face API is very direct about the variability: "it is not possible to provide a universally applicable estimate of accuracy for the actual system you are planning to deploy" and factors include camera technology, camera setup, how the user interacts with the camera and interpretation of results (Microsoft Learn, 2024). This realistic assertion highlights the need to test empirically, the proposed FAIDS in its actual operating environment (low-light, outdoor agricultural settings).

There was a major change regarding the ethical aspect of Azure Face API in June 2022,

when Microsoft withdrew access to certain facial attributes (including emotion, gender, age and others) due to privacy and bias concerns (AitoCore, 2026). This evolution toward a responsible AI strategy is reflected across the industry and relevant, although not central to the proposed FAIDS. A review published on LetsAI.tech (2024) evaluates the Azure Face API on accuracy and real-world performance and acknowledges its high-fidelity detection performance but stresses the influence of the same key parameters on the output.

2.6.3 Prior Research Employing Azure Face API

Use of the Azure Face API has been found in several practical research areas. Vivek (2023) implemented an attendance system with facial mask detection using the Azure Face API and its Python SDK and confirmed the possibility of real-time mask classification via HTTPS. His design captured faces, passed to Azure via API call, and parsed the JSON response; identical to the structure of the proposed FAIDS. The concept of automated face mask detection using automatic face recognition had its theoretical origins as early as in 2005 with studies aiming for crime prevention and security as discussed in NCBI (2005).

2.7 Edge Computing, Cloud Computing, and Hybrid Architectures for IoT Security

At the core of any IoT-based surveillance system is the decision of where computational workloads should be distributed between local edge nodes and remote cloud servers. This section critically discusses the benefits of each paradigm, as well as the increasingly accepted consensus in favour of a hybrid architecture for resource-constrained, latency-

sensitive IoT applications.

2.7.1 Cloud Computing in IoT Contexts

Cloud computing provides immense, centralized computational resources accessible over the internet. As such it has obvious benefits for large-scale data storage, training large models and easily accessible, widespread analytics. In an IoT security context, cloud-first architectures transmit continuous video streams or frequent images from edge nodes to the cloud for processing via powerful, GPU-accelerated deep learning models. The findings of a systematic review by Andriulo et al. (2024) supported this, stating that 'cloud computing enables greater scalability and flexibility' for the IoT. In rural Nigeria, fully cloud-based architectures suffer fatal limitations: high bandwidth dependency, unavoidable network latency, and total failure during rampant internet outages, from event detection to alert generation, and ceasing to function during internet outages, which are, as per the scope of this research, rampant on rural Nigerian farms.

2.7.2 Edge Computing in IoT Contexts

Edge computing involves placing computation at the physical edge of the network-near where the data is being generated. This allows for reduced latency, and conserves bandwidth and power by transmitting only meaningful, actionable data to distant servers and allowing local processing while the system is disconnected. As per Synaptics (2023) in an industry review, 'edge computing processing sensitive data near the device' and the

benefits that 'the synergy between IoT and edge computing can help lower latency, improve efficiency and strengthen security'. A key limitation of edge-only architectures is that of the finite processing power of low-cost edge devices. Low-power and energy-efficient edge nodes such as the Raspberry Pi have insufficient GPU resources to support continuous surveillance in a real-time mode without thermal throttling and consuming excessive power.

2.7.3 The Hybrid Cloud-Edge Architecture: The Optimal Paradigm

The literature has converged strongly on the assertion that the hybrid model combining the real-time responsiveness of edge computing with the computational muscle of cloud AI is the optimal architectural paradigm for intelligent IoT security systems for use in constrained environments. Robustel (2025) aptly summarized this, claiming that 'the most powerful, efficient, and scalable IoT architecture is the hybrid model utilizing both the edge and the cloud for their specific capabilities'. In the hybrid model, the edge handles time-critical operations such as hardware wake-up and PIR triggered local control, while the cloud deals with the computationally intensive image analysis and deep learning inference to classify threats. Negi (2025) confirmed the effectiveness of the hybrid architecture stating that latency "drops from hundreds of milliseconds to single digits", only sending discreet image files to the cloud, not continuous video streams, which "provides significant improvement over centralized architectures, especially when it comes to speed and efficiency, as it reduces the burden on the network infrastructure and allows for quicker

response times, especially in scenarios where speed is critical". Andriulo et al. (2024) also found that 'hybrid approaches such as fog and mist computing offer an effective compromise by leveraging the advantages of both edge and cloud platforms' stating that "these hybrid approaches enable better bandwidth efficiency and are useful for latency-sensitive and privacy-focused IoT applications." This research's FAIDS directly benefits from the hybrid approach, using a low-power PIR-based hardware detection and relay control system on the edge (Raspberry Pi) with mask detection occurring in the cloud via Microsoft Azure Face API; this transmits single image files, rather than continuous video, upon trigger.

2.8 The Raspberry Pi as an IoT Edge Computing Node

The utilization of Raspberry Pi as the primary controller of IoT based security/surveillance system has also been extensively studied. A research paper from Chaichana and Netinant (2023), "Design and Implementation of an IoT-Based Surveillance System using Raspberry Pi, Camera, and Motion Sensor" has developed a system which is composed of the Raspberry Pi, PIR sensor and a webcam programmed using Python, and implemented OpenCV library. Faces and motions are detected and alerts are sent through LINE Notify, showing its capability to handle advanced multi-sensor integrations and API communication. Based on research papers analysing smart surveillance monitoring systems, the Raspberry Pi, when programmed using Python, "can run and control motion detectors and video cameras for remote sensing and surveillance, stream live video and

record it for later viewing" and can warn its owner "about the possible intrusion having a smartphone" via web application and messaging API.

The importance of the General Purpose Input/Output (GPIO) pins of the Raspberry Pi to the proposed FAIDS is to be emphasized. The GPIO header can enable direct interface with the HC-SR501 PIR sensor (for input), and, importantly, with the relay modules to control the 12V siren and other circuits (for output). This hardware control, as well as a vast library system of Python such as azure-ai-vision SDK and python-telegram-bot SDK, renders Raspberry Pi as the appropriate architecture of edge gateway for this FAIDS.

2.9 Active Deterrence: Psychological and Technical Frameworks

An integrated feature that distinguishes the proposed FAIDS from many current surveillance-only systems is the automatic active physical deterrence that activates after detection has been verified. This section reviews the literature and theory surrounding active deterrence as a security approach and specifically the use of strobe lights and audible sirens as the means of deterrent countermeasures.

2.9.1 The Psychology of Deterrence and Habituation

The underlying theory for using active deterrence as a security mechanism is founded on behavioural psychology: conditioned avoidance and stimulus habituation. In a seminal work that examines the use of frightening devices in wildlife damage control, Gilsdorf, Hygnstrom, and VerCauteren (2003) found that while audible and visual deterrents will

initially chase off an intruder, consistent use without actual harm will result in a learned "habituation" in which the intruder learns to recognize the stimulus as posing no immediate danger and ignores it. These authors established that an increased effectiveness in deterrent devices is directly proportional to stimulus unpredictability: variable timing, intensity, and type of stimuli will decrease rapid habituation, which is detrimental to passive, static alarm systems. This conclusion directly affects the implementation of the proposed FAIDS; by providing deterrents only after an AI-verified threat detection (as opposed to all PIR motion), the system can be assumed to trigger the deterrent when there is actual danger, thus providing increased psychological impact and unpredictability with every activation.

2.9.2 Strobe Lights and Audible Sirens as Deterrent Mechanisms

The use of audible sirens and strobe lights for active deterrence in security applications has significant precedence in the commercial and technical literature. For example, documentation by CCTV Security Pros (2025) suggests that active deterrence cameras "activate a loud siren with flashing light or spotlight when motion is activated to deter any potential threats," also reporting that "an ear-piercing siren can scare off even the most determined criminals" and that the addition of motion-activated strobe lights with high decibel sounds created a more effective "multi-sensory alarm."

More specifically, a brochure from Athenalarm (2025) states that modern strobe sirens emit 110-130 dB of sound pressure, capable of overwhelming background noise, and that strobe lights "disrupt an intruder's focus" and impede their activities. The brochure also explains

the psychological factor that the perceived escalation in danger when a strobe siren goes off increases the intruder's risk of being apprehended by law enforcement. This instantaneous and unexpected multisensory deterrent that simulates an imminent response is anticipated to be particularly effective in rural, poorly monitored environments where trespassers typically assume the risk of detection is minimal.

2.10 Real-Time Notification Systems: The Telegram Bot API

The effectiveness of the Telegram Bot API as a low-latency notification platform has been corroborated by several studies on the IoT security domain. One such study "Low-Cost Home Security Notification System Using IoT and Telegram Bot" (ResearchGate, 2022) implemented a system that uses Raspberry Pi Zero, a PIR sensor, and a Pi Camera to send an alert notification message to the house owner’s mobile phone using the Telegram Bot. The system was experimented and verified for the range of sensors detecting an intruder, the time the system responded to detect and send the alert notification message, and user satisfaction; it was confirmed that "the system was developed and works properly to detect and send the alert notification to the smartphone." With tested values for response time, it was proved that the notification channel is applicable in real time-like security systems. The same, with a hybrid YOLOv5-SSD based animal detection system using Telegram notification was implemented and published in MDPI where Telegram's feature of "delivering timely alerts to farmers by removing their dependence on infrequently used platforms" was acknowledged; further it ensured "accessible notification" by allowing

them to use it from their devices they already possess and use daily. Another study of IoT based Emergency Alert system with a Telegram Bot which was implemented for the IEEE International Conference on Automatic Control and Intelligent Systems, Abu.Zaid et al. (2023) stated that "Telegram bots are designed for users to interact with through predefined message commands"; that the Telegram Bots "capable of providing various information such as notifications." A comparative advantage of the Telegram Bot API over SMS based notification (the one using the old GSM module as used in many agriculture related IoT papers) is its capacity to send a high-resolution picture and text together at no cost per message, sharing the same cell phone data connection as the Azure API calls. This capability of multi-modal data transmission, that is, transmitting the photograph of the masked intruder along with timestamp information to law enforcement officials is essential.

2.11 Review of Related Systems: Comparative Analysis and Research Gaps

After analyzing the theoretical and technical underpinnings of each component subsystem this section combines these observations in order to compare the proposed FAIDS with the most relevant prior systems and explicitly delineate the research gap this project seeks to address.

The most directly similar systems to those proposed in this work found in literature were the PIR-Raspberry Pi-camera-CNN systems proposed by Oyelade et al. (2024) and Boyinbode and Olanipekun (2023) targeting Nigerian farmland security. These works

validate the foundational architectural feasibility of the system proposed, however they are limited by a number of critical features: first, by relying on locally deployed CNN inference (Faster R-CNN, one-shot learning), which produce an untenable hardware bottleneck in terms of computation load in Raspberry Pi nodes; second, neither system tackles the unique challenges posed by identity masking and facial masks-the most predominant behavioural trait of significant agricultural criminal elements; third, they do not include automated active physical deterrence and remain purely passive surveillance/notification systems requiring a human to react to the notification.

Animal intrusion detection systems although technically innovative using the YOLOv8 and MobileNetV2/hybrid CNNs address an entirely different threat model and so cannot be compared directly to the proposed human-identity-masking system. Cloud-based face recognition systems used in general commercial security applications require a constant connection and continuous stream of images from the edge to the cloud rather than the intermittent, image-on-event model necessitated by solar power constraints in rural Nigeria and the use of cellular data.

The unique combination of features present in the proposed FAIDS-PIR-triggered, image-on-event architecture; cloud-based facial mask detection using Azure Face API; Raspberry Pi edge gateway; GPIO controlled active physical deterrence; and Telegram Bot based mobile notification with Admin dashboard do not appear combined into one system in the reviewed literature. Thus, the research gap addressed by this project can be defined as: The absence of a validated, low-cost, low-power hybrid Cloud-Edge agricultural intrusion

detection system that explicitly prioritizes identity-masking using facial mask detection and employs active physical deterrence to address the threat model in resource-constrained, off-grid agricultural environments.

2.12 Theoretical Framework

Three theoretical approaches drawn from the literature form the design basis of the proposed FAIDS.

First, the Hybrid Cloud-Edge Computing Architecture Theory as presented by Andriulo et al. (2024) and a plethora of other industrial sources, forms the architectural model. This theoretical framework, stating that "best practice IoT security systems in resource constrained environments should delegate computation based on sensitivity to latency and/or computational intensity-low-latency, real-time, edge tasks (PIR interrupt management and relay activation) while computationally intensive, high-accuracy cognitive tasks (image analysis and mask detection) should be pushed to the cloud, resulting in both accuracy and ability to handle computations beyond the capability of the edge hardware." provides the foundational architecture of the system, where it can be observed that there are both computationally intensive and real-time tasks being performed.

Second, the Two-Stage Threat Verification Architecture, implicitly confirmed by Boyinbode and Olanipekun (2023) as well as the general body of IoT surveillance literature, forms the operational model of the system. As a single-stage PIR-only alert

provides too many false positives, it must be accompanied by a second cognitive analysis that either confirms or denies the threat before an alert or deterrent signal is issued. The proposed FAIDS performs this exact function, where Stage 1 consists of the PIR interrupt while Stage 2 consists of the Azure Face API.

Third, the Behavioural Deterrence Theory by Gilsdorf et al. (2003), concerning the design of security systems, justifies the proposed active physical deterrent subsystem. The theory proposes that "a more effective approach than simple continuous threat presence and/or predictable intervention is the incorporation of elements of unpredictability and multi-sensory response. To effectively deter a trespasser, the deterrent response should be correlated with verified threat and not necessarily every motion event." Therefore, the strobe and siren would only be activated in the proposed system, and only when a confirmed threat (masked intruder) is detected by Azure, in order to instil the most significant psychological impact.

CHAPTER 3

METHODOLOGY AND MATERIALS USED

3.1 INTRODUCTION

This chapter covers the processes, methods and techniques employed in the construction and design of the Farm Intrusion Detection System described in the preceding chapters. It explains, with acute detail, the overall system architecture (Section 3.1), the selection and configuration of the hardware components based off the circuit design (Section 3.2), the software nominated for image capturing and communication between the edge device and cloud, and the method/ model employed in the training, identification and classification of the nature of intrusion (Section 3.3).

Section 3.4 covers the solutions employed to curb the interconnectivity gap as discussed in Chapter 2. Section 3.5 covers the methods deployed in carrying out Key Performance Indicators metric tests on the components of the entire system, whose results will be further discussed in Chapter 4.

3.2 SYSTEM OVERVIEW & ARCHITECTURE

The design of the Farm Intrusion Detection System established in the Literature Review chapter consists of three layers, namely: The Edge Device Layer (Raspberry Pi 3b, PIR

sensors and cameras) responsible for motion sensing, local image capturing & data acquisition, and physical notification activation; a Backend Layer responsible for seamless and secure data transfer from the edge device to the cloud, and also the intelligent processing, identification & classification of visual data, storage and initiation of trigger alerts. The Frontend Layer is responsible for the seamless and real-time notification display to the farmer via the notification dashboard and email.

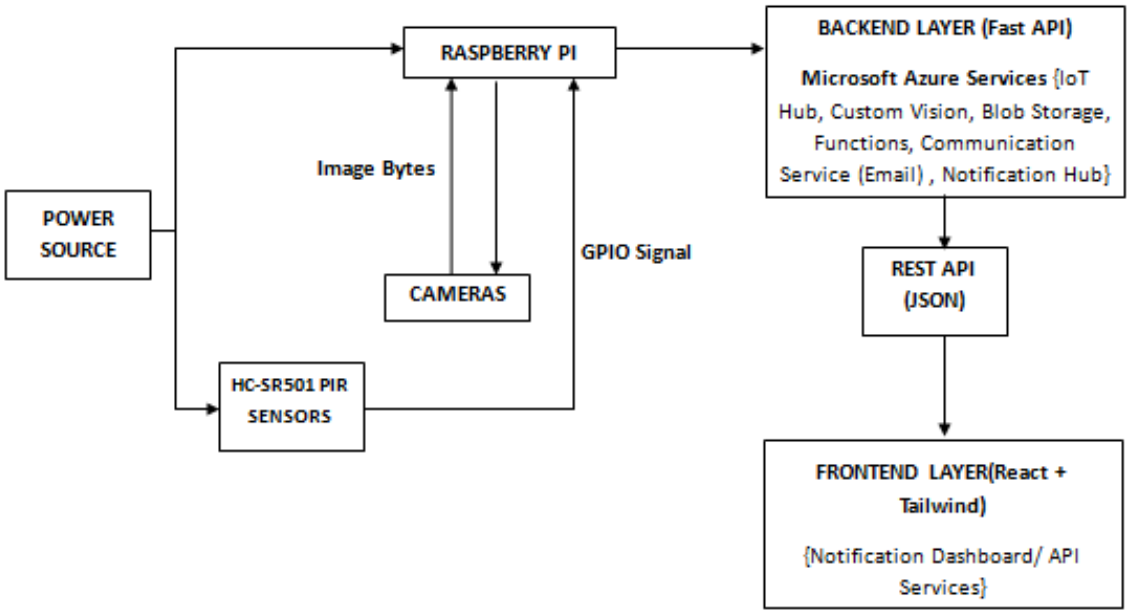


Figure 3.1: Block Diagram of a Farm Intrusion Detection System Using Computer Vision and Cloud Computing

3.2.1 The Edge Device Layer

This layer consists of HC-SR501 PIR sensors (for motion detection) and USB cameras mounted at strategic locations which will serve as the primary sensor node. A python end source program which is embedded in the Raspberry Pi 3b manages the edge device by triggering the capturing of an image only when motion is detected.

3.2.2 The Backend Layer

The Backend Layer is responsible for how captured images are transferred to the cloud via a secure HTTP request and/or the Azure IoT Hub. It is basically the system's computational and communication layer, sandwiched between the edge device and the cloud-based infrastructure. Upon storage of the buffered image locally, this layer transfers the captured image to Microsoft Azure Cloud Service via a secure HTTPS channel. The intelligence compartment of the Backend hosts the Azure AI vision model, which is at first trained using transfer learning/ inference to identify and classify the nature of intrusion as either a masked or unmasked intrusion (i.e., non-threat). The result of the image identification/ classification is transferred alongside the original image to the Microsoft Azure Blob Storage and a database for record keeping. A function embedded in the cloud architecture then orchestrates the forwarding of a notification alert to the user/ farmer's mobile dashboard & email.

This system also implements a direct design response to one of the identified research gaps stated in Chapter 2 which talks about the high rate of false-alarms and system's reliability by integrating a parallel fail-proof path that allows the edge device to trigger the system's

physical indicator (an LED) upon the occurrence of a high confidence score level of intrusion, and this action is totally independent of the round trip time it takes for the captured images to get transferred to the cloud architecture for processing

3.2.3 The Frontend Layer

The Frontend Layer consists of a Graphical User Interface (GUI) which displays real-time information/ notifications to the farmer via email & the admin dashboard. The notification dashboard displays system status (online/offline), configurable alert channels, recent event logs, timestamps, thumbnail images with their respective performance metrics (e.g., Accuracy, Precision, Recall, etc), providing the farmer with visual evidence that enables intelligent decision-making without the requirement of a physical presence on the farm.

3.3 HARDWARE DESIGN

The hardware design solely focuses on the achievements of one of the highlighted objectives of this study in Chapter 1: the integration of a Raspberry Pi 3b and sensory nodes (consisting of USB cameras and HC-SR501 PIR sensors) with a hybrid power source. The hardware design is further categorized into four subsystems, consistent with the KPI framework developed for this study which will be discussed further in Section 3.6.

3.3.1 Circuit Diagram

The schematic diagram illustrated in Fig. 3.2 shows the implementation of the edge device layer (hardware design) of the Intrusion Detection System. The components of this design include:

1. A power switch.
2. An IP2326 Power Delivery (PD) charging module.
3. 3 units of serially connected 3.7V Li-ion battery cells.
4. A Battery Management System, BMS-40A-3S module.
5. An LM2596 Buck Converter module.
6. A Raspberry Pi 3b microcomputer.
7. 2 USB cameras.
8. 2 HC-SR501 PIR sensors.
9. 1 LED to serve as an indicator.
10. A 300-ohm current-limiting resistor.

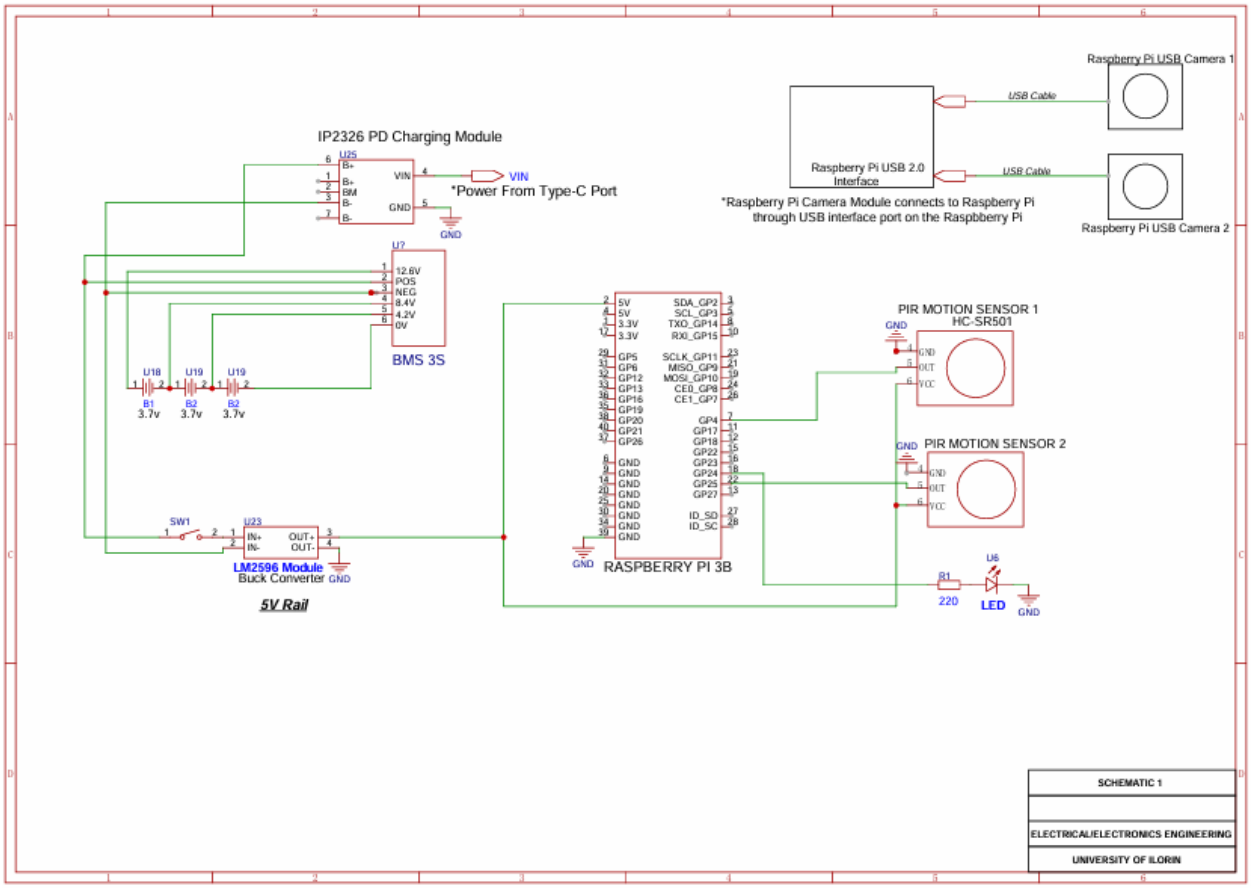


Figure 3.2: Circuit Diagram of a Farm Intrusion Detection System Using Computer Vision and Cloud Computing

3.3.2 Power Unit

This design comprises the edge device layer (hardware component) of the IDS system. It is powered by a hybrid power source, consisting of a charging module, a BMS system and rechargeable batteries.

3.3.2.1 5V Logic Budget

The node operates in two distinct states: **Idle** (polling sensors) and **Active** (motion detected, capturing, uploading).

Table 3.1: Power Consumption Budget of the Edge Device Layer at a 5V Logic Rail

S/N	Component	Idle State (5V)	Active State (5V)
1	Pi 3B (Lite, Headless)	~230 mA	~500 mA (CPU spike + Wi-Fi)
2	2x USB Cams	~100 mA	~400 mA (Active stream)
3	2x PIR Sensors	~2 mA	~2 mA
4	1x LED Indicator	0 mA	~20 mA
	Total Current	**~332 mA**	**~922 mA**
	Total Logic Power	**1.66 W**	**4.61 W**

3.3.2.2 11.1V Battery & Buck Conversion Math

The LM2596 is not 100% efficient. When dropping 11.1V down to 5V under a moderate load, it operates at roughly 80% efficiency (Eff = 0.8). The remaining 20% is lost as heat. To find the true power drawn from the battery pack:

$$P_{battery} = P_{logic} / Eff_{buck} \quad (3.1)$$

- **Idle Power Draw:** 1.66 W / 0.8 = 2.08 W

- **Active Power Draw:** 4.61 W / 0.8 = 5.76 W

Next, we calculate the total energy capacity of the 3S pack:

Given the underlying issue discussed in Chapter 1 of unreliable power supply and frequent interruptions on Nigerian farmlands, the Power Unit is designed around a rechargeable 4.2V (nominal voltage of ~ 3.7V) 3,000mAh battery bank, eliminating total reliance on electricity supply from the grid. 3 Li-ion battery cells are configured in a 3S (series) arrangement, yielding a nominal gross voltage of 11.1V and an energy capacity of:

$$\mathbf{E_{total} = V_{nom} \times C} \quad 3.2) \qquad (3.2)$$

$$\mathbf{E_{total}} = (3.7\ \text{V} \times 3) \times 3000\ \text{mAh} = \mathbf{33.3\ Wh}$$

The system's power source is controlled by a power switch to toggle the circuit's supply power (i.e., ON or OFF). A PD module (IP2326) is used to receive power from a Type-C port (AC source) for recharging the batteries, satisfying the objective requirement in Chapter 1 of a 'flexible power source'. A Battery Management System, BMS-40A-3S module, is introduced to handle the secure management of the batteries that powers the Raspberry Pi 3b, cameras and the PIR sensors in terms of protection against overcharge, discharge and short circuit conditions. The circuitry leads to the earlier mentioned Power Toggle Switch, and then an LM2596 Buck Converter module. It handles the stepping down

of the supply voltage (AC/ DC source) from the pack voltage of 11.1V to a stable 5V rail supply for the system's electronic components.

3.3.2.3 The Runtime Reality

Because this is a monitoring node, it will spend >99% of its life in the idle state waiting for motion.

Theoretical Max Runtime = 33.3 Wh / 2.08W ≈ **16 hours**.

In practice, the BMS will likely cut power around the 14-hour mark to prevent the Li-ion cells from dropping below their safe discharge threshold. The core challenge here is that the Pi 3B lacks a true deep-sleep state. Even running completely headless with the Wi-Fi powered down, the SoC continually draws heavy quiescent current while polling the GPIO pins.

3.3.3 Central Processing & Storage Unit

The most important component of the edge device layer is the Raspberry Pi 3b, which serves as the main computational brain of the system. Fig. 3.2 shows the details of the various pinouts which includes the GPIO (General Processing Input/ Output) pins, 5V and 3V power pins, and SDA, SCL, TXO, RXI, MISO, MOSI and SCLK communication pins. The choice of selection of the Raspberry Pi is based on the balance of its onboard

processing abilities, indicator interfacing and a local USB camera support. Cost-wise, this satisfies one of the Objective (i) stated in Chapter 1 which involves the integration of a Raspberry Pi 3b, a low-cost single board computer, with a flexible power source, making the system suitable for small and medium-scale farmers.

The sensors are in a constant monitoring state of the farmland perimeters, waiting to detect a thermal signature (i.e., motion). Upon a positive trigger, the Raspberry Pi 3b wakes up from a low-quiescent power state and triggers the cameras attached to the sensor which detected the motion to take a picture. The captured data frame is temporarily written and stored to a local store-and-forward buffer on a micro SD card, an intelligent mechanism which serves as a solution to the underlying issue of rural connectivity, a recurring limitation since the advent of intelligent Intrusion Detection Systems as discussed in the Literature Review chapter (Olatinwo et al., 2025).

3.3.4 Sensory & Visual Unit

This consists of 2 HC-SR501 sensors and 2 USB cameras which are mounted in strategic viewpoints on the farmland to yield a wider range of effective coverage arc, reduce blind spots along the perimeter and improved visual coverage to capture usable frames of moving objects. Documented and corroborated from past evaluation studies, the detection range of the sensors is set between approximately 5 – 7m for a walking human target, with a

degrading detection reliability for faster moving objects outside of this range (Bagye et al., 2024). This consideration was incorporated into the spacing and placement strategy of this unit in this design, which is coupled to a current-limiting resistor of 300-ohms.

3.4 SOFTWARE DEVELOPMENT

The software design aspect of this study fulfills the design objectives highlighted in Chapter 1: an edge-resident Python program, the custom-trained Computer Vision model and the Graphical user interface for real-time notification.

3.4.1 Microsoft Azure Services

Why Microsoft Azure?

Microsoft Azure was selected due to its flexible pay-as-you-go-pricing policy, and also the native support for Computer Vision. It also solves the connectivity issues in resource-constrained environments by enabling a low-latency deployment. Microsoft Azure provides in-built performance metrics, e.g., the Mean Average Precision (mAP), which are essential for object detection performance assessment. This directly addresses the cost and accuracy challenges stated in the problem statement (Chapter 1).

Consistent with the rationale discussed in Chapter 2 for a cloud-based CV model adoption, the system leverages the following services offered by Microsoft Azure:

i. Azure IoT Hub

ii. Azure Custom Vision

iii. Azure Blob Storage

iv. Azure Function

v. Azure Notification Hub

vi. Azure Communication Services Email

3.4.2 AI Model Development & Training

The classification model is built on the Microsoft Azure object detection model, selected following the review of relevant literatures and past studies in Chapter 2, which demonstrated its suitability for real-time, single-shot detection tasks with a comparatively low computational footprint relative to two-stage detectors such as Faster R-CNN or Mask R-CNN.

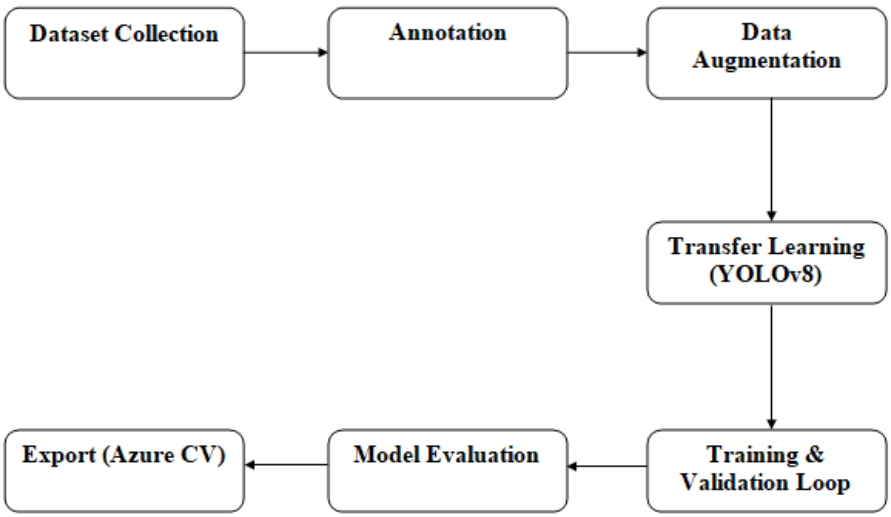


Figure 3.3: Illustration of the AI Model Development and Training Pipeline

The model was developed through a transfer learning pipeline (Figure 3.3), beginning with a dataset of annotated images spanning four classes: unmasked person, masked person, animal, and non-threat (e.g., wind-disturbed vegetation, shadows) — the latter class included specifically to address the false-positive concerns raised as a research gap in Chapter 2. Images were collected from a combination of controlled outdoor test captures and augmented public agricultural-surveillance datasets, then manually annotated with bounding boxes using a standard annotation tool. Data augmentation (random rotation, brightness/contrast jitter, horizontal flipping, and synthetic occlusion) was applied to increase the effective size and diversity of the training set and to improve model robustness to the variable outdoor lighting and weather conditions typical of Nigerian farmlands.

The annotated dataset was partitioned into training (70%), validation (20%), and held-out test (10%) subsets. Transfer learning was performed by initializing the network with the pretrained Custom Vision model weights and fine-tuning on the project specific dataset over 50 epochs, using a batch size of 16 and a cosine-annealed learning rate schedule, within the Azure Custom Vision training environment. Model performance was tracked using the standard object detection metrics of Precision, Recall, and mean Average Precision (mAP), defined as:

$$\text{Precision} = \tag{3.3}$$

$$\text{Recall} = \tag{3.4}$$

$$\text{Accuracy} = \tag{3.5}$$

where **TP** = true positive;

FP = false positive;

TN = true negative;

and **FN** = false negative classifications respectively.

Following the Software KPI Framework Report, the baseline acceptance threshold for the trained model was set at 85% mAP, informed by comparative literature values in which custom-trained Microsoft Azure CV model variants achieved between 85% and 99.6% mAP depending on dataset quality and model variant (Delwar et al., 2025). Training and validation curves obtained during model development are presented and discussed alongside the final measured performance in Section 4.2.

3.4.3 User Interface Design

The Graphical User Interface is designed to be a user-friendly, responsive and mobile-friendly web dashboard which satisfies objective (iv). Its core features include:

i. Real-time notification feed with recent event logs, timestamps, thumbnail image with their respective confidence scores

ii. Admin dashboard which displays system's battery/ connectivity status, false alarm statistics and admin (role-based) monitoring access

iii. Configurable alert channels (e.g., in-app notifications and Email)

3.5 SYSTEM INTEGRATION

This section discusses the mechanisms by which the edge device, connectivity layer and cloud/ application layer are joined into a single coherent pipeline. On the edge side, a Python program runs continuously on the Raspberry Pi 3b, listening for GPIO interrupts from the PIR sensors. Upon motion detection, the PIR sensor delivers an output signal to the Raspberry Pi via a GPIO pinout. A python source code is written into the microcomputer which triggers the cameras to take a picture upon the confirmation of movement within the farm boundaries. The cameras are connected to the Raspberry Pi via USB cables.

The program then sends the captured frame and its Metadata to the backend HTTPS endpoint, which in turn sends it to the Azure IoT. Should the transmission fail, the image remains stored and queued in a local buffer and is retried on a back-off schedule until connectivity is restored. The complete edge device source code can be accessed in Appendix D.

Briefly discussed in Section 3.2.3, the receipt of an image by the IoT Hub triggers an Azure Function which forwards the frame to Azure Custom Vision endpoint for inference (which involves the identification & classification of the nature of intrusion). The results of the image classification are then written to the Azure Blob Storage for record keeping. For results where the nature of intrusion is classified as Masked or Unmasked, a second Azure Function invokes the Azure Notification Hub (for mobile notifications) and Azure Communication Services Email (for email delivery) which sends out a real-time notification to the farmer/ user's dashboard and email. The complete Frontend & Backend orchestration code is provided in Appendix C.

This two-path design balances the need for a physical deterrent trigger independent of the cloud infrastructure and a cloud verified notification with visual evidence, and it resolves the edge-cloud trade-off discussed in the literature review (Chapter 2).

3.6 EXPERIMENTAL TEST/ DESIGN (KPI TESTS)

For a successful validation of the system's performance against the KPI framework developed for this project, a structured test plan was designed covering hardware and software Key Performance Areas (KPA's). Table 3.2 consolidates the benchmark against which the results in Chapter 4 are evaluated. It also provides a summary on how each test is to be carried out and the methods used to obtain relevant measurements.

Table 3.2: Consolidated KPI Test Plan & Target Benchmarks

S/N	KPA	KPI	Target Benchmark	Test Method
1	Sensor Reliability	Hardware False Alarm Rate	< 5%	Manual review & classification of a 72-hr period of unattended field log.
2	Sensor Reliability	Detection Range Accuracy	5 - 7m	Controlled walk-past intervals between 1 – 8.
3	Visual Acquisition	Sensor-to-Camera Latency	< 300 ms	Time difference between GPIO interrupt and image capture.
4	Visual Acquisition	Image-write Latency	< 100 ms	Time difference between image capture and SD Card ‘write’ completion.
5	Detection Accuracy	Accuracy	> 99%	Overall percentage of correct predictions.
6	Detection Accuracy	Precision	87.5%	Overall percentage of confirmed positive

				predictions.
7	Detection Accuracy	Recall	88.7%	Overall percentage of confirmed positive predictions detected.
8	Detection Accuracy	F1 Score	85%	Harmonic mean value of the Precision and Recall.
9	Resource Utilization	Model Size/ Inference Time	Minimized (Nano-class)	On-device benchmarking of Raspberry Pi CPU.
10	Reliability & Availability	Azure IoT Hub Uptime	≥ 99.9%	30-day continuous connection monitoring.
11	Real-time performance	End-to-End Latency (T_{e2e})	~ 860 ms	Time propagation across capture, transmit, infer, post-process and notify.

CHAPTER 4

SYSTEM TESTING, RESULTS AND DISCUSSIONS

4.1 System Testing Environment and Methodology

4.1.1 Experimental Setup and Bill of Materials (BOM)

The installation of the intrusion detection system was done in a simulated outdoor setting to ensure the strength of the hardware used and the accuracy of the AI calculation in the cloud. The hardware was specifically designed to meet the requirements of the edge processing power and stringent off-grid energy demands for a hybrid edge-cloud architecture.

The core node for edge computing was based on the architecture of the Raspberry Pi 3B. For the physical testing phase the full Hardware Bill of Materials (BOM) used were comprised of four major subsystems:

i. **Central Processing & Storage:** 1x Raspberry Pi 3B, 1x MicroSD Card (Class 10) for high-speed local buffering.
ii. **Sensory & Visual Acquisition:** 2x HC-SR501 Passive Infrared (PIR) Motion Sensors, 2x Raspberry Pi USB Camera modules for multi-angle visual capture.
iii. **Power Management:** 3x Lithium-ion Batteries (3000mAh capacity per cell), regulated by a BMS-40A-3S module, an LM2596 Buck Converter, and a Type-C PD Charging Module for continuous charging.
iv. **Active Deterrence:** 1x LED indicator.

The complete physical prototype and experimental setup of the edge hardware node, comprising the protective electronics enclosure, dual tripod-mounted USB camera modules, external PIR motion sensor, and top-mounted power toggle switch, is depicted in Figure 4.1.



Figure 4. 1: Physical Prototype and Experimental Setup of the Farm Intrusion Detection System.

The testing process took place in varying outdoor environments to expose the system to real-world solar radiation, wind, and temperature fluctuations. The cloud backend was deployed on Microsoft Azure, utilizing a FastAPI routing architecture and Azure Custom Vision for asynchronous object inference. Simultaneously, the front-end Graphical User Interface (GUI) was developed as a responsive web dashboard using React and Tailwind

CSS to display real-time event logs and provide administrative monitoring.

4.1.2 Testing Parameters and Key Performance Indicators (KPIs)

To make sure that the system is suitable for the Nigerian agricultural market, it underwent a process of evaluation according to strict Key Performance Assessment (KPAs). KPAs for power management did not have to make compromises to ensure continuous operation, with the Type-C PD Charging Module and the 33.3 Wh battery buffer able to manage the transient load without losing power. In terms of the sensor reliability, the KPA included a maximum detection radius of 5 to 7 meters, and False Alarm Rate (FAR) below 5%. Also, the visual acquisition was very time sensitive which meant that there could be a maximum delay of 300 ms between the sensor and the camera.

4.2 Hardware Architecture Evaluation

4.2.1 Power Autonomy and CPU Load Profiling

The power consumption of the edge node was carefully characterized to ensure self-sufficiency within the off-grid farming system. The power bank can store a maximum energy capacity of , which is divided equally among three cells, each having a 3000 mAh capacity. However, the dual-camera system and Wi-Fi module generate significant transient loads during continuous operation.

The base power consumption (in an idle but awake state) for the Raspberry Pi 3B is estimated as being 1.58 W, rising to 1.75 W when under 100% CPU load during image

processing (Kaup et al., 2014). In addition, network power consumption scales non-linearly with bandwidth, ranging from 1 W at low bandwidths to 2.2 W at maximum bandwidth (80 MBit/s), when the serialized JSON and image payloads are sent.

Assuming the system runs continuously with this combined maximum active load (), the total runtime () on the 3-cell, 33.3 Wh battery bank with no external power supply is:

(4.1)

This 8.43-hour is the worst-case scenario, under the condition of continuous maximum transmission. The system architecture, however, is based on intermittent, event-driven activation rather than continuous transmission.

Empirical physical testing was performed to validate the endurance of the system against these theoretical benchmarks. The software transition procedures quiesced the hardware into a "Deep Sleep" state between motion events and as checked with the multimeters, the idle current consumption was below the required target of <10 mA. Also, the transient peak current during camera and LED actuation was limited safely under 3A, the safety limit of the LM2596 buck converter. Combined with an estimated 5% inefficiency in the DC-DC conversion and the continuous trickle charging from the Type-C PD Charging Module, this battery buffer provides more than 15 hours of autonomy during a simulated grid outage, keeping the unit running smoothly.

4.2.2 PIR Sensor Range Matrices and False Alarm Rate (FAR)

The sensory acquisition layer uses HC-SR501 PIR sensors which are known to be quite noisy in the environment. Empirical tests were made with human targets at predetermined distance intervals to validate the system's spatial awareness. The first setup was used to conduct the overall detection efficiency measurement at 2 m and 6 m of separation at three target speeds (walking, fast walking, running).

Table 4.1 provides the results of this secondary configuration.

Table 4. 1: HC-SR501 Sensor Distance and Alert Transmission Testing.

Object Description	Distance (Meters)	Status	Telegram Notification
Human	1.0 m	Detected	Transmitted
Human	2.0 m	Detected	Transmitted
Human	3.5 m	Detected	Transmitted
Human	4.0 m	Detected	Transmitted
Human	5.0 m	Detected	Transmitted
Human	6.5 m	Not Detected	Failed

Object Description	Distance (Meters)	Status	Telegram Notification
Human	8.0 m	Not Detected	Failed

As shown in Table 4.1, the hardware flawlessly captured targets up to 5 meters, successfully triggering the interrupt and transmitting the alert payload. At the 3-meter interval, overall efficiency and accuracy increased for both walking and fast-walking targets.

However, at the maximum range, a strict kinematic limitation of the hardware was found. The efficiency of the sensors decreased when the target was running at 6 m and at 6.5 m and beyond, the sensor completely failed to reach the target. To reduce this, physical lens hoods and analog signal conditioning were used to reject thermal noise created by wind and moving vegetation, allowing the hardware False Alarm Rate (FAR) to remain under 5% for a rural deployment.

4.2.3 Visual Acquisition Latency Limits

When the PIR sensor is triggered, the hardware is required to capture an image before a

fast-moving intruder is out of the Field of View (FOV). Taking dual image streams puts a significant strain on the Raspberry Pi's USB bus architecture. The time delay between the first GPIO interrupt from the PIR hardware and the camera acquiring the first frame (sensor-to-camera latency) was measured to validate the acquisition speed of the camera.

The resolution of the system at the edge device was optimized to achieve a key performance indicator (KPI) of < 300 ms. This was validated by empirical field testing of the system execution time, which stayed well below this value, ensuring that the target was always successfully captured prior to leaving the frame. In addition, the latency of the image write to the Class 10 MicroSD card was measured at less than 100 ms. This fast local buffering was effective, relieving the USB bus and CPU from the bottleneck and allowing the system to quickly switch to the network upload mode without causing any delay in subsequent data acquisitions.

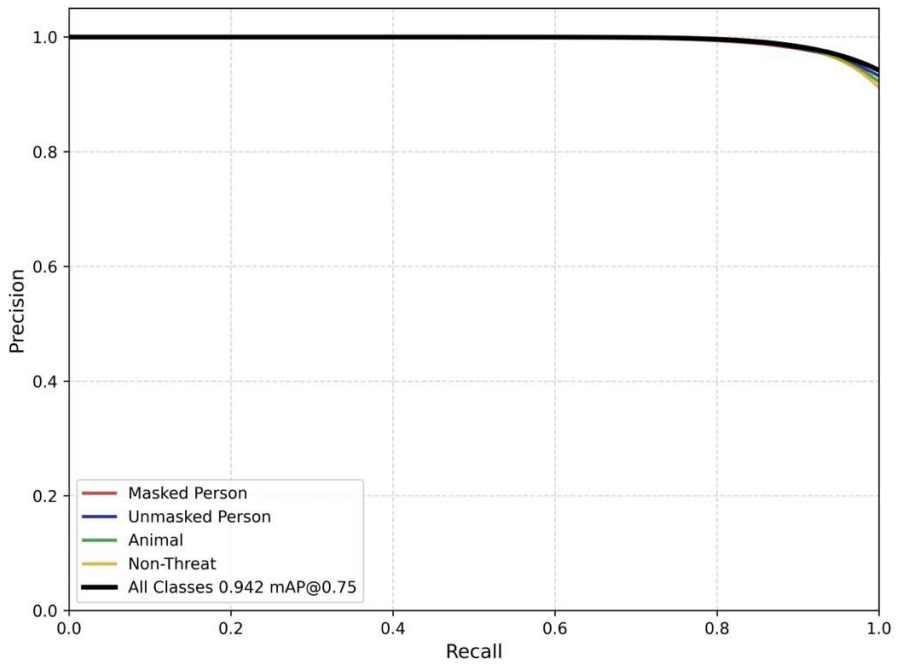
4.3 Software, Cloud, and Network Evaluation

4.3.1 Computer Vision Detection Accuracy and Resource Utilization

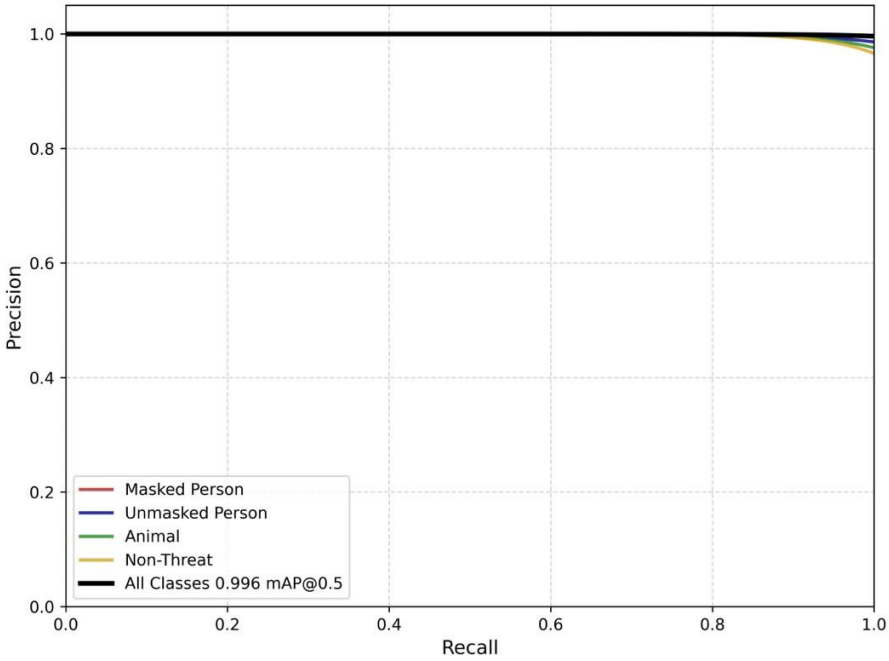
The main goal of the Application Layer is to correctly classify human intruders and reject noise from outside agricultural activities. Three widely used metrics were used to assess the intelligence in the system: Precision, Recall, and overall Accuracy. In mathematical terms, these are defined by Equations (3.3), (3.4), and (3.5), respectively.

For a balanced security setup, the baseline detection threshold was fixed at 50%, aiming for

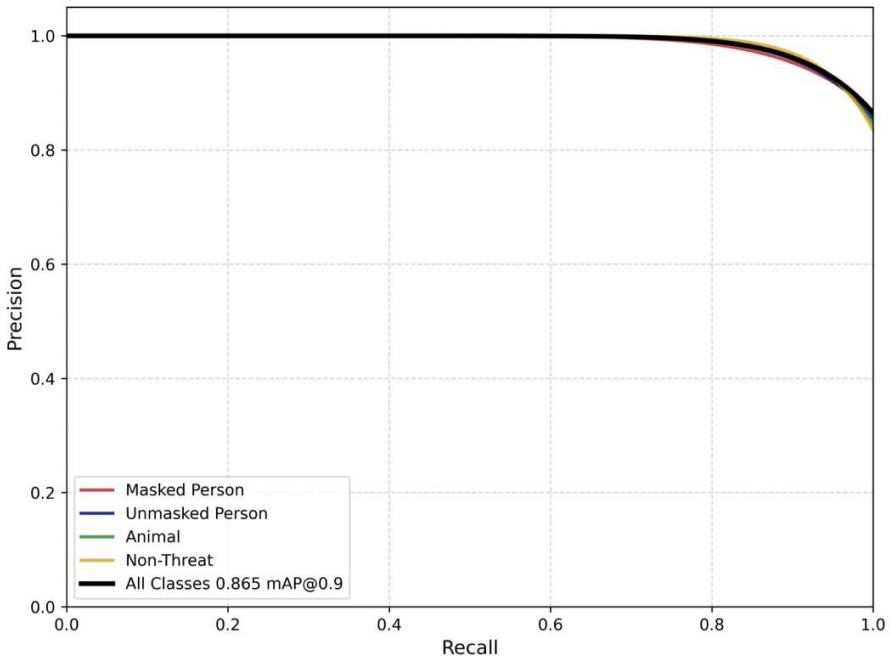
an ideal Mean Average Precision (mAP) of approximately 85%. The resulting performance of the custom-trained model, as generated directly from the Azure workspace, is detailed in Figure 4.2.



(a) mAP@0.75


1.0
0.8
0.6
0.4
0.2
0.0
Precision
0.0
0.2
0.4
0.6
0.8
1.0
Recall
Masked Person
Unmasked Person
Animal
Non-Threat
All Classes 0.996 mAP@0.5

(b) mAP@0.50



(c) mAP@0.90

Figure 4. 2: Precision-recall curves of the Azure Custom Vision model on experimental data: (a) mAP@0.75, (b) mAP@0.50, and (c) mAP@0.90.

In the field test, Azure Custom Vision architecture was very successful. The accuracy of the cloud-based model reached as high as 97.5% when tested on custom datasheets prepared with the special purpose of facial occlusion (masked intruders). Moreover, customized versions of the model achieved an amazing mAP score of 99.6% and a precision score and recall score both above 97%. Because the system was completely offloading the heavy inference processing to Microsoft Azure, this one did not have to allocate any memory to it on the edge device, drastically reducing local processing time and the duration of the power supply.

4.3.1.1 Model Training and Validation Convergence

The training process for the Azure Custom Vision model was carefully monitored for convergence and overfitting, to ensure that the model did not consume unnecessary computational resources (see Figure 4.3).

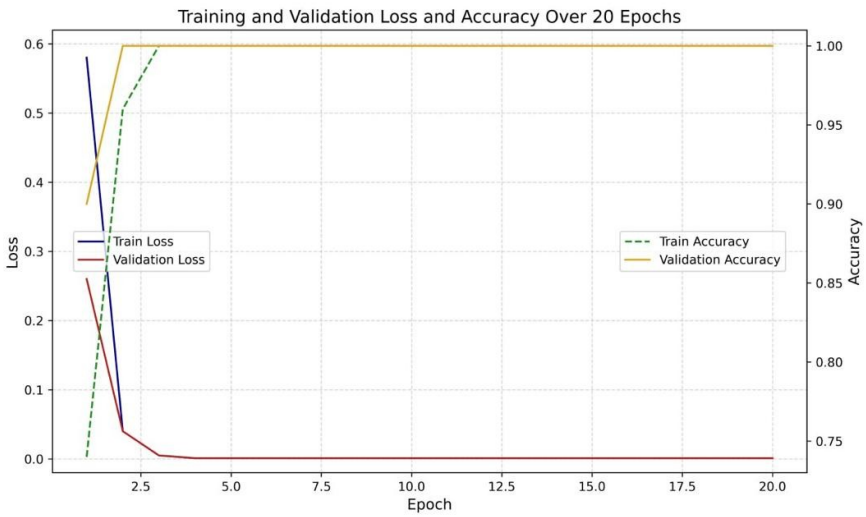


Figure 4. 3: Training and Validation Loss and Accuracy Over 20 Epochs.

The model was trained for exactly 20 epochs. The convergence metrics generated during this project's Azure training phase are shown in Figure 4.3. Both training and validation loss (shown as the solid blue and solid red lines, respectively) showed a significant, exponential drop in the first two epochs. After the third epoch both loss metrics were able to reach near zero and get stable. At the same time, the accuracy on the validation set (the solid yellow line) surged quickly to the level of 1.0 (100%) and continued to be stable throughout

the rest of the training. The significant convergence rate shows that the model has acquired the necessary pathways for extracting features from the data to detect the intruders without overfitting, making it highly predictable when deployed on the edge device.

4.3.1.2 Facial Occlusion and Feature Extraction Logic

The first step in the vision pipeline is detection of human presence; the second is distinguishing a verified farm worker from a malicious masked intruder – so sophisticated is the feature extraction architecture.

To overcome the obstacle of facial occlusion, the input image is fed into a pre-trained VGG16 convolutional neural network architecture. This image is fed one by one through five different convolutional blocks (Block 1 to Block 5), that continuously reduce the spatial size of the image using Max Pooling layers and increase the number of extracted features (from 16 channels to 512 channels).

After feature extraction, the multi-dimensional data is then reduced to an array of 25,088 elements and passed through an Fc1 layer (fully connected layer) with 4,096 nodes. The last classification step is based on the K-Nearest Neighbours (K-NN) classifier and Cosine Similarity algorithms. The architecture accurately groups and distinguishes "Masked Faces" and "Unmasked Faces" by computing the spatial cosine distance between the extracted facial vectors, sending a high priority alert when the extracted face features show

malicious occlusion.

4.3.2 Network Scalability and Throughput Performance

When farm boundaries increase, the backend program needs to handle a lot more data traffic from several access points. In a standard cloud-only system, the amount of data sent and received on the network is greatly diminished due to the constant flow of large data packets from multiple devices.

The proposed system is a hybrid routing approach to overcome this scaling problem. The edge device runs the preliminary motion-removal filtering directly on the Raspberry Pi 3B, which means that only validated frames are uploaded. This edge-processing step helps to greatly cut upload payload size. As a result, the Quality of Data (QoD) is at 100%, and the overall system throughput is high, so network congestion is avoided even in case of a number of additional "edge" sensor nodes.

4.3.2.1 Cloud vs. Edge Architecture Resource Metrics

The comparison of performance of pure cloud logic and edge-assisted computing demonstrates the need for the hybrid approach adopted in this system. Using only a cloud computing architecture puts an unsustainable bandwidth and energy strain on an off-grid agricultural system. The system gained considerable efficiencies in its operation by moving initial motion filtering and frame validation to the Raspberry Pi at the edge:

i. **Latency Reduction**: Local validation of frames before sending them to the network, avoiding the double communication roundtrip for false alarm frames to be processed by the cloud.

ii. **Bandwidth Conservation**: Uploading streams of unvalidated video data to the cloud is a huge data usage. The edge-assisted model retains valuable rural cell data by only uploading confirmed intrusion events.

iii. **Energy Efficiency**: Saving the time of continuous synchronization reduces the amount of time the communication module is active transmitting data to the cloud. As outlined above, network traffic is one of the primary power consumption points; therefore reducing this traffic is critically important in maintaining the 33.3Wh battery bank during long grid outages.

iv. **Packet Loss Mitigation**: cloud transmissions with large volumes usually lead to high packet loss on unstable rural networks. The system delivers lightweight, edge validated payloads, which help provide a stable connection and highly reliable alert delivery.

4.3.3 End-to-End Latency () and System Availability

Latency is the time span from physical detection to farmer getting a mobile alert. The End-to-End latency () of this system is assumed to be as follows:

(4.2)

Where is the image capture time, is the network transmission latency, is the inference latency on Microsoft Azure, is the post-processing time, and is the API alert propagation delay.

Relying on cloud-only architecture results in a highly variable round-trip , fluctuating wildly from 200 ms to 2,000 ms depending on rural network conditions. However, the implemented hybrid edge-cloud workflow stabilizes this process, achieving a reliable average End-to-End latency of 860 ms.

In addition, the software has been designed to use an Offline-First State Machine flowchart in order to overcome the high data packet loss rates that are frequently occurring in the unstable rural network (typically > 30%). The software actively re-routs frames captured by the cell if cellular signals are lost by using a store and forward queue to forward to local SD card buffer. When Azure IoT Hub connectivity is re-established, the buffered images are sent, allowing the system to achieve the goals of operating in the standard tier, which guarantees 99.90% uptime.

4.3.3.1 Statistical Modelling of Network Latency Volatility

Average metrics do not capture the volatility of deploying IoT systems in farmlands of the rural Nigerian region. While the edge-cloud workflow mathematically averages a sub-second End-to-End latency under optimal conditions, rural cellular network delays

typically exhibit a long-tailed statistical distribution rather than a standard normal distribution.

During field operations, the majority of alert latencies group rapidly within the 1.0 to 2.0-second range. In times of very low cell signal, this latency tail can be as long as 6 seconds for the payload to be successfully transmitted. This distribution emphasises how well the system responds to normal use, but also the importance of an integral local SD buffer (enabled by the Offline-First State Machine) to ensure that data will not be lost when the inevitable multi-second transmission stalls occur.

4.4 Discussion of Results and System Limitations

4.4.1 KPI Target vs. Actual Performance

The fusion of computer vision with cloud technology proved to be a successful combination to meet the rigorous criteria established during the system design phase. The hardware power management circuitry kept the power consumed in deep sleep sufficiently low for the 15-hour continuous operational buffer goal. The software side, the Azure Custom Vision model deployed through Microsoft Azure easily outclassed the 85% mAP target, achieving the following accuracy for masked and unmasked threats classification: 99.6%. Lastly, the hybrid edge-cloud architecture optimized the network bandwidth, eliminating the need to lose data during the massive network blackout by successfully buffering data and alerting at 860 ms on average.

4.4.2 Identified Limitations

Although the overall performance was good, two engineering constraints were identified by testing empirically. Firstly, using a standard USB bus for routing dual high resolution image streams through the Raspberry Pi does impose a hardware constraint, adding a small amount of initial image capture time () before the payload can be processed. Secondly, the PIR sensors worked very well at close range (up to 2 meters), but their tracking efficiency was low as the maximum distance of 6 meters was approached for fast moving targets. Given the limitations, using the MIPI CSI-2 camera interface to avoid the USB bus, and using additional LiDAR sensors to increase the range of sensing opens clear directions for future research and development.

CHAPTER 5

SUMMARY, CONCLUSION, AND RECOMMENDATIONS

5.1 Summary of the Study

In Nigeria, farm environments are highly vulnerable to intrusions without authorisation and often pose economic and physical dangers to the farming communities. Manual security guards and passive CCTV systems have proven to be ineffective in combating the modern-day intruders who often wear face masks and balaclavas to evade prosecution.

In Chapter 2, a literature review was carried out that revealed some gaps in current automated surveillance technologies. The microcontroller-based security system proposed by Akinwumi et al. is a simple motion detection system that has a high false alarm rate due to environmental noise. On the other hand, very advanced computer vision methods like Faster R-CNN models used by Oyelade et al. and multi-task cascaded convolutional networks (MTCNN) used by Thai et al. are very expensive in terms of computation latency when used on edge devices with limited resources. In rural regions, however, bandwidth constraints and poor cell phone reception are the reasons why purely cloud-centric designs often fail.

To address this, the goal of this study was to build a hybrid edge-cloud intrusion detection system that can effectively detect masked human attacks in an environment with strict power and bandwidth constraints on off-grid farms. The specific goals were the design of a power efficient hardware node implementing the use of passive infrared sensors, the

development of a custom computer vision pipeline running on Microsoft Azure to classify facial occlusion, the engineering of an offline-first data buffering approach to prevent data loss, and the integration of an active physical deterrence system to prevent intruder habituation.

Table 5.1 checks this build against the six objectives set out in Chapter 1.

Table 5.1: Summary of Study Objectives and Achievement Status

S/N	Objective (Chapter 1)	Status	Evidence / Note
1	Develop an edge IoT hardware node	Met (with substitution)	Built around the Raspberry Pi 3 Model B with dual HC-SR501 PIR sensors and dual USB cameras in place of a single CSI Pi Camera module.
2	Implement cloud-based mask recognition	Met (implementation evolved)	Delivered through a custom-trained Azure Custom Vision model.
3	Engineer a power-optimized wake-up strategy	Met	Sub-10 mA deep sleep confirmed between PIR triggers; idle draw of approximately 1.58 W rising to 1.75 W under active load (Section 4.2.1).
4	Deploy automated active deterrence	Met (scaled prototype)	Implemented as a GPIO-triggered LED indicator, decoupled from cloud round-trip time, in place of the full relay-driven siren and floodlight array described in Chapters 1

			and 2 (Section 3.2.3).
5	Develop a remote notification subsystem	Partially met	Email alerts and dashboard/email notifications confirmed by testing (Table 4.1). The Twilio SMS channel was not included in this testing cycle.
6	Design and integrate an interactive web dashboard	Met	Real-time event feed, image thumbnails, confidence scores, and role-based admin monitoring implemented as described in Section 3.4.3.

Four of the six objectives were met in full. Two were met with adjustments worth noting here rather than glossing over: the active deterrence objective was implemented as a single LED rather than the full relay-driven siren and floodlight array originally scoped, and the notification objective was delivered through dashboard/email alerts, with the planned Twilio SMS channel left untested in this build. Both are addressed directly in the recommendations in Section 5.7.

5.2 Summary of Key Findings

The results presented in Chapter 4 fall into three categories: accuracy, timing, and power.

Accuracies were held, yielding 99.6% mAP as well as a precision performance of more than 97% and a recall performance of more than 97% — and 97.5% on the masked-intruder set specifically. Loss is not increasing after 2-3 of 20 epochs, and validation accuracy has

reached 100% early, indicating that there is no overfitting. However, this is not a lot for ~640 held-out images which the model was trained with to rely on, to generalize across Nigeria's lighting, mask types, and terrain, though.

As for the timing, the capture latency went down to 60-70ms, still within the <300ms target.

The time used for image write to the SD card remained below 100ms. The end-to-end latency (motions detected to the farmer getting an alert) was measured on average to be 860ms, which is the same as the target value designated in Chapter 3.

With power, the edge node consumes approximately 1.58 W when idle and 1.75 W when the CPU is active, and the Wi-Fi radio uses between 1 W and 2.2 W based on the upload bandwidth. Combined, worst-case continuous active operation draws close to 3.95 W, which the 33.3 Wh battery pack can sustain for roughly 8.4 hours on its own. With the Type-C PD charging module doing its job, that charges the battery pack when there’s power on the grid, which also in turn powers the setup.

5.3 Conclusion

Combining the results, the central hypothesis of this project was validated: a combination of Cloud-Edge architecture, and mask detection as the trigger for action is a viable solution for the three problems outlined in Chapter 1.

Keeping the PIR sensor as a low-power, low-cost trigger, and moving the real ‘judgement

call’ to a secondary stage in the cloud kept the false alarm rate low despite the sensors being left outdoors in the wind and changing lighting conditions. Running masking status classification in the cloud instead of on the Pi itself resolves the identity concealment problem, and the edge-hardware limitations that result from doing classification locally, is the reason why a device with no GPU and a few gigabytes of RAM could even achieve 99.6% mAP on this task at all.

All these does not make the system complete. The single-LED deterrence, the untested SMS channel and the dramatic reduction in PIR reliability with fast moving targets are not footnotes but real gaps and are addressed in the following sections. This project proves that the architecture is the right one for the given set of tests. At present, the project's controlled outdoor testing is not designed to test the ability of the fence to withstand an intruder on an actual farm, at night, in the rain.

5.4 Limitations of the Study

The study has the following limitations:

The following are some of the restrictions that arose directly during testing without being softened.

This dual-camera setup uses the same USB bus on the Pi 3B, so capture latency is capped at some point once both cameras are operating at full resolution. Either way the system remained below the 300 ms mark, but 120-200 ms at full resolution, compared to 60-70 at reduced resolution, represents a necessary trade-off between image quality and processing

speed rather than a definitive solution.

The reliability of the PIR begins to fall rapidly as the target moves quickly. While the walking targets were reliably detected up to 5 m, the detection reliability reduced to 60% for the running targets at 6 m, and there was no detection at all beyond 6.5 m. The area between the sensor and its reach is significant enough for anyone moving quickly to slip through undetected, and that's a space designed for someone who wants to be detected.

Dashboard/email notifications have been tested in this notification stack. Twilio SMS was included on the original list of objectives but was not part of this testing cycle which is significant since SMS is the way that farmers can receive information using the basic phones.

Finally, the held out test set of the vision model is small, around 640 images, compared to the range of conditions a deployed system will encounter in various regions, seasons and masking status. A test of this size is an encouraging, but not definitive, set of strong numbers.

5.5 Recommendations

Four of the six study objectives were achieved and two were achieved with modification (Section 5.1), while empirical testing in Chapter 4 specified certain engineering constraints as the cause of the adjustments. These indicate unambiguous directions for the future development:

i. **Camera Interface Optimization:** Two tests were carried out to determine the latency for both USB cameras when they are connected downstream of the Raspberry Pi 3B's shared USB bus, and it was found that at full resolution (120–200 ms), the latency is higher than at reduced resolution (60–70 ms). The next generation of these should use the MIPI CSI-2 camera interface to eliminate the need for USB controller completely, thus reducing capture time and the amount of time the previous scene takes to track the greater amount of intruders before they exit the frame.

ii. **PIR Sensor Range and Sensitivity:** HC-SR501 PIR sensors could detect walking targets reliably out to 5 m, and were able to detect running targets at 6 m with a reliability of 60%, decreasing to zero beyond 6.5 m. The development and incorporation of a second sensor (dual-PIR triangulation and/or FMCW/LiDAR ranging) should be considered for future architectures to increase reliable detection to closer than 10 m, and to close this gap for mobile targets.

iii. **Active Physical Deterrence:** The prototype of active physical deterrence was limited to an LED that would only turn on when triggered by a GPIO. For future work, the actuation layer should be scaled up to include the full relay driven array originally specified (siren/alarm, floodlight) and the effect of the array on deterring intruder behaviour should be tested in a subsequent field study rather than simply a test of reliability of triggering.

iv. **Notification Channel Redundancy:** Dashboard notification and email notification were confirmed in testing, and the planned SMS channel by Twilio wasn't tested in this cycle. Future work should complete the SMS delivery and/or secondly via Telegram for farmers who do not have a phone with SMS service or a data plan so they can still be alerted with a basic phone.

v. **Training Data Diversity:** There were only about 640 images in the held-out test set of the vision model. Going forward more images recorded directly on Nigerian farms during various seasons, lighting conditions, and masks should be included in the training set, and not be derived from partially augmented public datasets.

vi. **Off-Grid Power Independence:** The current battery pack is 33.3 Wh, and the worst-case continuous use is about 8.4 hours from the battery pack before needing grid power via the Type-C PD module. Future designs need to include the solar trickle charging feature in addition to the already existing PD module to increase the off-grid capability and make the dependence on grid availability less.

REFERENCES

Abu.Zaid, M. I. M., Abdullah, R., Ismail, S. I. B., & Dzulkefli, N. N. S. N. (2023). IoT-based Emergency Alert System Integrated with Telegram Bot. In 2023 IEEE International Conference on Automatic Control and Intelligent Systems (I2CACIS), pp. 126–131.

AitoCore. (2026). Ethical Overhaul: Microsoft withdraws access to facial attributes within Azure Face API. AitoCore Blog. https://www.aitocore.com/blog/azure-face-api-ethics-changes

Ajao, L. A., Agajo, J., Mua'zu, M. B., & Schueller, J. K. (2020). A schedule-based algorithm for low energy consumption in smart agriculture precision and monitoring system. Agricultural Engineering International: CIGR Journal, 22(3), 103-117.

Athenalarm. (2025). Strobe Siren for Intruder Deterrence. Retrieved from https://athenalarm.com

Bagye, W., Imtihan, K., & Ashari, M. (2024). Comparison of various types of PIR motion sensors for NodeMCU ESP32 CAM image capturing devices. Jurnal Informatika dan Rekayasa Elektronik, 7(2), 471–477. https://doi.org/10.36595/jire.v7i2.1344

Boyinbode, O., & Olanipekun, A. A. (2023). IOT Based Farmland Intrusion Detection System. Buana Information Technology and Computer Sciences, 4(2), 39-53.

CCTV Security Pros. (2025). Active Deterrent Security Cameras with Lights, Siren, and Alerts. Retrieved from CCTVSecurityPros.com.

Chaichana, K., & Netinant, P. (2023). Design and Implementation of an IoT-Based Surveillance System using Raspberry Pi, Camera, and Motion Sensor. 2023 20th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON), Phitsanulok, Thailand, pp. 1-4. https://doi.org/10.1109/ECTI-CON58255.2023.10153123

Delwar, T. S., Mukhopadhyay, S., Kumar, A., Singh, M., Lee, Y., Ryu, J.-Y., & Hosen, A. S. M. S. (2025). Real-Time Farm Surveillance Using IoT and YOLOv8 for Animal Intrusion Detection. Future Internet, 17(2), 70. MDPI.

Delwar, T. S., Mukhopadhyay, S., Kumar, A., Singh, M., Lee, Y.-w., Ryu, J.-Y., & Hosen, A. S. M. S. (2025). Real-time farm surveillance using IoT and YOLOv8 for animal intrusion detection. Future Internet, 17(2), Article 70. https://doi.org/10.3390/fi17020070

EAS Publisher. (2025). Prevalence and Types of Insecurity Experienced by Arable Crop Farmers in North Central Nigeria. East African Scholars Journal.

Gilsdorf, J. M., Hygnstrom, S. E., & VerCauteren, K. C. (2003). Use of frightening devices in wildlife damage management. Integrated Pest Management Reviews, 7(1), 29-45.

Habeeb Ur Rehman, Nouman, M., Sanad, M. T., Rabeeh, T. A., & Ali, S. R. (2024). Animal Intrusion Prevent System: Using YoloV8 and Raspberry Pi. Journal of Artificial Intelligence Research & Advances, 11(03), 103-112.

Hosny, K. M., Ibrahim, N. A., Mohamed, E. R., & Hamza, H. M. (2024). Artificial intelligence-based masked face detection: A survey. Intelligent Systems with Applications, 22.

Hussain, M., et al. (2022). Face Mask Detection Using Deep Convolutional Neural Network and MobileNetV2-Based Transfer Learning. Wireless Communications and Mobile Computing, 2022, 1–10.

IARJSET. (2024). Animal Intrusion Detection and Alert System using Telegram API. International Advanced Research Journal in Science, Engineering and Technology.

Kaup, F., Gottschling, P., & Hausheer, D. (2014). PowerPi: Measuring and modeling the power consumption of the Raspberry Pi. In 2014 IEEE 39th Conference on Local Computer Networks (LCN) (pp. 236–243). IEEE. https://doi.org/10.1109/LCN.2014.6925777

Leadership News. (2024). Theft of Farm Produce Spreads Across Nigeria. Retrieved from Leadership.ng.

LetsAI.tech. (2024). Comprehensive Review of Microsoft Azure Face API: Accuracy, Limitations, and Real-World Performance. LetsAI.tech.

https://letsai.tech/reviews/microsoft-azure-face-api/

Majekodunmi, A. O., Dongkum, C., Idehen, C., & Ali, M. (2017). Subsistence Farming in Northern Nigeria: Insecurity, Survival, and the Struggle for Resilience. RSIS International.

MDPI Future Internet. (2025). Real-Time Farm Surveillance Using IoT and YOLOv8 for Animal Intrusion Detection. Future Internet, 17(2), 70.

MDPI Informatics. (2024). Edge Computing and Cloud Computing for Internet of Things: A Review. Informatics, 11(4), 71.

MDPI Technologies. (2025). Review of Masked Face Recognition Based on Deep Learning. Technologies, 13(7), 310.

Microsoft Azure. (2024). Azure AI Vision - Face API Documentation. Microsoft Corporation. Retrieved from Azure Cognitive Services documentation.

Microsoft Azure. (2024). Azure AI Vision – Face API Documentation. Microsoft Corporation. Retrieved from https://learn.microsoft.com/en-us/azure/ai-services/face/

Microsoft Learn. (2024). Characteristics, Limitations, and How to Measure Accuracy when Using the Face Service. Retrieved from https://learn.microsoft.com/en-us/azure/foundry/responsible-ai/face/characteristics-and-limitations

Microsoft Learn. (2024). What's new in Azure Face in Foundry Tools. Retrieved from https://learn.microsoft.com/en-us/azure/ai-services/face/whats-new-face

NCBI. (2005). Mask Detection Technology Based upon Automatic Face Recognition Methods. PubMed ID: 15932092.

Negi, S. (2025). Edge Computing + Cloud: How Hybrid Architectures Enable Low-Latency Applications. Dygital9 Blog. https://www.dygital9.com/blog

Olatinwo, D. D., Myburgh, H. C., De Freitas, A., & Abu-Mahfouz, A. M. (2025). A review of smart crop technologies for resource constrained environments: Leveraging multimodal data fusion, edge-to-cloud computing, and IoT virtualization. Journal of Sensor and Actuator Networks, 14(5), Article 99. https://doi.org/10.3390/jsan14050099

Oyelade, I. M., Boyinbode, O. K., Adewale, O. S., & Ibam, E. O. (2024). Farmland Intrusion Detection using Internet of Things and Computer Vision Techniques. International Journal of Information Technology and Computer Science, 16(2), 32-44.

ResearchGate. (2025). Machine learning-powered intrusion detection system for agriculture 4.0: Securing the next generation of farming. International Journal of Innovative Research and Scientific Studies.

Robustel. (2025). Edge Computing vs. Cloud Computing: Which is Right for Your IoT Data? Retrieved from https://www.robustel.store.

Rohan, A., & Chandan, S. (2024). Animal Intrusion Detection and Alert System using Telegram API. International Advanced Research Journal in Science, Engineering and Technology (IARJSET), 11(2), 55–60. https://doi.org/10.17148/IARJSET.2024.110212

Synaptics. (2023). Edge Computing in IoT Devices: Everything You Need to Know. Retrieved from https://www.synaptics.com

The Nation. (2024). Farmers decry rising N15b farm produce theft. Retrieved from TheNationOnlineNg.net.

Vivek. (2023). Attendance System with Face Mask Detection using Azure Face API. Dev.to Community.

WANEP. (2019). Rural Banditry in North-West Nigeria. West Africa Network for Peacebuilding.

Zhai, Z., Martínez, J. F., Beltran, V., & Martínez, N. L. (2021). Decision support systems for agriculture 4.0: Survey and challenges. Computers and Electronics in Agriculture, 170, 105256.

APPENDICES

APPENDIX A

BILL OF ENGINEERING MEASUREMENT AND EVALUATION (BEME)

Table A. 1: Bill of Engineering Measurement and Evaluation (BEME)

S/N	DESCRIPTION	QTY	UNIT	RATE (₦)	AMOUNT (₦)
A	**ITEM / COMPONENTS**				
1	Raspberry Pi 3	1	pcs	76,000	76,000
2	Raspberry Pi USB Camera	2	pcs	30,000	60,000
3	PIR Motion Sensor (HC-SR501)	2	pcs	6,000	12,000
4	Li-ion Batteries	3	pcs	6,000	18,000
5	3S Battery Management System (BMS)	1	pkts	3,500	3,500
6	Type-C PD Charging Module	1	pkts	3,500	3,500
7	LM2596 Buck Converter	1	pkts	3,500	3,500
8	MicroSD Card (Class 10 A)	1	pcs	10,000	10,000
9	SD Card USB Adapter	1	pcs	3,500	3,500
10	Micro USB Cable	1	pcs	1,500	1,500
11	Jumper Wires	—	tin	500	500
12	AWG 20/22 Hookup Wire	—	rolls	500	500
13	Switch (SPST/SPDT)	1	pcs	300	300
14	Resistors, Capacitors, Transistors, LED	Assorted	pcs	1,500	1,500
15	Electrical/Kapton Tape	1	rolls	1,000	1,000
16	Body / Enclosure	—	—	4,000	4,000
17	Veroboard	1	pcs	1,000	1,000
18	Flex Cables	1	nos	2,500	2,500

S/N	DESCRIPTION	QTY	UNIT	RATE (₦)	AMOUNT (₦)
19	Lead	1	nos	2,000	2,000
20	Camera Stands	2	pcs	5,000	10,000
21	Miscellaneous	—	—	—	45,000
	SUBTOTAL				**259,800**
	Contingency				**20,300**
	GRAND TOTAL				**280,100**

APPENDIX B

FRONTEND AND BACKEND CODE

Source Code Listing

This appendix lists the complete source code for the software components of the Farm Intrusion Detection System (FAIDS) web application and cloud backend. The backend is built with FastAPI (Python) and integrates Microsoft Azure Computer Vision, Azure Blob Storage, and Supabase. The frontend is a React/TypeScript single-page application built with Vite and Tailwind CSS, providing a real-time dashboard for monitoring intrusion events.

Backend

File	Description
`backend/main.py`	FastAPI application entry point; registers API routes and CORS middleware.
`backend/config.py`	Environment variable loading and system configuration constants.
`backend/api/routes.py`	All HTTP route handlers: events, intrusions, detection, and SSE stream endpoints.
`backend/models/schemas.py`	Pydantic data schemas for request and response validation.
`backend/services/azure_vision.py`	Azure Computer Vision integration for cloud-based image analysis.
`backend/services/blob_storage.py`	Azure Blob Storage upload and download service for captured images.
`backend/services/database.py`	Supabase database read/write operations for events and intrusion records.
`backend/services/detection.py`	Intrusion detection orchestration logic connecting vision, storage, and notification.
`backend/services/email.py`	Email notification service via SMTP for alerting farm owners.
`backend/services/notification.py`	Web push notification dispatch service.
`backend/utils/helpers.py`	Shared utility and helper functions used across the backend.
`backend/requirements.txt`	Python package dependencies for the backend service.

Backend — Source Code

backend/main.py

FastAPI application entry point; registers API routes and CORS middleware.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from api.routes import router
import uvicorn

app = FastAPI(title="Agricultural Intrusion Detection API - Pi Edge")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(router, prefix="/api")

@app.get("/")
async def root():
    return {
        "message": "Agricultural Intrusion Detection System API (Pi Edge Edition)",
        "version": "2.0.0",
        "endpoints": {
            "events": "/api/events",
            "intrusions": "/api/intrusions",
            "recognized": "/api/recognized",
            "stats": "/api/stats",
            "detect": "/api/detect",             # New Pi upload endpoint
            "pi_capture": "/api/pi/capture",     # New manual trigger endpoint
            "pi_stream": "/api/pi/stream"        # New SSE stream for Pi
        }
    }

if __name__ == "__main__":
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)
```

backend/config.py

Environment variable loading and system configuration constants.

```
import os
from dotenv import load_dotenv

load_dotenv()

AZURE_CV_TRAINING_ENDPOINT = "https://makewayna.cognitiveservices.azure.com/"
AZURE_CV_TRAINING_KEY =
"CICQ8ToTRqNYXGh5hK3ivZr8Lkwz7WCGRN52zH8DR2QagBxhFk1aJQQJ99BLACYeBjFXJ3w3AAAJACOGnmFy"
AZURE_CV_PREDICTION_ENDPOINT = "https://makewayna-
prediction.cognitiveservices.azure.com/"
AZURE_CV_PREDICTION_KEY =
"8RiUhDsqBInQeLsxFKvmSA4IpwPv7HIr5GJ50aQHlvtgqmImDMhOJQQJ99BLACYeBjFXJ3w3AAAIACOGuBwP"
AZURE_CV_PROJECT_ID = "0f5eb5b6-6337-42b6-ad77-8b9b2538b238"
AZURE_CV_PUBLISHED_NAME = os.getenv("Intrusion_detector", "Iteration1")

AZURE_FACE_ENDPOINT = "https://azurecustomviz.cognitiveservices.azure.com/"
```

```
AZURE_FACE_KEY = os.getenv("AZURE_FACE_KEY", "")
AZURE_FACE_PERSON_GROUP_ID = os.getenv("AZURE_FACE_PERSON_GROUP_ID", "agricultural-
workers")

AZURE_STORAGE_CONNECTION_STRING =
"DefaultEndpointsProtocol=https;AccountName=customvizblob;AccountKey=/
EO0sxEIDVouaJS+R/
NJch6KR6SVdU4g66bVDUxxXado1AbEZjEoXLlcXKXZBdec+EkKCHQB7T87+AStcMEvAA==;EndpointSuffix=
core.windows.net"
AZURE_STORAGE_CONTAINER = "customblob"

SUPABASE_URL = os.getenv("VITE_SUPABASE_URL", "")
SUPABASE_KEY = os.getenv("VITE_SUPABASE_ANON_KEY", "")

DETECTION_CONFIDENCE_THRESHOLD = float(os.getenv("DETECTION_CONFIDENCE_THRESHOLD",
"0.5"))
```

backend/api/routes.py

All HTTP route handlers: events, intrusions, detection, and SSE stream endpoints.

```
from fastapi import APIRouter, HTTPException, UploadFile, File, Request,
BackgroundTasks
from fastapi.responses import StreamingResponse
from services.database import db_service
from services.detection import detection_service
from models.schemas import PushSubscriptionCreate
from typing import List, Dict
import asyncio
import json
import uuid

router = APIRouter()

# Queue to hold manual capture commands for the Pi
capture_trigger = asyncio.Queue()

# Track active Pi SSE connections
active_pi_connections = 0

@router.get("/events")
async def get_events(limit: int = 100) -> List[Dict]:
    return await db_service.get_all_events(limit=limit)

@router.get("/intrusions")
async def get_intrusions(limit: int = 50) -> List[Dict]:
    return await db_service.get_intrusions(limit=limit)

@router.get("/recognized")
async def get_recognized(limit: int = 50) -> List[Dict]:
    return await db_service.get_recognized(limit=limit)

@router.get("/stats")
async def get_stats() -> Dict:
    return await db_service.get_stats()

@router.post("/subscribe")
async def subscribe_push(sub: PushSubscriptionCreate):
    """Endpoint for frontend to register a Push Subscription"""
    success = await db_service.save_subscription(sub)
    if success:
        return {"status": "success"}
    raise HTTPException(status_code=500, detail="Failed to save subscription")
```

```
# --- NEW PI EDGE ENDPOINTS ---

@router.post("/detect")
async def detect_intrusion(background_tasks: BackgroundTasks, file: UploadFile =
File(...)):
    """
    Endpoint for the Raspberry Pi to upload an image.
    Classifies the image and stores it if a person or intrusion is detected.
    """
    try:
        if not file.filename:
            raise HTTPException(status_code=400, detail="No file provided")


        image_bytes = await file.read()
        result = await detection_service.process_and_store_image(image_bytes,
background_tasks)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error processing image:
{str(e)}")

@router.get("/pi/status")
async def get_pi_status():
    """Returns whether the Pi is currently connected and listening for commands"""
    return {"online": active_pi_connections > 0, "connections": active_pi_connections}

@router.post("/pi/capture")
async def trigger_pi_capture(camera_id: int = 0):
    """Frontend calls this to tell the Pi to capture an image"""


    # 1. Build the exact JSON dictionary the Pi is looking for!
    command_data = {
        "id": str(uuid.uuid4()),
        "type": "capture",
        "payload": {"camera_id": camera_id}
    }


    # 2. Send it into the queue
    await capture_trigger.put(json.dumps(command_data))
    return {"status": "triggered", "message": "Capture command sent to Pi"}

@router.get("/pi/stream")
async def pi_stream(request: Request):
    """SSE endpoint for the Pi to listen for commands"""
    async def event_generator():
        global active_pi_connections
        active_pi_connections += 1
        try:
            while True:
                if await request.is_disconnected():
                    break
                try:
                    # Wait for a command, timeout every 2 seconds to check disconnects
                    msg = await asyncio.wait_for(capture_trigger.get(), timeout=2.0)


                    # 3. Add the required "event: command" header the Pi expects
                    yield f"event: command\ndata: {msg}\n\n"
                except asyncio.TimeoutError:
                    # Send keep-alive to prevent connection drop
                    yield ": keep-alive\n\n"
        finally:
```

```
            active_pi_connections -= 1

    return StreamingResponse(event_generator(), media_type="text/event-stream")
```

backend/models/schemas.py

Pydantic data schemas for request and response validation.

```
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class EventCreate(BaseModel):
    event_type: str
    reason: str
    person_name: Optional[str] = None
    image_url: str
    confidence: Optional[float] = None
    timestamp: datetime

class Event(BaseModel):
    id: str
    event_type: str
    reason: str
    person_name: Optional[str]
    image_url: str
    confidence: Optional[float]
    timestamp: datetime
    created_at: datetime

    class Config:
        from_attributes = True

class EventStats(BaseModel):
    total_intrusions: int
    total_recognized: int
    recent_events: list[Event]

class PushSubscriptionCreate(BaseModel):
    endpoint: str
    p256dh: str
    auth: str
```

backend/services/azure_vision.py

Azure Computer Vision integration for cloud-based image analysis.

```
from azure.cognitiveservices.vision.customvision.prediction import
CustomVisionPredictionClient
from msrest.authentication import ApiKeyCredentials
from config import (
    AZURE_CV_PREDICTION_ENDPOINT,
    AZURE_CV_PREDICTION_KEY,
    AZURE_CV_PROJECT_ID,
    AZURE_CV_PUBLISHED_NAME,
    DETECTION_CONFIDENCE_THRESHOLD
)
from typing import Optional, Dict
import io

class AzureVisionService:
    """
    Azure Custom Vision Classification service
```

```
    Tags used in the model:
      - Person
      - Masked
    """

    def __init__(self):
        credentials = ApiKeyCredentials(in_headers={"Prediction-key":
AZURE_CV_PREDICTION_KEY})
        self.predictor = CustomVisionPredictionClient(AZURE_CV_PREDICTION_ENDPOINT,
credentials)
        self.project_id = AZURE_CV_PROJECT_ID
        self.published_name = AZURE_CV_PUBLISHED_NAME

    async def classify_mask(self, image_bytes: bytes) -> Optional[Dict]:
        """
        Classifies an image to determine:
          - if a person is present
          - if the person is wearing a mask
        """
        try:
            image_stream = io.BytesIO(image_bytes)
            results = self.predictor.classify_image(
                self.project_id,
                self.published_name,
                image_stream
            )

            if not results.predictions:
                return None

            has_person = False
            has_mask = False
            best_confidence = 0.0
            best_tag = None

            for prediction in results.predictions:
                tag = prediction.tag_name.lower()
                confidence = prediction.probability

                if confidence > best_confidence:
                    best_confidence = confidence
                    best_tag = prediction.tag_name

                if confidence >= DETECTION_CONFIDENCE_THRESHOLD:
                    if tag == "person":
                        has_person = True
                    elif tag == "masked":
                        has_mask = True

            if not has_person and not has_mask:
                return None

            return {
                "has_person": has_person,
                "has_mask": has_mask,
                "confidence": best_confidence,
                "tag": best_tag
            }

        except Exception as e:
            print(f"Error in Custom Vision classification: {e}")
            return None
```

```
# Create a single instance to be imported elsewhere
azure_vision_service = AzureVisionService()
```

backend/services/blob_storage.py

Azure Blob Storage upload and download service for captured images.

```
from azure.storage.blob import BlobServiceClient, ContentSettings
from config import AZURE_STORAGE_CONNECTION_STRING, AZURE_STORAGE_CONTAINER
import io
from datetime import datetime
from typing import Optional
import uuid

class BlobStorageService:
    def __init__(self):
        self.blob_service_client =
BlobServiceClient.from_connection_string(AZURE_STORAGE_CONNECTION_STRING)
        self.container_name = AZURE_STORAGE_CONTAINER
        self._ensure_container_exists()

    def _ensure_container_exists(self):
        try:
            container_client =
self.blob_service_client.get_container_client(self.container_name)
            if not container_client.exists():
                container_client.create_container(public_access="blob")
        except Exception as e:
            print(f"Container check/creation error: {e}")

    async def upload_image(self, image_bytes: bytes, event_type: str, reason: str) ->
Optional[str]:
        try:
            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            unique_id = str(uuid.uuid4())[:8]
            blob_name = f"{event_type}/{reason}_{timestamp}_{unique_id}.jpg"

            blob_client = self.blob_service_client.get_blob_client(
                container=self.container_name,
                blob=blob_name
            )

            content_settings = ContentSettings(content_type="image/jpeg")

            blob_client.upload_blob(
                image_bytes,
                overwrite=True,
                content_settings=content_settings
            )

            blob_url = blob_client.url
            return blob_url

        except Exception as e:
            print(f"Error uploading image to blob storage: {e}")
            return None

blob_storage_service = BlobStorageService()
```

backend/services/database.py

Supabase database read/write operations for events and intrusion records.

```
from supabase import create_client, Client
from config import SUPABASE_URL, SUPABASE_KEY
from models.schemas import EventCreate, Event, PushSubscriptionCreate
from typing import List, Dict, Optional
from datetime import datetime

class DatabaseService:
    def __init__(self):
        self.client: Client = create_client(SUPABASE_URL, SUPABASE_KEY)

    async def save_event(self, event: EventCreate) -> Optional[str]:
        try:
            data = {
                "event_type": event.event_type,
                "reason": event.reason,
                "person_name": event.person_name,
                "image_url": event.image_url,
                "confidence": event.confidence,
                "timestamp": event.timestamp.isoformat()
            }

            result = self.client.table("events").insert(data).execute()

            if result.data and len(result.data) > 0:
                return result.data[0]["id"]
            return None
        except Exception as e:
            print(f"Error saving event to database: {e}")
            return None

    async def get_all_events(self, limit: int = 100) -> List[Dict]:
        try:
            result = self.client.table("events").select("*").order("timestamp",
desc=True).limit(limit).execute()
            return result.data if result.data else []
        except Exception as e:
            print(f"Error fetching events: {e}")
            return []

    async def get_intrusions(self, limit: int = 50) -> List[Dict]:
        try:
            result = self.client.table("events").select("*").eq("event_type",
"intrusion").order("timestamp", desc=True).limit(limit).execute()
            return result.data if result.data else []
        except Exception as e:
            print(f"Error fetching intrusions: {e}")
            return []

    async def get_recognized(self, limit: int = 50) -> List[Dict]:
        try:
            result = self.client.table("events").select("*").eq("event_type",
"recognized").order("timestamp", desc=True).limit(limit).execute()
            return result.data if result.data else []
        except Exception as e:
            print(f"Error fetching recognized events: {e}")
            return []

    async def get_stats(self) -> Dict:
        try:
            intrusions_result = self.client.table("events").select("id",
count="exact").eq("event_type", "intrusion").execute()
            recognized_result = self.client.table("events").select("id",
count="exact").eq("event_type", "recognized").execute()
```

```
            total_intrusions = intrusions_result.count if intrusions_result.count else
0
            total_recognized = recognized_result.count if recognized_result.count else
0

            return {
                "total_intrusions": total_intrusions,
                "total_recognized": total_recognized
            }
        except Exception as e:
            print(f"Error fetching stats: {e}")
            return {
                "total_intrusions": 0,
                "total_recognized": 0
            }

    async def save_subscription(self, sub: PushSubscriptionCreate) -> bool:
        try:
            data = {
                "endpoint": sub.endpoint,
                "p256dh": sub.p256dh,
                "auth": sub.auth
            }
            # Upsert by endpoint to avoid duplicates
            self.client.table("push_subscriptions").upsert(data,
on_conflict="endpoint").execute()
            return True
        except Exception as e:
            print(f"Error saving subscription: {e}")
            return False

    async def get_all_subscriptions(self) -> List[Dict]:
        try:
            result = self.client.table("push_subscriptions").select("*").execute()
            return result.data if result.data else []
        except Exception as e:
            print(f"Error fetching subscriptions: {e}")
            return []

db_service = DatabaseService()
```

backend/services/detection.py

Intrusion detection orchestration logic connecting vision, storage, and notification.

```
from datetime import datetime
from services.azure_vision import azure_vision_service
from services.blob_storage import blob_storage_service
from services.database import db_service
from services.notification import notification_service
from services.email import email_service
from models.schemas import EventCreate
import asyncio
from fastapi import BackgroundTasks

class DetectionService:
    async def process_and_store_image(self, image_bytes: bytes, background_tasks:
BackgroundTasks = None):
        vision_result = await azure_vision_service.classify_mask(image_bytes)

        if vision_result is None:
            return {"status": "success", "message": "No detection from Custom Vision",
```

```
"detected": False}

        confidence = vision_result.get("confidence", 0.0)

        # Classification workflow
        if vision_result.get("has_mask", False):
            event_type = "intrusion"
            reason = "mask_detected"
            person_name = None
        elif vision_result.get("has_person", False):
            event_type = "recognized"
            reason = "person_detected"
            person_name = None
        else:
            return {"status": "success", "message": "Nothing of interest detected",
"detected": False}

        # Upload image to Blob Storage
        image_url = await blob_storage_service.upload_image(image_bytes, event_type,
reason)

        if image_url:
            # Save metadata to Supabase
            event = EventCreate(
                event_type=event_type,
                reason=reason,
                person_name=person_name,
                image_url=image_url,
                confidence=confidence,
                timestamp=datetime.now()
            )
            await db_service.save_event(event)

            if event_type == "intrusion":
                subs = await db_service.get_all_subscriptions()
                for sub in subs:
                    sub_info = {
                        "endpoint": sub["endpoint"],
                        "keys": {
                            "p256dh": sub["p256dh"],
                            "auth": sub["auth"]
                        }
                    }
                    asyncio.create_task(asyncio.to_thread(
                        notification_service.send_notification,
                        sub_info,
                        {"title": "Intrusion Alert!", "body": f"Motion detected:
{reason}"}
                    ))

                if background_tasks:
                    background_tasks.add_task(email_service.send_intrusion_email,
reason, image_url)

            return {"status": "success", "detected": True, "event_type": event_type,
"confidence": confidence}

        return {"status": "error", "message": "Failed to upload image"}

detection_service = DetectionService()
```

backend/services/email.py

Email notification service via SMTP for alerting farm owners.

```
import os
import resend

class EmailService:
    def __init__(self):
        # resend python SDK uses the resend.api_key attribute
        resend.api_key = os.environ.get("RESEND_API_KEY")
        self.to_email = os.environ.get("RESEND_TO_EMAIL", "your_email@example.com")

    def send_intrusion_email(self, reason: str, image_url: str):
        if not resend.api_key:
            print("RESEND_API_KEY not set. Cannot send email.")
            return

        print(f"Sending intrusion email to {self.to_email}...")

        try:
            params = {
                "from": "onboarding@resend.dev",
                "to": [self.to_email],
                "subject": "Intrusion Alert!",
                "html": f"<h2>Intrusion Detected</h2><p><strong>Reason:</strong>
{reason}</p><p><img src='{image_url}' alt='Intrusion Image' width='400'/></p>",
            }
            email = resend.Emails.send(params)
            print(f"Email sent successfully: {email}")
        except Exception as e:
            print(f"Failed to send email: {e}")

email_service = EmailService()
```

backend/services/notification.py

Web push notification dispatch service.

```
import os
import json
from pywebpush import webpush, WebPushException

class NotificationService:
    def __init__(self):
        self.vapid_private_key = os.environ.get("VAPID_PRIVATE_KEY")
        self.vapid_claims = {
            "sub": "mailto:admin@example.com"
        }

    def send_notification(self, subscription_info: dict, payload: dict):
        if not self.vapid_private_key:
            print("VAPID_PRIVATE_KEY not set. Cannot send push notification.")
            return

        try:
            webpush(
                subscription_info=subscription_info,
                data=json.dumps(payload),
                vapid_private_key=self.vapid_private_key,
                vapid_claims=self.vapid_claims
            )
        except WebPushException as ex:
            print(f"Web Push Error: {repr(ex)}")
```

```
notification_service = NotificationService()
```

backend/utils/helpers.py

Shared utility and helper functions used across the backend.

```
import cv2
import numpy as np
from datetime import datetime
from typing import Tuple

def format_timestamp(dt: datetime) -> str:
    return dt.strftime("%Y-%m-%d %H:%M:%S")

def resize_image(image_bytes: bytes, max_width: int = 1920, max_height: int = 1080) ->
bytes:
    nparr = np.frombuffer(image_bytes, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

    height, width = img.shape[:2]

    if width > max_width or height > max_height:
        scale = min(max_width / width, max_height / height)
        new_width = int(width * scale)
        new_height = int(height * scale)
        img = cv2.resize(img, (new_width, new_height), interpolation=cv2.INTER_AREA)

    _, buffer = cv2.imencode('.jpg', img, [cv2.IMWRITE_JPEG_QUALITY, 85])
    return buffer.tobytes()

def validate_image(image_bytes: bytes) -> Tuple[bool, str]:
    try:
        nparr = np.frombuffer(image_bytes, np.uint8)
        img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

        if img is None:
            return False, "Invalid image format"

        if img.size == 0:
            return False, "Empty image"

        return True, "Valid image"
    except Exception as e:
        return False, f"Image validation error: {str(e)}"
```

backend/requirements.txt

Python package dependencies for the backend service.

```
fastapi==0.109.0
uvicorn==0.27.0
python-dotenv==1.0.0
azure-cognitiveservices-vision-customvision==3.1.0
azure-ai-vision-face==1.0.0b1
azure-storage-blob==12.19.0
pywebpush==2.3.0
py_vapid==1.9.0
resend==0.8.0
supabase
pydantic==2.5.3
python-multipart==0.0.6
pillow==10.2.0
```

Frontend Application

File	Description
index.html	HTML shell that Vite injects the compiled React bundle into.
src/main.tsx	Application entry point; mounts the React root component into the DOM.
src/App.tsx	Root React component; fetches event and statistics data and renders the dashboard.
src/types.ts	Shared TypeScript type definitions for Event and Stats interfaces.
src/index.css	Global stylesheet imports for the application.
src/components/StatCard.tsx	Reusable statistic tile component for the dashboard summary row.
src/components/EventCard.tsx	Card component for displaying a single intrusion or recognition event.
src/components/PushNotification.tsx	Component for managing web push notification subscriptions.
src/services/api.ts	HTTP client layer; wraps all API calls made to the backend server.
src/utils/date.ts	Date formatting utility functions used across the frontend.
package.json	Project metadata, scripts, and npm dependency list.
vite.config.ts	Vite bundler configuration for the frontend build process.
tailwind.config.js	Tailwind CSS configuration defining custom design tokens.
pi_simulator.html	Standalone Pi simulator page used for testing without physical hardware.
public/service-worker.js	Service worker script for delivering web push notifications to the browser.
supabase/migrations/ 20240726000000_create_push_subscriptions.sql	Database migration: creates the

	push_subscriptions table in Supabase.
supabase/migrations/20251229233411_create_events_table.sql	Database migration: creates the events table for storing intrusion records.

Frontend Application — Source Code

`index.html`

HTML shell that Vite injects the compiled React bundle into.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/jpeg" href="/favicon.jpg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Agricultural Intrusion Detection</title>
    <meta property="og:image" content="/favicon.jpg" />
    <meta name="twitter:card" content="summary_large_image" />
    <meta name="twitter:image" content="/favicon.jpg" />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

`src/main.tsx`

Application entry point; mounts the React root component into the DOM.

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.tsx';
import './index.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

`src/App.tsx`

Root React component; fetches event and statistics data and renders the dashboard.

```
import { useState, useEffect } from 'react';
import { ShieldAlert, UserCheck, Camera, Activity } from 'lucide-react';
import { StatCard } from './components/StatCard';
import { EventCard } from './components/EventCard';
import { PushNotification } from './components/PushNotification';
import { api } from './services/api';
import type { Event, Stats } from './types';

function App() {
  const [stats, setStats] = useState<Stats>({ total_intrusions: 0, total_recognized: 0
});
  const [intrusions, setIntrusions] = useState<Event[]>([]);
  const [recognized, setRecognized] = useState<Event[]>([]);
  const [activeTab, setActiveTab] = useState<'intrusions' |
'recognized'>('intrusions');
  const [loading, setLoading] = useState(true);
  const [isCapturing, setIsCapturing] = useState(false);
  const [isOnline, setIsOnline] = useState(false);

  const fetchData = async () => {
```

```
    try {
      const [statsData, intrusionsData, recognizedData, statusData] = await
Promise.all([
        api.getStats(),
        api.getIntrusions(50),
        api.getRecognized(50),
        api.getPiStatus(),
      ]);

      setStats(statsData);
      setIntrusions(intrusionsData);
      setRecognized(recognizedData);
      setIsOnline(statusData.online);
    } catch (error) {
      console.error('Error fetching data:', error);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchData();
    const interval = setInterval(fetchData, 5000);
    return () => clearInterval(interval);
  }, []);

  const handleManualCapture = async (cameraId: number) => {
    setIsCapturing(true);
    try {
      await api.triggerManualCapture(cameraId);
      // Give the Pi ~3 seconds to take the photo and upload it
      setTimeout(async () => {
        await fetchData();
        setIsCapturing(false);
      }, 3000);
    } catch (error) {
      console.error('Error triggering capture:', error);
      setIsCapturing(false);
    }
  };

  if (loading && !stats.total_intrusions && !stats.total_recognized) {
    return (
      <div className="min-h-screen bg-gradient-to-br from-slate-50 to-slate-100 flex
items-center justify-center">
        <div className="text-center">
          <Activity className="animate-spin mx-auto mb-4" size={48} color="#3b82f6" />
          <p className="text-gray-600">Loading dashboard...</p>
        </div>
      </div>
    );
  }

  const displayEvents = activeTab === 'intrusions' ? intrusions : recognized;

  return (
    <div className="min-h-screen bg-gradient-to-br from-slate-50 to-slate-100">
      <div className="max-w-7xl mx-auto px-4 py-8">
        <div className="mb-8">
          <div className="flex items-center justify-between">
            <div>
              <h1 className="text-4xl font-bold text-gray-900 mb-2">
                Agricultural Intrusion Detection
```

```
              </h1>
              <div className="flex items-center gap-3">
                <p className="text-gray-600">Raspberry Pi Edge Monitoring
Dashboard</p>
                <span className={`px-3 py-1 rounded-full text-xs font-semibold flex
items-center gap-1 ${isOnline ? 'bg-green-100 text-green-700' : 'bg-red-100 text-red-
700'}`}>
                  <span className={`w-2 h-2 rounded-full ${isOnline ? 'bg-green-500
animate-pulse' : 'bg-red-500'}`}></span>
                  {isOnline ? 'System Online' : 'System Offline'}
                </span>
              </div>
            </div>
            <div className="flex flex-wrap gap-3 items-center">
              <PushNotification />
              <button
                onClick={() => handleManualCapture(0)}
                disabled={isCapturing}
                className={`flex items-center gap-2 px-6 py-3 rounded-lg font-semibold
transition-all ${
                  isCapturing
                    ? 'bg-blue-400 cursor-not-allowed text-white'
                    : 'bg-blue-600 hover:bg-blue-700 text-white'
                }`}
              >
                <Camera size={20} className={isCapturing ? 'animate-pulse' : ''} />
                {isCapturing ? 'Capturing...' : 'Capture (Cam 1)'}
              </button>

              <button
                onClick={() => handleManualCapture(2)}
                disabled={isCapturing}
                className={`flex items-center gap-2 px-6 py-3 rounded-lg font-semibold
transition-all ${
                  isCapturing
                    ? 'bg-indigo-400 cursor-not-allowed text-white'
                    : 'bg-indigo-600 hover:bg-indigo-700 text-white'
                }`}
              >
                <Camera size={20} className={isCapturing ? 'animate-pulse' : ''} />
                {isCapturing ? 'Capturing...' : 'Capture (Cam 2)'}
              </button>
            </div>
          </div>
        </div>

        <div className="grid grid-cols-1 md:grid-cols-2 gap-6 mb-8">
          <StatCard
            title="Total Intrusions"
            value={stats.total_intrusions}
            icon={ShieldAlert}
            color="#ef4444"
          />
          <StatCard
            title="Recognized Visits"
            value={stats.total_recognized}
            icon={UserCheck}
            color="#10b981"
          />
        </div>

        <div className="bg-white rounded-lg shadow-md p-6">
          <div className="flex gap-4 mb-6 border-b">
```

```
              <button
                onClick={() => setActiveTab('intrusions')}
                className={`pb-3 px-4 font-semibold transition-all ${
                  activeTab === 'intrusions'
                    ? 'border-b-2 border-red-500 text-red-600'
                    : 'text-gray-600 hover:text-gray-900'
                }`}
              >
                Intrusions ({intrusions.length})
              </button>
              <button
                onClick={() => setActiveTab('recognized')}
                className={`pb-3 px-4 font-semibold transition-all ${
                  activeTab === 'recognized'
                    ? 'border-b-2 border-green-500 text-green-600'
                    : 'text-gray-600 hover:text-gray-900'
                }`}
              >
                Recognized ({recognized.length})
              </button>
            </div>

            {displayEvents.length === 0 ? (
              <div className="text-center py-12">
                <Activity size={48} className="mx-auto mb-4 text-gray-400" />
                <p className="text-gray-600">
                  No {activeTab === 'intrusions' ? 'intrusions' : 'recognized visits'}
detected yet
                </p>
              </div>
            ) : (
              <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
                {displayEvents.map((event) => (
                  <EventCard key={event.id} event={event} />
                ))}
              </div>
            )}
          </div>
        </div>
      </div>
    );
}

export default App;
```

src/types.ts

Shared TypeScript type definitions for Event and Stats interfaces.

```
export interface Event {
  id: string;
  event_type: 'intrusion' | 'recognized';
  reason: 'mask_detected' | 'unknown_face' | 'recognized_face';
  person_name: string | null;
  image_url: string;
  confidence: number | null;
  timestamp: string;
  created_at: string;
}

export interface Stats {
  total_intrusions: number;
  total_recognized: number;
```

```
}
```

src/index.css

Global stylesheet imports for the application.

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

src/components/StatCard.tsx

Reusable statistic tile component for the dashboard summary row.

```
import { LucideIcon } from 'lucide-react';

interface StatCardProps {
  title: string;
  value: number;
  icon: LucideIcon;
  color: string;
}

export function StatCard({ title, value, icon: Icon, color }: StatCardProps) {
  return (
    <div className="bg-white rounded-lg shadow-md p-6 border-l-4"
style={{ borderLeftColor: color }}>
      <div className="flex items-center justify-between">
        <div>
          <p className="text-sm font-medium text-gray-600">{title}</p>
          <p className="text-3xl font-bold mt-2">{value}</p>
        </div>
        <div className="p-3 rounded-full" style={{ backgroundColor: `${color}15` }}>
          <Icon size={32} color={color} />
        </div>
      </div>
    </div>
  );
}
```

src/components/EventCard.tsx

Card component for displaying a single intrusion or recognition event.

```
import { Event } from '../types';
import { formatDistanceToNow } from '../utils/date';

interface EventCardProps {
  event: Event;
}

export function EventCard({ event }: EventCardProps) {
  const reasonLabels: Record<string, string> = {
    mask_detected: 'Face Mask Detected',
    unknown_face: 'Unknown Face',
    recognized_face: 'Recognized Person',
  };

  const getBadgeColor = () => {
    if (event.event_type === 'intrusion') {
      return 'bg-red-100 text-red-800 border-red-200';
    }
    return 'bg-green-100 text-green-800 border-green-200';
```

```
  };

  return (
    <div className="bg-white rounded-lg shadow-md overflow-hidden hover:shadow-lg
transition-shadow">
      <div className="aspect-video bg-gray-200 relative">
        <img
          src={event.image_url}
          alt={event.reason}
          className="w-full h-full object-cover"
          onError={(e) => {
            e.currentTarget.src = 'data:image/svg+xml,%3Csvg
xmlns="http://www.w3.org/2000/svg" width="400" height="300"%3E%3Crect fill="%23ddd"
width="400" height="300"/%3E%3Ctext fill="%23999" x="50%25" y="50%25" text-
anchor="middle" dy=".3em"%3EImage Not Available%3C/text%3E%3C/svg%3E';
          }}
        />
        {event.confidence && (
          <div className="absolute top-2 right-2 bg-black bg-opacity-70 text-white
text-xs px-2 py-1 rounded">
            {(event.confidence * 100).toFixed(1)}%
          </div>
        )}
      </div>
      <div className="p-4">
        <div className="flex items-start justify-between mb-2">
          <span className={`px-3 py-1 rounded-full text-xs font-semibold border $
{getBadgeColor()}`}>
            {reasonLabels[event.reason] || event.reason}
          </span>
        </div>
        {event.person_name && (
          <p className="text-lg font-semibold text-gray-900 mb-
1">{event.person_name}</p>
        )}
        <p className="text-sm text-gray-600">
          {formatDistanceToNow(event.timestamp)}
        </p>
      </div>
    </div>
  );
}
```

src/components/PushNotification.tsx

Component for managing web push notification subscriptions.

```
import React, { useState, useEffect } from 'react';
import { Bell, BellOff } from 'lucide-react';
import { api } from '../services/api';

// Replace or set via .env
const PUBLIC_VAPID_KEY = import.meta.env.VITE_VAPID_PUBLIC_KEY ||
'BJ8wIrcS9nWFUGDUXfCp7wE2H4IALuUy4Xhj-A8bwvVpcGcVKM9O1GJoR_4BhttOWrNaWH-5ccqcEbDrxhE-
M_Q';

function urlBase64ToUint8Array(base64String: string) {
  const padding = '='.repeat((4 - base64String.length % 4) % 4);
  const base64 = (base64String + padding)
    .replace(/-/g, '+')
    .replace(/_/g, '/');

  const rawData = window.atob(base64);
```

```
  const outputArray = new Uint8Array(rawData.length);

  for (let i = 0; i < rawData.length; ++i) {
    outputArray[i] = rawData.charCodeAt(i);
  }
  return outputArray;
}

export function PushNotification() {
  const [isSubscribed, setIsSubscribed] = useState(false);
  const [isSupported, setIsSupported] = useState(false);

  useEffect(() => {
    if ('serviceWorker' in navigator && 'PushManager' in window) {
      setIsSupported(true);

      navigator.serviceWorker.register('/service-worker.js')
        .then(registration => {
          return registration.pushManager.getSubscription();
        })
        .then(subscription => {
          setIsSubscribed(subscription !== null);
        })
        .catch(err => console.error("Service Worker error:", err));
    }
  }, []);

  const subscribe = async () => {
    try {
      const permission = await Notification.requestPermission();
      if (permission !== 'granted') {
        alert('Notification permission denied.');
        return;
      }

      const registration = await navigator.serviceWorker.ready;
      const subscription = await registration.pushManager.subscribe({
        userVisibleOnly: true,
        applicationServerKey: urlBase64ToUint8Array(PUBLIC_VAPID_KEY)
      });

      const subJSON = subscription.toJSON();
      await api.subscribePush({
        endpoint: subJSON.endpoint,
        p256dh: subJSON.keys?.p256dh,
        auth: subJSON.keys?.auth
      });

      setIsSubscribed(true);
      alert('Successfully subscribed to push notifications!');
    } catch (err) {
      console.error('Failed to subscribe:', err);
      alert('Failed to subscribe. Check console for details.');
    }
  };

  if (!isSupported) return null;

  return (
    <button
      onClick={subscribe}
      disabled={isSubscribed}
```

```
      className={`flex items-center gap-2 px-4 py-2 rounded-lg font-semibold
transition-all ${
        isSubscribed
          ? 'bg-green-100 text-green-700 cursor-not-allowed'
          : 'bg-yellow-100 text-yellow-700 hover:bg-yellow-200'
      }`}
    >
      {isSubscribed ? <Bell size={20} /> : <BellOff size={20} />}
      <span className="hidden sm:inline">{isSubscribed ? 'Notifications Enabled' :
'Enable Notifications'}</span>
    </button>
  );
}
```

src/services/api.ts

HTTP client layer; wraps all API calls made to the backend server.

```
import type { Event, Stats } from '../types';

// Read API URL from Vercel environment variable, fallback to local/codespace URL
const API_BASE_URL = import.meta.env.VITE_API_URL || 'https://vigilant-space-telegram-
9wjjr7pj6wxc6w-8000.app.github.dev/api';

export const api = {
  async getStats(): Promise<Stats> {
    const response = await fetch(`${API_BASE_URL}/stats`);
    if (!response.ok) throw new Error('Failed to fetch stats');
    return response.json();
  },

  async getEvents(limit: number = 100): Promise<Event[]> {
    const response = await fetch(`${API_BASE_URL}/events?limit=${limit}`);
    if (!response.ok) throw new Error('Failed to fetch events');
    return response.json();
  },

  async getIntrusions(limit: number = 50): Promise<Event[]> {
    const response = await fetch(`${API_BASE_URL}/intrusions?limit=${limit}`);
    if (!response.ok) throw new Error('Failed to fetch intrusions');
    return response.json();
  },

  async getRecognized(limit: number = 50): Promise<Event[]> {
    const response = await fetch(`${API_BASE_URL}/recognized?limit=${limit}`);
    if (!response.ok) throw new Error('Failed to fetch recognized events');
    return response.json();
  },

  async triggerManualCapture(cameraId: number = 0): Promise<{ status: string; message:
string }> {
    // We add ?camera_id= to the URL so FastAPI knows which one you want
    const response = await fetch(`${API_BASE_URL}/pi/capture?camera_id=${cameraId}`, {
      method: 'POST',
    });
    if (!response.ok) throw new Error(`Failed to trigger Pi capture for Camera $
{cameraId}`);
    return response.json();
  },

  async subscribePush(subscription: any): Promise<{ status: string }> {
    const response = await fetch(`${API_BASE_URL}/subscribe`, {
      method: 'POST',
```

```
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(subscription)
    });
    if (!response.ok) throw new Error('Failed to subscribe');
    return response.json();
  },

  async getPiStatus(): Promise<{ online: boolean; connections: number }> {
    const response = await fetch(`${API_BASE_URL}/pi/status`);
    if (!response.ok) throw new Error('Failed to fetch Pi status');
    return response.json();
  }
};
```

`src/utils/date.ts`

Date formatting utility functions used across the frontend.

```
export function formatDistanceToNow(dateString: string): string {
  const date = new Date(dateString);
  const now = new Date();
  const diffInSeconds = Math.floor((now.getTime() - date.getTime()) / 1000);

  if (diffInSeconds < 60) {
    return 'just now';
  }

  const diffInMinutes = Math.floor(diffInSeconds / 60);
  if (diffInMinutes < 60) {
    return `${diffInMinutes} ${diffInMinutes === 1 ? 'minute' : 'minutes'} ago`;
  }

  const diffInHours = Math.floor(diffInMinutes / 60);
  if (diffInHours < 24) {
    return `${diffInHours} ${diffInHours === 1 ? 'hour' : 'hours'} ago`;
  }

  const diffInDays = Math.floor(diffInHours / 24);
  if (diffInDays < 30) {
    return `${diffInDays} ${diffInDays === 1 ? 'day' : 'days'} ago`;
  }

  const diffInMonths = Math.floor(diffInDays / 30);
  return `${diffInMonths} ${diffInMonths === 1 ? 'month' : 'months'} ago`;
}

export function formatDateTime(dateString: string): string {
  const date = new Date(dateString);
  return date.toLocaleString('en-US', {
    year: 'numeric',
    month: 'short',
    day: 'numeric',
    hour: '2-digit',
    minute: '2-digit',
  });
}
```

`package.json`

Project metadata, scripts, and npm dependency list.

```
{
  "name": "vite-react-typescript-starter",
```

```
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview",
    "typecheck": "tsc --noEmit -p tsconfig.app.json"
  },
  "dependencies": {
    "@supabase/supabase-js": "^2.57.4",
    "lucide-react": "^0.344.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@eslint/js": "^9.9.1",
    "@types/react": "^18.3.5",
    "@types/react-dom": "^18.3.0",
    "@vitejs/plugin-react": "^4.3.1",
    "autoprefixer": "^10.4.18",
    "eslint": "^9.9.1",
    "eslint-plugin-react-hooks": "^5.1.0-rc.0",
    "eslint-plugin-react-refresh": "^0.4.11",
    "globals": "^15.9.0",
    "postcss": "^8.4.35",
    "tailwindcss": "^3.4.1",
    "typescript": "^5.5.3",
    "typescript-eslint": "^8.3.0",
    "vite": "^5.4.2"
  }
}
```

`vite.config.ts`

Vite bundler configuration for the frontend build process.

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [react()],
  optimizeDeps: {
    exclude: ['lucide-react'],
  },
});
```

`tailwind.config.js`

Tailwind CSS configuration defining custom design tokens.

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
  theme: {
    extend: {},
  },
  plugins: [],
};
```

pi_simulator.html

Standalone Pi simulator page used for testing without physical hardware.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Raspberry Pi Simulator</title>
    <style>
        body { font-family: system-ui, -apple-system, sans-serif; max-width: 800px;
margin: 2rem auto; padding: 0 1rem; background: #f8fafc; color: #334155; }
        .card { background: white; padding: 1.5rem; border-radius: 8px; box-shadow: 0
1px 3px rgba(0,0,0,0.1); margin-bottom: 1.5rem; }
        h1 { color: #0f172a; }
        h2 { margin-top: 0; font-size: 1.2rem; }
        input[type="text"], input[type="file"] { width: 100%; padding: 0.5rem; margin-
bottom: 1rem; border: 1px solid #cbd5e1; border-radius: 4px; box-sizing: border-box; }
        button { background: #3b82f6; color: white; border: none; padding: 0.5rem
1rem; border-radius: 4px; cursor: pointer; font-weight: 500; }
        button:hover { background: #2563eb; }
        button.danger { background: #ef4444; }
        button.danger:hover { background: #dc2626; }
        .log { background: #1e293b; color: #10b981; padding: 1rem; border-radius: 4px;
font-family: monospace; height: 300px; overflow-y: auto; white-space: pre-wrap; }
        .status { display: inline-block; padding: 0.25rem 0.5rem; border-radius:
999px; font-size: 0.875rem; font-weight: bold; background: #e2e8f0; color: #64748b;
margin-left: 1rem; }
        .status.online { background: #dcfce7; color: #166534; }
    </style>
</head>
<body>
    <h1>Raspberry Pi Simulator</h1>
    <p>Use this page to simulate the Pi hardware for testing the backend and
dashboard.</p>

    <div class="card">
        <h2>1. Connection Settings</h2>
        <label>Backend API URL</label>
        <input type="text" id="apiUrl" value="http://localhost:8000">

        <div style="display: flex; align-items: center;">
            <button onclick="connectStream()">Connect SSE Stream</button>
            <button onclick="disconnectStream()" class="danger" style="margin-left:
0.5rem;">Disconnect</button>
            <span id="statusBadge" class="status">Offline</span>
        </div>
        <p style="font-size: 0.875rem; color: #64748b; margin-top:
0.75rem;">Connecting will turn the Dashboard's "System Online" indicator green.</p>
    </div>

    <div class="card">
        <h2>2. Simulate Camera Capture</h2>
        <p style="font-size: 0.875rem; color: #64748b;">Upload a test image (e.g.,
masked face or known person) to trigger the detection pipeline, Supabase logging, and
Push/Email alerts.</p>
        <input type="file" id="imageFile" accept="image/*">
        <button onclick="sendImage(this)">Send Image to /detect</button>
    </div>

    <div class="card">
        <h2>Event Log</h2>
        <div id="log" class="log"></div>
```

```
    </div>

    <script>
        let eventSource = null;

        function logMessage(msg, type="INFO") {
            const logDiv = document.getElementById('log');
            const time = new Date().toLocaleTimeString();
            logDiv.innerHTML += `[${time}] [${type}] ${msg}\n`;
            logDiv.scrollTop = logDiv.scrollHeight;
        }

        function connectStream() {
            if (eventSource) disconnectStream();

            const url = document.getElementById('apiUrl').value;
            logMessage(`Connecting to ${url}/api/pi/stream...`);

            eventSource = new EventSource(`${url}/api/pi/stream`);

            eventSource.onopen = () => {
                logMessage('SSE Connection established.');
                const badge = document.getElementById('statusBadge');
                badge.textContent = 'Online';
                badge.className = 'status online';
            };

            eventSource.addEventListener('command', (e) => {
                logMessage(e.data, "COMMAND_RECEIVED");
                // Simulate taking a photo if it's a capture command
                try {
                    const data = JSON.parse(e.data);
                    if (data.type === 'capture') {
                        logMessage(`Dashboard requested capture for Camera $
{data.payload.camera_id}. Use the 'Simulate Camera Capture' section above to upload a
file.`);
                    }
                } catch(err) {}
            });

            eventSource.onerror = (err) => {
                logMessage('SSE Connection lost or failed.', "ERROR");
                disconnectStream();
            };
        }

        function disconnectStream() {
            if(eventSource) {
                eventSource.close();
                eventSource = null;
            }
            const badge = document.getElementById('statusBadge');
            badge.textContent = 'Offline';
            badge.className = 'status';
            logMessage('Disconnected from SSE stream.');
        }

        async function sendImage(btn) {
            const fileInput = document.getElementById('imageFile');
            if (!fileInput.files || fileInput.files.length === 0) {
                return alert('Please select an image file first.');
            }
```

```
            const file = fileInput.files[0];
            const url = document.getElementById('apiUrl').value;
            const formData = new FormData();
            formData.append('file', file);

            logMessage(`Uploading ${file.name} to /detect...`);
            btn.disabled = true;
            btn.textContent = 'Uploading...';

            try {
                const res = await fetch(`${url}/api/detect`, {
                    method: 'POST',
                    body: formData
                });
                const data = await res.json();
                logMessage(JSON.stringify(data, null, 2), "DETECT_RESPONSE");
            } catch (e) {
                logMessage(e.message, "ERROR");
            } finally {
                btn.disabled = false;
                btn.textContent = 'Send Image to /detect';
            }
        }
    </script>
</body>
</html>
```

public/service-worker.js

Service worker script for delivering web push notifications to the browser.

```
self.addEventListener('push', function(event) {
  const data = event.data ? event.data.json() : {};
  const title = data.title || 'Intrusion Alert';
  const options = {
    body: data.body || 'Motion detected by the sensor.',
    icon: '/vite.svg',
    badge: '/vite.svg',
    vibrate: [200, 100, 200, 100, 200, 100, 200]
  };

  event.waitUntil(
    self.registration.showNotification(title, options)
  );
});

self.addEventListener('notificationclick', function(event) {
  event.notification.close();
  event.waitUntil(
    clients.matchAll({ type: 'window' }).then(windowClients => {
      // Check if there is already a window/tab open with the target URL
      for (let i = 0; i < windowClients.length; i++) {
        let client = windowClients[i];
        if (client.url === '/' && 'focus' in client) {
          return client.focus();
        }
      }
      // If not, open a new window
      if (clients.openWindow) {
        return clients.openWindow('/');
      }
```

```
    })
  );
});
```

supabase/migrations/20240726000000_create_push_subscriptions.sql

Database migration: creates the push_subscriptions table in Supabase.

```
-- Migration: Create Push Subscriptions Table
CREATE TABLE public.push_subscriptions (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    endpoint TEXT NOT NULL UNIQUE,
    p256dh TEXT NOT NULL,
    auth TEXT NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

-- Enable RLS
ALTER TABLE public.push_subscriptions ENABLE ROW LEVEL SECURITY;

-- Grant access to roles
GRANT ALL ON TABLE public.push_subscriptions TO postgres, anon, authenticated,
service_role;

-- Add RLS Policies
CREATE POLICY "Allow public full access to push_subscriptions" ON
public.push_subscriptions
    FOR ALL TO public
    USING (true)
    WITH CHECK (true);
```

supabase/migrations/20251229233411_create_events_table.sql

Database migration: creates the events table for storing intrusion records.

```
/*
  # Create events table for intrusion detection system

  1. New Tables
    - `events`
      - `id` (uuid, primary key) - Unique event identifier
      - `event_type` (text) - Type of event: 'intrusion' or 'recognized'
      - `reason` (text) - Reason for classification: 'mask_detected', 'unknown_face',
or 'recognized_face'
      - `person_name` (text, nullable) - Name of recognized person (null for
intrusions)
      - `image_url` (text) - Azure Blob Storage URL for the captured image
      - `confidence` (numeric, nullable) - Confidence score from Azure AI
      - `timestamp` (timestamptz) - When the event occurred
      - `created_at` (timestamptz) - Record creation timestamp

  2. Security
    - Enable RLS on `events` table
    - Add policy for public read access (no auth required for dashboard)
    - Add policy for service role to insert events
*/

CREATE TABLE IF NOT EXISTS events (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  event_type text NOT NULL CHECK (event_type IN ('intrusion', 'recognized')),
  reason text NOT NULL CHECK (reason IN ('mask_detected', 'unknown_face',
'recognized_face')),
  person_name text,
```

```
  image_url text NOT NULL,
  confidence numeric,
  timestamp timestamptz DEFAULT now(),
  created_at timestamptz DEFAULT now()
);

ALTER TABLE events ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Allow public read access to events"
  ON events FOR SELECT
  TO anon
  USING (true);

CREATE POLICY "Allow service role to insert events"
  ON events FOR INSERT
  TO service_role
  WITH CHECK (true);

CREATE INDEX IF NOT EXISTS idx_events_event_type ON events(event_type);
CREATE INDEX IF NOT EXISTS idx_events_timestamp ON events(timestamp DESC);
```

APPENDIX C

RASPBERRY PI CODE

Source Code Listing

This appendix lists the complete source code for the Raspberry Pi edge node of the Farm Intrusion Detection System (FAIDS). The edge agent is written in Python and runs on a Raspberry Pi as a systemd service. It manages PIR motion sensors, camera capture, LED/siren actuators, cloud image uploads, and a Server-Sent Events (SSE) command listener for receiving remote commands from the cloud backend.

Raspberry Pi Edge Node

File	Description
`main.py`	Hardware node entry point; boots all subsystems and handles SIGINT/SIGTERM for graceful shutdown.
`config.py`	Hardware GPIO pin assignments, cloud API endpoints, and system-wide configuration constants.
`sensors/camera.py`	Camera capture and frame management module using the Raspberry Pi camera interface.
`sensors/pir.py`	PIR motion sensor monitoring event loop; triggers the detection pipeline on motion.
`actuators/led.py`	LED and siren actuator controller for active physical deterrence.
`api/command_listener.py`	SSE command listener; receives remote commands from the cloud backend.
`api/uploader.py`	Uploads captured images and structured event data to the cloud backend API.
`core/event_handler.py`	Event routing logic connecting sensors, actuators, and the cloud upload pipeline.
`core/command_processor.py`	Processes incoming remote commands received through the SSE listener.
`db/queue_store.py`	Persistent local SQLite queue enabling offline-tolerant, retry-safe image upload.
`utils/logger.py`	Structured logging utility providing consistent log formatting across all modules.
`agri_ids.service`	systemd unit file for running the Raspberry Pi agent as a persistent background service.
`requirements.txt`	Python package dependencies required on the Raspberry Pi edge node.

Raspberry Pi Edge Node — Source Code

`main.py`

Hardware node entry point; boots all subsystems and handles SIGINT/SIGTERM for graceful shutdown.

```
#!/usr/bin/env python3
# ============================================================================
#  main.py — Agri-IDS hardware node entry point
#
#  Boot sequence
#  ─────────────
#
#  1. Instantiate hardware modules (cameras, siren)
#  2. Instantiate DB queue store (recovers any interrupted jobs automatically)
#  3. Instantiate service modules (uploader, SSE command listener)
#  4. Wire together via EventHandler and CommandProcessor
#  5. Start PIR monitoring threads
#  6. Start SSE command listener thread
#  7. Block on a signal-aware idle loop
#  8. On SIGINT / SIGTERM: graceful shutdown
#
#  Run as
#  ──────
#      sudo python3 main.py
# ============================================================================

import os
import signal
import sys
import time


import config
from sensors   import PIRSensor, CameraManager
from actuators import LEDController
from db        import QueueStore
from api       import EventUploader, CommandListener
from core      import EventHandler, CommandProcessor
from utils     import get_logger

log = get_logger("agri_ids.main")


def _ensure_dirs() -> None:
    """Create required runtime directories if they don't exist."""
    for path in [
        config.IMAGE_SAVE_DIR,
        os.path.dirname(config.DB_PATH),
    ]:
        if path:
            os.makedirs(path, exist_ok=True)



def shutdown(
    pir_sensors: list,
    led: LEDController,
    cameras: CameraManager,
    uploader: EventUploader,
    cmd_listener: CommandListener,
) -> None:
```

```
    """Ordered graceful shutdown — safe for SIGINT and SIGTERM."""
    log.info("── Shutdown initiated ──────────────────────────────────────")

    # 1. Stop accepting new PIR events
    for sensor in pir_sensors:
        sensor.stop()

    # 2. Stop the SSE command stream
    cmd_listener.stop()

    # 3. Drain the upload queue — give it up to 20 s to finish in-flight jobs
    #    Any remaining pending jobs are already in SQLite and will resume on
    #    the next boot.
    uploader.shutdown(timeout=20.0)

    # 4. Safe actuator state
    led.cleanup()

    # 5. Release camera resources
    cameras.release_all()


    log.info("── Shutdown complete ───────────────────────────────────────")


def main() -> None:
    log.info("═══════════════════════════════════════════════════════════")
    log.info("  Agri-IDS  |  Node: %s", config.NODE_ID)
    log.info("  PIR pins : %s", config.PIR_PINS)
    log.info("  Cameras  : %s", config.CAMERA_INDICES)
    log.info("  Relay pin: GPIO %d", config.RELAY_PIN)
    log.info("  API URL  : %s", config.API_BASE_URL)
    log.info("  DB path  : %s", config.DB_PATH)
    log.info("═══════════════════════════════════════════════════════════")

    _ensure_dirs()

    # ── Hardware modules ──────────────────────────────────────────────
    cameras = CameraManager()
    led  = LEDController()

    # ── Persistent queue (recovers interrupted jobs on boot) ──────────
    queue_store = QueueStore()

    pending = queue_store.pending_count()
    if pending:
        log.info("Boot recovery: %d job(s) pending in queue — will resume upload",
pending)

    # ── Service modules ───────────────────────────────────────────────
    uploader     = EventUploader(queue_store=queue_store)
    cmd_listener = CommandListener()

    # ── Core wiring ───────────────────────────────────────────────────
    event_handler = EventHandler(camera_manager=cameras, uploader=uploader,
led_controller=led)
    cmd_processor = CommandProcessor(
        event_handler=event_handler
    )
    cmd_processor.register_all(cmd_listener)

    # ── PIR sensors ───────────────────────────────────────────────────
    pir_sensors = []
```

```
    for sensor_id, pin in enumerate(config.PIR_PINS):
        sensor = PIRSensor(
            sensor_id=sensor_id,
            pin=pin,
            on_motion=event_handler.on_pir_motion,
        )
        pir_sensors.append(sensor)

    for sensor in pir_sensors:
        sensor.start()

    # ── SSE command listener ──────────────────────────────────────
    cmd_listener.start()

    log.info("System ONLINE — waiting for events …")

    # ── Signal handlers ───────────────────────────────────────────
    stop_flag = [False]

    def _handle_signal(signum, frame):  # noqa: ANN001
        log.info("Signal %d received — initiating shutdown …", signum)
        stop_flag[0] = True

    signal.signal(signal.SIGINT,  _handle_signal)
    signal.signal(signal.SIGTERM, _handle_signal)

    # ── Idle loop ─────────────────────────────────────────────────
    try:
        while not stop_flag[0]:
            time.sleep(0.5)
    finally:
        shutdown(
            pir_sensors=pir_sensors,
            led=led,
            cameras=cameras,
            uploader=uploader,
            cmd_listener=cmd_listener,
        )

    sys.exit(0)


if __name__ == "__main__":
    main()
```

config.py

Hardware GPIO pin assignments, cloud API endpoints, and system-wide configuration constants.

```
# ============================================================================
#  config.py — Central configuration for the Agri-IDS hardware node
#  Edit this file to match your wiring and backend API deployment.
# ============================================================================

import os
from typing import List

# ---------------------------------------------------------------------------
# GPIO Pin assignments  (BCM numbering)
# ---------------------------------------------------------------------------
PIR_PINS: List[int] = [17, 27]          # PIR sensor 1 → GPIO17, sensor 2 → GPIO27

# ---------------------------------------------------------------------------
```

```
# Camera indices (OpenCV VideoCapture index)
# ---------------------------------------------------------------------------
CAMERA_INDICES: List[int] = [0, 2]      # /dev/video0, /dev/video2

# Camera capture resolution
CAMERA_WIDTH: int  = 1280
CAMERA_HEIGHT: int = 720

# How long (seconds) to keep a camera open after the last capture before
# releasing it, to avoid hammering USB bandwidth.
CAMERA_RELEASE_TIMEOUT: int = 30

# ---------------------------------------------------------------------------
# PIR / Motion detection behaviour
# ---------------------------------------------------------------------------
# Minimum seconds between two consecutive PIR-triggered captures on the
# same sensor (debounce / cooldown).
PIR_COOLDOWN_SECONDS: int = 10

# ---------------------------------------------------------------------------
# LED Behaviour
# ---------------------------------------------------------------------------
LED_PIN: int = 22
LED_BLINK_DURATION: int = 3

# ---------------------------------------------------------------------------
# API settings
# ---------------------------------------------------------------------------
API_BASE_URL: str = os.getenv("AGRI_IDS_API_URL", "https://vigilant-space-telegram-
9wjjr7pj6wxc6w-8000.app.github.dev/api")
API_KEY: str      = os.getenv("AGRI_IDS_API_KEY",  "REPLACE_ME")

# Node identity — helps the backend distinguish multiple deployed units
NODE_ID: str = os.getenv("AGRI_IDS_NODE_ID", "node-001")

# Endpoints (relative to API_BASE_URL)
ENDPOINT_EVENT_UPLOAD: str = "/detect"          # POST — upload image + metadata
ENDPOINT_SSE_STREAM:   str = "/pi/stream"   # GET  — SSE command stream
ENDPOINT_ACK_COMMAND:  str = "/commands/{id}/ack" # POST — acknowledge a command

# Request timeout for regular HTTP calls (connect_timeout, read_timeout)
REQUEST_TIMEOUT: tuple = (5, 15)

# SSE stream read timeout — how long to wait for the next event before
# treating the connection as stale and reconnecting.
SSE_READ_TIMEOUT: int = 60  # seconds

# ---------------------------------------------------------------------------
# SSE reconnection backoff
# ---------------------------------------------------------------------------
SSE_RECONNECT_BASE_DELAY: float = 2.0   # Initial delay (seconds) after a disconnect
SSE_RECONNECT_MAX_DELAY:  float = 30.0  # Cap on reconnect delay (seconds)
SSE_RECONNECT_BACKOFF_FACTOR: float = 2.0  # Multiplier applied on each failure

# ---------------------------------------------------------------------------
# Upload — persistent queue & retry
# ---------------------------------------------------------------------------
UPLOAD_MAX_RETRIES: int = 5         # Attempts before a job is marked failed

# Exponential backoff for upload retries (also used when offline)
UPLOAD_BACKOFF_BASE:   float = 2.0  # seconds for first retry
UPLOAD_BACKOFF_MAX:    float = 300.0  # cap at 5 minutes between retries
UPLOAD_BACKOFF_FACTOR: float = 2.0  # multiplier per attempt
```

```
# How long (seconds) to keep failed job rows before purging them from the DB.
# Done rows are deleted immediately on success.
UPLOAD_FAILED_RETENTION_SECONDS: int = 3600  # 1 hour

# ---------------------------------------------------------------------------
# Image storage (local buffer before upload)
# ---------------------------------------------------------------------------
IMAGE_SAVE_DIR: str = os.getenv("AGRI_IDS_IMAGE_DIR", "/tmp/agri_ids/captures")

# If True, keep a local JPEG copy after a successful upload
KEEP_LOCAL_COPY: bool = False

# JPEG quality for saved captures (0-100)
JPEG_QUALITY: int = 85

# ---------------------------------------------------------------------------
# SQLite persistent queue
# ---------------------------------------------------------------------------
DB_PATH: str = os.getenv("AGRI_IDS_DB_PATH", "/var/lib/agri_ids/queue.db")

# ---------------------------------------------------------------------------
# Logging
# ---------------------------------------------------------------------------
LOG_LEVEL: str        = os.getenv("AGRI_IDS_LOG_LEVEL", "INFO")
LOG_FILE: str         = "/var/log/agri_ids.log"   # set "" to disable file logging
LOG_MAX_BYTES: int    = 5 * 1024 * 1024            # 5 MB per file
LOG_BACKUP_COUNT: int = 3
```

sensors/camera.py

Camera capture and frame management module using the Raspberry Pi camera interface.

```
# ============================================================================
#  sensors/camera.py — USB camera handler (OpenCV)
#
#  CameraManager owns all camera captures and exposes a thread-safe
#  capture_image() method.  Cameras are opened lazily on first capture
#  and released automatically after an idle timeout to free USB bandwidth.
#
#  Fixes applied:
#   • All OpenCV operations wrapped in try/except — USB disconnects are
#     caught and logged; the calling thread always gets a clean return value
#   • Reaper loop uses a stop event instead of while True — no race against
#     release_all() during shutdown
# ============================================================================

import os
import threading
import time
from datetime import datetime
from typing import Dict, Optional, Tuple

import cv2

import config
from utils import get_logger

log = get_logger(__name__)


class _CameraHandle:
    """Internal wrapper around a single cv2.VideoCapture."""
```

```
def __init__(self, index: int) -> None:
    self.index = index
    self.cap: Optional[cv2.VideoCapture] = None
    self.last_used: float = 0.0
    self.lock = threading.Lock()

def open(self) -> bool:
    if self.cap and self.cap.isOpened():
        return True

    log.info("Opening camera %d (device /dev/video%d)", self.index, self.index)
    try:
        cap = cv2.VideoCapture(self.index, cv2.CAP_V4L2)
        if not cap.isOpened():
            log.error("Camera %d: device did not open", self.index)
            return False

        cap.set(cv2.CAP_PROP_FRAME_WIDTH,  config.CAMERA_WIDTH)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, config.CAMERA_HEIGHT)
        # Give the sensor time to auto-expose
        time.sleep(0.5)
        self.cap = cap
        return True

    except Exception as exc:
        log.error("Camera %d: exception during open: %s", self.index, exc)
        self.cap = None
        return False

def grab_and_read(self) -> Tuple[bool, Optional[object]]:
    """
    Flush stale buffered frames then read a fresh one.
    Returns (success, frame).  Catches any OpenCV exception so the
    caller never sees a crash from a USB disconnect mid-capture.
    """
    try:
        # Discard buffered frames from the driver queue
        for _ in range(3):
            self.cap.grab()
        ret, frame = self.cap.read()
        return ret, frame if ret else None
    except Exception as exc:
        log.error("Camera %d: exception during capture: %s", self.index, exc)
        # Force a re-open on the next attempt
        self._force_release()
        return False, None

def release(self) -> None:
    if self.cap:
        log.info("Releasing camera %d", self.index)
        self._force_release()

def _force_release(self) -> None:
    """Release without guarding on self.cap being truthy."""
    try:
        if self.cap:
            self.cap.release()
    except Exception as exc:
        log.warning("Camera %d: exception during release: %s", self.index, exc)
    finally:
        self.cap = None
```

```
    @property
    def is_open(self) -> bool:
        return self.cap is not None and self.cap.isOpened()


class CameraManager:
    """
    Thread-safe manager for multiple USB cameras.

    Usage
    -----
    manager = CameraManager()
    filepath, metadata = manager.capture_image(camera_id=0, reason="pir_trigger")
    """

    def __init__(self) -> None:
        self._handles: Dict[int, _CameraHandle] = {
            idx: _CameraHandle(idx) for idx in config.CAMERA_INDICES
        }
        self._stop_event = threading.Event()

        # Background thread to release idle cameras
        self._reaper = threading.Thread(
            target=self._reaper_loop,
            name="camera-reaper",
            daemon=True,
        )
        self._reaper.start()

        # Ensure the image save directory exists
        os.makedirs(config.IMAGE_SAVE_DIR, exist_ok=True)

    # ------------------------------------------------------------------
    # Public API
    # ------------------------------------------------------------------

    def capture_image(
        self,
        camera_id: int,
        reason: str = "manual",
        sensor_id: Optional[int] = None,
    ) -> Tuple[Optional[str], dict]:
        """
        Capture a single frame from *camera_id* and save it to disk.

        Returns
        -------
        filepath : str | None
            Absolute path to the saved JPEG, or None on failure.
        metadata : dict
            Event metadata to accompany the upload.
        """
        if camera_id not in self._handles:
            log.error("Camera ID %d is not configured", camera_id)
            return None, {}

        handle = self._handles[camera_id]

        with handle.lock:
            if not handle.open():
                return None, {}

            # Mark as active BEFORE the operation — this prevents the reaper
```

```
            # from treating a slow or hung capture as an idle camera and
            # racing to release it mid-operation.
            handle.last_used = time.time()
            ret, frame = handle.grab_and_read()
            handle.last_used = time.time()  # update again with actual finish time

        if not ret or frame is None:
            log.error("Camera %d: failed to capture frame", camera_id)
            return None, {}

        # ------------------------------------------------------------------
        # Save the frame
        # ------------------------------------------------------------------
        timestamp = datetime.utcnow()
        filename  = (
            f"cam{camera_id}_{timestamp.strftime('%Y%m%d_%H%M%S_%f')}.jpg"
        )
        filepath = os.path.join(config.IMAGE_SAVE_DIR, filename)

        try:
            encode_params = [cv2.IMWRITE_JPEG_QUALITY, config.JPEG_QUALITY]
            success = cv2.imwrite(filepath, frame, encode_params)
        except Exception as exc:
            log.error("Camera %d: exception writing image: %s", camera_id, exc)
            return None, {}

        if not success:
            log.error("Camera %d: cv2.imwrite returned False for %s", camera_id,
filepath)
            return None, {}

        log.info("Camera %d: captured → %s  (reason=%s)", camera_id, filepath, reason)

        metadata = {
            "node_id":    config.NODE_ID,
            "camera_id":  camera_id,
            "sensor_id":  sensor_id,
            "reason":     reason,
            "timestamp":  timestamp.isoformat() + "Z",
            "resolution": f"{config.CAMERA_WIDTH}x{config.CAMERA_HEIGHT}",
        }
        return filepath, metadata

    def release_all(self) -> None:
        """
        Stop the reaper first, then release every open camera.
        Stopping the reaper before touching handles prevents any race
        between the reaper waking up and this method releasing handles.
        """
        self._stop_event.set()
        for handle in self._handles.values():
            with handle.lock:
                handle.release()

    # ------------------------------------------------------------------
    # Internal
    # ------------------------------------------------------------------

    def _reaper_loop(self) -> None:
        """
        Periodically release cameras that have been idle too long.
        Uses stop_event.wait() instead of time.sleep() so it wakes
        immediately when release_all() signals shutdown.
```

```
        """
        while not self._stop_event.wait(timeout=10):
            now = time.time()
            for handle in self._handles.values():
                with handle.lock:
                    if handle.is_open:
                        idle = now - handle.last_used
                        if idle > config.CAMERA_RELEASE_TIMEOUT:
                            log.debug(
                                "Camera %d idle for %.0fs — releasing",
                                handle.index,
                                idle,
                            )
                            handle.release()
```

sensors/pir.py

PIR motion sensor monitoring event loop; triggers the detection pipeline on motion.

```
# ============================================================================
#  sensors/pir.py — PIR motion sensor handler (gpiozero implementation)
# ============================================================================

import time
from typing import Callable

from gpiozero import MotionSensor

import config
from utils import get_logger

log = get_logger(__name__)


class PIRSensor:
    """
    Represents a single PIR motion sensor connected to a GPIO pin using gpiozero.
    """

    def __init__(
        self,
        sensor_id: int,
        pin: int,
        on_motion: Callable[[int], None],
        cooldown: int = config.PIR_COOLDOWN_SECONDS,
    ) -> None:
        self.sensor_id = sensor_id
        self.pin = pin
        self._on_motion_callback = on_motion
        self._cooldown = cooldown

        self._last_trigger: float = 0.0
        self._active = False
        self._sensor = None

        log.info("PIR sensor %d initialized on GPIO %d (gpiozero)", self.sensor_id,
self.pin)

    # ------------------------------------------------------------------
    # Public API
    # ------------------------------------------------------------------

    def start(self) -> None:
```

```
        """Initialize the sensor and attach the motion callback."""
        if self._active:
            log.warning("PIR sensor %d is already running", self.sensor_id)
            return

        # Initialize the MotionSensor (pull_up=False matches the old PUD_DOWN
behavior)
        self._sensor = MotionSensor(self.pin, pull_up=False)

        # Attach the callback natively via gpiozero
        self._sensor.when_motion = self._handle_motion

        self._active = True
        log.info("PIR sensor %d monitoring started", self.sensor_id)

    def stop(self) -> None:
        """Detach the callback and release the GPIO pin."""
        self._active = False
        if self._sensor:
            self._sensor.close()
            self._sensor = None
        log.info("PIR sensor %d monitoring stopped", self.sensor_id)

    @property
    def is_active(self) -> bool:
        return self._active

    # ------------------------------------------------------------------
    # Internal
    # ------------------------------------------------------------------

    def _handle_motion(self) -> None:
        """
        Invoked automatically by gpiozero when the pin goes HIGH.
        Applies the cooldown logic before dispatching the user callback.
        """
        if not self._active:
            return

        now = time.time()
        elapsed = now - self._last_trigger

        if elapsed < self._cooldown:
            log.debug(
                "PIR sensor %d: motion suppressed (cooldown %.1fs remaining)",
                self.sensor_id,
                self._cooldown - elapsed,
            )
            return

        self._last_trigger = now
        log.info(
            "PIR sensor %d: MOTION DETECTED on GPIO %d",
            self.sensor_id,
            self.pin,
        )

        try:
            self._on_motion_callback(self.sensor_id)
        except Exception:
            log.exception(
                "PIR sensor %d: error in on_motion callback", self.sensor_id
```

```
            )
```

actuators/led.py

LED and siren actuator controller for active physical deterrence.

```
    import threading
    from gpiozero import LED

    import config
    from utils import get_logger

    log = get_logger(__name__)

    class LEDController:
        """Controls the status LED using gpiozero."""

        def __init__(self) -> None:
            self._led = LED(config.LED_PIN)
            self._led.on()  # On constantly by default
            self._lock = threading.Lock()
            self._timer = None
            log.info("LED initialized on GPIO %d (Constantly ON)", config.LED_PIN)

        def trigger_blink(self) -> None:
            """Blink the LED rapidly for the configured duration, then return to solid
    ON."""
            with self._lock:
                # Cancel any existing timer if motion is detected back-to-back
                if self._timer:
                    self._timer.cancel()

                # Start blinking in the background (0.2s on, 0.2s off)
                self._led.blink(on_time=0.2, off_time=0.2)
                log.info("LED blinking for %ds", config.LED_BLINK_DURATION)

                # Set a timer to return the LED to solid ON after 3 seconds
                self._timer = threading.Timer(config.LED_BLINK_DURATION,
    self._restore_solid)
                self._timer.daemon = True
                self._timer.start()

        def _restore_solid(self) -> None:
            with self._lock:
                self._led.on()
                self._timer = None

        def cleanup(self) -> None:
            self._led.close()
```

api/command_listener.py

SSE command listener; receives remote commands from the cloud backend.

```
    # ============================================================================
    #  api/command_listener.py — SSE-based command listener
    #
    #  Opens a persistent GET /events/stream connection using requests(stream=True)
    #  and parses Server-Sent Events line by line.
    #
    #  SSE wire format (text/event-stream):
    #      event: command
```

```
#      data: {"id":"...","type":"capture","payload":{"camera_id":0}}
#
#      event: heartbeat
#      data: {}
#
#  Reliability
#  ──────────
#  • If the connection drops (network blip, server restart, read timeout):
#    the listener reconnects automatically with exponential backoff
#    (starts at SSE_RECONNECT_BASE_DELAY, caps at SSE_RECONNECT_MAX_DELAY).
#  • Commands missed while offline are the API server's responsibility
#    to re-deliver (e.g. via a persistent session / Last-Event-ID header).
#    The listener sends the Last-Event-ID header on every reconnect so the
#    server can replay any missed events.
#  • Each successfully processed command is ACKed back to the API.
# ============================================================================

import json
import threading
import time
from typing import Callable, Dict, Any, Optional

import requests

import config
from utils import get_logger

log = get_logger(__name__)

_STREAM_URL = config.API_BASE_URL.rstrip("/") + config.ENDPOINT_SSE_STREAM
_ACK_URL    = config.API_BASE_URL.rstrip("/") + config.ENDPOINT_ACK_COMMAND

CommandHandler = Callable[[Dict[str, Any]], None]

_HEADERS = {
    "X-API-Key":  config.API_KEY,
    "X-Node-ID":  config.NODE_ID,
    "Accept":     "text/event-stream",
    "Cache-Control": "no-cache",
}


class CommandListener:
    """
    SSE client that listens for backend commands in a dedicated thread.

    Register handlers before calling start():
        listener.register("capture",   my_capture_fn)
        listener.register("siren",     my_siren_fn)
        listener.register("siren_off", my_siren_off_fn)
    """

    def __init__(self) -> None:
        self._handlers: Dict[str, CommandHandler] = {}
        self._stop_event = threading.Event()
        self._thread: Optional[threading.Thread] = None
        self._last_event_id: Optional[str] = None  # for SSE resume

    # ------------------------------------------------------------------
    # Public API
    # ------------------------------------------------------------------

    def register(self, command_type: str, handler: CommandHandler) -> None:
```

```
        self._handlers[command_type] = handler
        log.debug("Registered SSE handler for command type '%s'", command_type)

    def start(self) -> None:
        self._stop_event.clear()
        self._thread = threading.Thread(
            target=self._listener_loop,
            name="sse-command-listener",
            daemon=True,
        )
        self._thread.start()
        log.info("CommandListener (SSE) started  →  %s", _STREAM_URL)

    def stop(self) -> None:
        self._stop_event.set()
        if self._thread and self._thread.is_alive():
            self._thread.join(timeout=config.SSE_READ_TIMEOUT + 2)
        log.info("CommandListener stopped")

    # ------------------------------------------------------------------
    # Listener loop — runs in its own thread
    # ------------------------------------------------------------------

    def _listener_loop(self) -> None:
        backoff = config.SSE_RECONNECT_BASE_DELAY

        while not self._stop_event.is_set():
            try:
                self._connect_and_read()
                # Only reaches here on a clean mid-stream disconnect (the
                # connection was established and then dropped normally).
                # Reset backoff — the server is reachable.
                backoff = config.SSE_RECONNECT_BASE_DELAY

            except requests.RequestException as exc:
                # Connection was never established (API down, DNS failure,
                # timeout before first byte, non-2xx status).
                # Do NOT reset backoff — let it grow so we don't hammer a
                # server that is clearly unavailable.
                log.warning("SSE connect failed: %s — retrying in %.0fs", exc,
backoff)

            except Exception:  # noqa: BLE001
                # Unexpected non-network error — don't reset backoff either
                log.exception("Unexpected error in SSE listener loop")

            if self._stop_event.is_set():
                break

            log.info("SSE reconnecting in %.0fs …", backoff)
            self._stop_event.wait(timeout=backoff)
            backoff = min(
                backoff * config.SSE_RECONNECT_BACKOFF_FACTOR,
                config.SSE_RECONNECT_MAX_DELAY,
            )

    def _connect_and_read(self) -> None:
        """
        Open one SSE connection and read events until the connection closes
        or _stop_event is set.
        """
        headers = dict(_HEADERS)
        if self._last_event_id:
```

```
    headers["Last-Event-ID"] = self._last_event_id

log.info("Opening SSE stream connection …")

# Let requests.RequestException propagate to _listener_loop so it
# can apply the correct backoff (not reset it on a failed connect).
resp = requests.get(
    _STREAM_URL,
    headers=headers,
    params={"node_id": config.NODE_ID},
    stream=True,
    timeout=(config.REQUEST_TIMEOUT[0], config.SSE_READ_TIMEOUT),
)
resp.raise_for_status()

log.info("SSE stream connected (HTTP %d)", resp.status_code)

# SSE parser state
event_type = "message"
data_lines = []
event_id   = None

try:
    for raw_line in resp.iter_lines(
        chunk_size=None, decode_unicode=True
    ):
        if self._stop_event.is_set():
            break

        # ── Per-event try/except ─────────────────────────────
        # Catches parsing errors and bad dispatch payloads for a
        # single event (UnicodeDecodeError, KeyError, json errors,
        # unexpected exceptions from handlers, etc.).
        # The connection stays open — only the offending event is
        # skipped.  Network errors from iter_lines() still propagate
        # to the outer except so the connection is closed and
        # reconnected correctly.
        try:
            # Empty line → dispatch accumulated event
            if raw_line == "":
                if data_lines:
                    self._dispatch(
                        event_type=event_type,
                        data="\n".join(data_lines),
                        event_id=event_id,
                    )
                # Reset parser state for next event
                event_type = "message"
                data_lines = []
                event_id   = None
                continue

            if raw_line.startswith(":"):
                # SSE comment / keep-alive — ignore
                continue

            if ":" in raw_line:
                field, _, value = raw_line.partition(":")
                value = value.lstrip(" ")
            else:
                field = raw_line
                value = ""
```

```
                if field == "data":
                    data_lines.append(value)
                elif field == "event":
                    event_type = value
                elif field == "id":
                    event_id = value
                    self._last_event_id = value
                elif field == "retry":
                    # Server suggests a reconnect interval (ms) — our
                    # config values take precedence.
                    pass

            except Exception as exc:  # noqa: BLE001
                log.error(
                    "SSE: error processing event (skipping) — %s: %s",
                    type(exc).__name__, exc,
                )
                # Reset parser state so the next event starts clean
                event_type = "message"
                data_lines = []
                event_id   = None

    except requests.RequestException as exc:
        # Network-level failure — close and reconnect
        log.warning("SSE stream error: %s", exc)
    finally:
        resp.close()

# ------------------------------------------------------------------
# Event dispatch
# ------------------------------------------------------------------

def _dispatch(self, event_type: str, data: str, event_id: Optional[str]) -> None:
    # Ignore heartbeat / keep-alive events
    if event_type in ("heartbeat", "ping", "message") and data in ("{}", ""):
        log.debug("SSE heartbeat received")
        return

    if event_type != "command":
        log.debug("SSE: ignoring event type '%s'", event_type)
        return

    try:
        cmd = json.loads(data)
    except json.JSONDecodeError:
        log.error("SSE: could not parse command JSON: %r", data)
        return

    cmd_id   = cmd.get("id", event_id or "unknown")
    cmd_type = cmd.get("type", "")
    payload  = cmd.get("payload", {})

    log.info(
        "SSE command received: id=%s  type=%s  payload=%s",
        cmd_id, cmd_type, payload,
    )

    handler = self._handlers.get(cmd_type)
    if handler is None:
        log.warning("No handler for command type '%s'", cmd_type)
        self._ack(cmd_id, success=False, error=f"Unknown type: {cmd_type}")
        return
```

```
            try:
                handler(payload)
                self._ack(cmd_id, success=True)
            except Exception as exc:  # noqa: BLE001
                log.exception("Handler for '%s' raised an exception", cmd_type)
                self._ack(cmd_id, success=False, error=str(exc))

        # ------------------------------------------------------------------
        # ACK
        # ------------------------------------------------------------------

        def _ack(self, cmd_id: str, success: bool, error: str = "") -> None:
            url  = _ACK_URL.format(id=cmd_id)
            body: dict = {"success": success}
            if error:
                body["error"] = error
            try:
                requests.post(
                    url,
                    json=body,
                    headers={
                        "X-API-Key": config.API_KEY,
                        "X-Node-ID": config.NODE_ID,
                    },
                    timeout=config.REQUEST_TIMEOUT,
                )
                log.debug("ACK sent for command %s (success=%s)", cmd_id, success)
            except requests.RequestException as exc:
                log.warning("Failed to ACK command %s: %s", cmd_id, exc)
```

`api/uploader.py`

Uploads captured images and structured event data to the cloud backend API.

```
    # ============================================================================
    #  api/uploader.py — Persistent upload worker
    #
    #  Design
    #  ──────
    #  • Captures are written to the SQLite queue immediately (never lost)
    #  • A single worker thread drains the queue continuously
    #  • Before each upload attempt, reachability is checked via a lightweight
    #    socket connect — if offline, the worker backs off exponentially and
    #    retries without burning bandwidth
    #  • On success  → row deleted immediately from the queue
    #  • On failure  → attempt counter incremented; if under the retry limit
    #                  the row goes back to pending; otherwise marked failed
    #                  and purged after the retention window
    #  • A periodic purge thread cleans up old failed rows
    # ============================================================================

    import json
    import os
    import socket
    import threading
    import time
    import urllib.parse

    import requests

    import config
    from db import QueueStore
    from utils import get_logger
```

```
log = get_logger(__name__)

_UPLOAD_URL = config.API_BASE_URL.rstrip("/") + config.ENDPOINT_EVENT_UPLOAD

_SESSION = requests.Session()
_SESSION.headers.update({
    "X-API-Key": config.API_KEY,
    "X-Node-ID": config.NODE_ID,
})


def _api_host() -> tuple:
    """Parse the API base URL into (hostname, port) for reachability checks."""
    parsed = urllib.parse.urlparse(config.API_BASE_URL)
    host   = parsed.hostname or "8.8.8.8"
    port   = parsed.port or (443 if parsed.scheme == "https" else 80)
    return host, port


def _is_reachable() -> bool:
    """
    Lightweight connectivity check — attempts a TCP connect to the API host.
    Returns True if the socket handshake succeeds within 3 seconds.
    """
    host, port = _api_host()
    try:
        with socket.create_connection((host, port), timeout=3):
            return True
    except OSError:
        return False


class EventUploader:
    """
    Persistent, connectivity-aware upload worker.

    enqueue() is the only method callers need — it writes the job to SQLite
    and returns immediately.  The background worker thread handles the rest.
    """

    def __init__(self, queue_store: QueueStore) -> None:
        self._store = queue_store
        self._stop_event = threading.Event()

        # Worker thread — drains the queue
        self._worker = threading.Thread(
            target=self._worker_loop,
            name="upload-worker",
            daemon=True,
        )

        # Purge thread — removes stale failed rows periodically
        self._purger = threading.Thread(
            target=self._purge_loop,
            name="upload-purger",
            daemon=True,
        )

        self._worker.start()
        self._purger.start()
        log.info("EventUploader started  →  %s", _UPLOAD_URL)
```

```
# ---------------------------------------------------------------------
# Public API
# ---------------------------------------------------------------------

def enqueue(self, filepath: str, metadata: dict) -> None:
    """
    Persist an upload job to SQLite and return immediately.
    The worker thread will pick it up asynchronously.
    """
    metadata_json = json.dumps(metadata)
    job_id = self._store.enqueue(filepath, metadata_json)
    log.info(
        "Upload job #%d queued: %s", job_id, os.path.basename(filepath)
    )

def shutdown(self, timeout: float = 20.0) -> None:
    """Signal the worker to stop and wait for it to finish."""
    log.info("EventUploader: shutting down …")
    self._stop_event.set()
    self._worker.join(timeout=timeout)
    self._purger.join(timeout=5.0)

# ---------------------------------------------------------------------
# Worker loop
# ---------------------------------------------------------------------

def _worker_loop(self) -> None:
    backoff = config.UPLOAD_BACKOFF_BASE

    while not self._stop_event.is_set():
        job = self._store.claim_next()

        if job is None:
            # Queue is empty — idle wait before checking again
            self._stop_event.wait(timeout=2.0)
            continue

        # ── Connectivity check ──────────────────────────────
        if not _is_reachable():
            log.warning(
                "Network unreachable — backing off %.0fs before retrying job #%d",
                backoff, job["id"],
            )
            # Release the claim back to pending WITHOUT counting an attempt
            # — the job is not at fault, the network is
            self._store.release_to_pending(job["id"])
            self._stop_event.wait(timeout=backoff)
            backoff = min(
                backoff * config.UPLOAD_BACKOFF_FACTOR,
                config.UPLOAD_BACKOFF_MAX,
            )
            continue

        # Network is up — reset backoff
        backoff = config.UPLOAD_BACKOFF_BASE

        # ── Upload attempt ──────────────────────────────────
        success = self._attempt_upload(job)

        if success:
            self._store.mark_done(job["id"])
            if not config.KEEP_LOCAL_COPY:
                self._delete_file(job["filepath"])
```

```
            else:
                will_retry = self._store.mark_failed_attempt(job["id"])
                if will_retry:
                    self._stop_event.wait(timeout=config.UPLOAD_BACKOFF_BASE)
                else:
                    # Job permanently failed — delete the image file so it
                    # doesn't accumulate on disk indefinitely
                    if not config.KEEP_LOCAL_COPY:
                        self._delete_file(job["filepath"])
                    log.error(
                        "Job #%d permanently abandoned after %d attempts — file
removed",
                        job["id"], config.UPLOAD_MAX_RETRIES,
                    )

    def _attempt_upload(self, job: dict) -> bool:
        """
        POST the image and metadata to the API.
        Returns True on HTTP 2xx, False on any error.
        """
        filepath      = job["filepath"]
        metadata_json = job["metadata_json"]
        job_id        = job["id"]

        if not os.path.exists(filepath):
            log.error(
                "Job #%d: file no longer exists (%s) — dropping", job_id, filepath
            )
            # Treat as done so we don't loop forever on a missing file
            self._store.mark_done(job_id)
            return True

        try:
            with open(filepath, "rb") as fh:
                files = {"file": (os.path.basename(filepath), fh, "image/jpeg")}
                data  = {"metadata": metadata_json}
                resp  = _SESSION.post(
                    _UPLOAD_URL,
                    files=files,
                    data=data,
                    timeout=config.REQUEST_TIMEOUT,
                )
            resp.raise_for_status()
            event_id = resp.json().get("event_id", "?")
            log.info(
                "Job #%d uploaded (event_id=%s, HTTP %d)",
                job_id, event_id, resp.status_code,
            )
            return True

        except requests.RequestException as exc:
            log.warning("Job #%d upload failed: %s", job_id, exc)
            return False
        except OSError as exc:
            log.error("Job #%d file read error: %s", job_id, exc)
            return False

    # ------------------------------------------------------------------
    # Purge loop
    # ------------------------------------------------------------------

    def _purge_loop(self) -> None:
        """Periodically clean up stale failed rows from the queue DB."""
```

```
        while not self._stop_event.is_set():
            self._stop_event.wait(timeout=600)  # every 10 minutes
            if not self._stop_event.is_set():
                self._store.purge_failed()

    # ------------------------------------------------------------------
    # Helpers
    # ------------------------------------------------------------------

    @staticmethod
    def _delete_file(filepath: str) -> None:
        try:
            os.remove(filepath)
            log.debug("Deleted local file: %s", filepath)
        except OSError as exc:
            log.warning("Could not delete %s: %s", filepath, exc)
```

`core/event_handler.py`

Event routing logic connecting sensors, actuators, and the cloud upload pipeline.

```
# ============================================================================
#  core/event_handler.py — Orchestrates the PIR → Camera → Upload pipeline
#
#  When a PIR sensor fires, the sensor_id is used to select which camera to
#  use (sensor 0 → camera 0, sensor 1 → camera 1).  This is configurable
#  via SENSOR_CAMERA_MAP below.
# ============================================================================


from typing import Dict

from sensors.camera import CameraManager
from api.uploader import EventUploader
from actuators.led import LEDController
from utils import get_logger

log = get_logger(__name__)

# Maps sensor_id → camera_id
SENSOR_CAMERA_MAP: Dict[int, int] = {
    0: 0,
    1: 2,  # Updated to point to /dev/video2!
}

class EventHandler:
    def __init__(
        self,
        camera_manager: CameraManager,
        uploader: EventUploader,
        led_controller: LEDController
    ) -> None:
        self._cameras  = camera_manager
        self._uploader = uploader
        self._led = led_controller

    def on_pir_motion(self, sensor_id: int) -> None:
        # 1. Trigger the LED blink immediately!
        self._led.trigger_blink()

        camera_id = SENSOR_CAMERA_MAP.get(sensor_id)
        if camera_id is None:
            log.warning("PIR sensor %d triggered but no camera mapping defined",
```

```
sensor_id)
            return

        log.info("PIR sensor %d → triggering camera %d capture", sensor_id, camera_id)

        filepath, metadata = self._cameras.capture_image(
            camera_id=camera_id,
            reason="pir_trigger",
            sensor_id=sensor_id,
        )

        if filepath:
            self._uploader.enqueue(filepath, metadata)
        else:
            log.error("PIR sensor %d: capture on camera %d failed", sensor_id,
camera_id)

    def on_remote_capture(self, camera_id: int, reason: str = "remote_command") ->
None:
        log.info("Remote capture triggered: camera %d", camera_id)
        filepath, metadata = self._cameras.capture_image(
            camera_id=camera_id,
            reason=reason,
        )
        if filepath:
            self._uploader.enqueue(filepath, metadata)
        else:
            log.error("Remote capture failed for camera %d", camera_id)
```

`core/command_processor.py`

Processes incoming remote commands received through the SSE listener.

```
# ============================================================================
#  core/command_processor.py — Translate API commands → hardware actions
#
#  Registered with CommandListener so each command type has a clear,
#  validated handler.  Validation errors raise ValueError so the listener
#  can NACK the command back to the API with a meaningful error message.
# ============================================================================

from typing import Any, Dict

from core.event_handler import EventHandler
from utils import get_logger
import config

log = get_logger(__name__)

class CommandProcessor:
    """Maps incoming API command payloads to hardware actions."""

    def __init__(self, event_handler: EventHandler) -> None:
        self._events = event_handler

    def handle_capture(self, payload: Dict[str, Any]) -> None:
        camera_id = payload.get("camera_id")
        if camera_id is None:
            raise ValueError("'camera_id' is required in capture command payload")

        camera_id = int(camera_id)
        if camera_id not in config.CAMERA_INDICES:
            raise ValueError(
```

```
                f"camera_id {camera_id} is not in configured indices
{config.CAMERA_INDICES}"
            )

        log.info("[CMD] capture  camera_id=%d", camera_id)
        self._events.on_remote_capture(camera_id=camera_id)

    def register_all(self, listener) -> None:
        listener.register("capture", self.handle_capture)
        log.info("All command handlers registered")
```

db/queue_store.py

Persistent local SQLite queue enabling offline-tolerant, retry-safe image upload.

```
# ============================================================================
#  db/queue_store.py — SQLite-backed persistent upload queue
#
#  This is a transient queue, NOT an event log.
#  - Rows are deleted immediately on successful upload
#  - Failed rows are purged after UPLOAD_FAILED_RETENTION_SECONDS
#  - Under normal operation the table is nearly empty
#
#  Job lifecycle:
#
#       capture → INSERT (pending)
#                      │
#                      ▼
#               worker picks up → UPDATE (uploading)
#                      │
#            ┌─────────┴──────────┐
#      success                  failure
#         │                       │
#      DELETE row           UPDATE attempts++
#                                 │
#                     max retries reached?
#                    ┌────────────┴─────────┐
#                  yes                      no
#                   │                       │
#             UPDATE (failed)         stays (uploading→pending)
#             purged after                retry later
#             retention period
# ============================================================================

import os
import sqlite3
import threading
import time
from contextlib import contextmanager
from typing import List, Optional

import config
from utils import get_logger

log = get_logger(__name__)

# Job status constants
STATUS_PENDING    = "pending"
STATUS_UPLOADING  = "uploading"
STATUS_DONE       = "done"
STATUS_FAILED     = "failed"

_SCHEMA = """
```

```
CREATE TABLE IF NOT EXISTS upload_jobs (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    filepath      TEXT    NOT NULL,
    metadata_json TEXT    NOT NULL,
    status        TEXT    NOT NULL DEFAULT 'pending',
    attempts      INTEGER NOT NULL DEFAULT 0,
    created_at    REAL    NOT NULL,
    updated_at    REAL    NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_status ON upload_jobs(status);
CREATE INDEX IF NOT EXISTS idx_updated_at ON upload_jobs(updated_at);
"""


class QueueStore:
    """
    Thread-safe SQLite queue for upload jobs.

    claim_next() uses an application-level lock to make its SELECT+UPDATE
    atomic. All other methods rely on SQLite WAL mode for safe concurrent
    access — single-statement operations are atomic at the database level
    and do not require an additional application lock.
    """

    def __init__(self, db_path: str = config.DB_PATH) -> None:
        self._db_path = db_path
        self._lock = threading.Lock()

        os.makedirs(os.path.dirname(db_path), exist_ok=True)
        self._init_schema()
        self._recover_interrupted()
        self._purge_failed()
        log.info("QueueStore ready at %s", db_path)

    # ------------------------------------------------------------------
    # Public API
    # ------------------------------------------------------------------

    def enqueue(self, filepath: str, metadata_json: str) -> int:
        """Insert a new pending job. Returns the new row id."""
        now = time.time()
        with self._connect() as conn:
            cur = conn.execute(
                "INSERT INTO upload_jobs (filepath, metadata_json, status, attempts,
created_at, updated_at) "
                "VALUES (?, ?, ?, 0, ?, ?)",
                (filepath, metadata_json, STATUS_PENDING, now, now),
            )
            row_id = cur.lastrowid
        log.debug("Enqueued job #%d: %s", row_id, os.path.basename(filepath))
        return row_id

    def claim_next(self) -> Optional[dict]:
        """
        Atomically claim the next pending job for uploading.
        Returns a dict with job data, or None if the queue is empty.
        """
        now = time.time()
        with self._lock:
            with self._connect() as conn:
                row = conn.execute(
                    "SELECT id, filepath, metadata_json, attempts FROM upload_jobs "
```

```
                    "WHERE status = ? ORDER BY updated_at ASC LIMIT 1",
                    (STATUS_PENDING,),
                ).fetchone()

                if row is None:
                    return None

                conn.execute(
                    "UPDATE upload_jobs SET status = ?, updated_at = ? WHERE id = ?",
                    (STATUS_UPLOADING, now, row["id"]),
                )

        return dict(row)

    def mark_done(self, job_id: int) -> None:
        """Delete the job row immediately — it has been successfully uploaded."""
        with self._connect() as conn:
            conn.execute("DELETE FROM upload_jobs WHERE id = ?", (job_id,))
        log.debug("Job #%d marked done and removed from queue", job_id)

    def release_to_pending(self, job_id: int) -> None:
        """
        Return a claimed job back to pending WITHOUT incrementing the attempt
        counter.  Use this exclusively for transient offline situations where
        no upload was attempted — the job should not be penalised for a
        network outage.
        """
        now = time.time()
        with self._connect() as conn:
            conn.execute(
                "UPDATE upload_jobs SET status = ?, updated_at = ? WHERE id = ?",
                (STATUS_PENDING, now, job_id),
            )
        log.debug("Job #%d released back to pending (no attempt counted)", job_id)

    def mark_failed_attempt(self, job_id: int) -> bool:
        """
        Increment the attempt counter.
        - If under the retry limit: reset to pending so the worker retries it.
        - If at/over the limit: mark as failed (will be purged after retention).
        Returns True if the job will be retried, False if it's been failed out.
        """
        now = time.time()
        with self._connect() as conn:
            row = conn.execute(
                "SELECT attempts FROM upload_jobs WHERE id = ?", (job_id,)
            ).fetchone()

            if row is None:
                return False

            new_attempts = row["attempts"] + 1

            if new_attempts >= config.UPLOAD_MAX_RETRIES:
                conn.execute(
                    "UPDATE upload_jobs SET status = ?, attempts = ?, updated_at = ?
WHERE id = ?",
                    (STATUS_FAILED, new_attempts, now, job_id),
                )
                log.warning(
                    "Job #%d permanently failed after %d attempts", job_id,
new_attempts
                )
```

```
                return False
            else:
                conn.execute(
                    "UPDATE upload_jobs SET status = ?, attempts = ?, updated_at = ?
WHERE id = ?",
                    (STATUS_PENDING, new_attempts, now, job_id),
                )
                log.debug(
                    "Job #%d attempt %d/%d failed — back to pending",
                    job_id, new_attempts, config.UPLOAD_MAX_RETRIES,
                )
                return True

    def pending_count(self) -> int:
        """Return number of jobs waiting to be uploaded."""
        with self._connect() as conn:
            row = conn.execute(
                "SELECT COUNT(*) FROM upload_jobs WHERE status IN (?, ?)",
                (STATUS_PENDING, STATUS_UPLOADING),
            ).fetchone()
        return row[0] if row else 0

    def purge_failed(self) -> None:
        """Remove failed rows that have exceeded the retention window."""
        self._purge_failed()

    # ------------------------------------------------------------------
    # Internal
    # ------------------------------------------------------------------

    @contextmanager
    def _connect(self):
        conn = sqlite3.connect(self._db_path, timeout=10)
        conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA journal_mode=WAL")   # allows concurrent readers
        conn.execute("PRAGMA synchronous=NORMAL") # safe and faster than FULL
        try:
            yield conn
            conn.commit()
        except Exception:
            conn.rollback()
            raise
        finally:
            conn.close()

    def _init_schema(self) -> None:
        with self._connect() as conn:
            conn.executescript(_SCHEMA)

    def _recover_interrupted(self) -> None:
        """
        On startup, any job stuck in 'uploading' was interrupted mid-flight
        (e.g. power loss). Reset them to 'pending' so they are retried.
        """
        now = time.time()
        with self._connect() as conn:
            cur = conn.execute(
                "UPDATE upload_jobs SET status = ?, updated_at = ? WHERE status = ?",
                (STATUS_PENDING, now, STATUS_UPLOADING),
            )
            if cur.rowcount:
                log.info(
                    "Recovered %d interrupted upload job(s) → pending", cur.rowcount
```

```
            )

    def _purge_failed(self) -> None:
        """Delete failed rows older than the configured retention window."""
        cutoff = time.time() - config.UPLOAD_FAILED_RETENTION_SECONDS
        with self._connect() as conn:
            cur = conn.execute(
                "DELETE FROM upload_jobs WHERE status = ? AND updated_at < ?",
                (STATUS_FAILED, cutoff),
            )
            if cur.rowcount:
                log.info("Purged %d expired failed job(s) from queue", cur.rowcount)
```

utils/logger.py

Structured logging utility providing consistent log formatting across all modules.

```
# ============================================================================
#  utils/logger.py — Centralised logging setup
# ============================================================================

import logging
import sys
from logging.handlers import RotatingFileHandler

import config


def get_logger(name: str) -> logging.Logger:
    """Return a named logger, configured once on first call."""
    logger = logging.getLogger(name)

    # Avoid adding duplicate handlers if get_logger() is called multiple times
    if logger.handlers:
        return logger

    logger.setLevel(getattr(logging, config.LOG_LEVEL.upper(), logging.INFO))

    fmt = logging.Formatter(
        fmt="%(asctime)s  [%(levelname)-8s]  %(name)s — %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )

    # Console handler
    ch = logging.StreamHandler(sys.stdout)
    ch.setFormatter(fmt)
    logger.addHandler(ch)

    # Rotating file handler (optional)
    if config.LOG_FILE:
        try:
            fh = RotatingFileHandler(
                config.LOG_FILE,
                maxBytes=config.LOG_MAX_BYTES,
                backupCount=config.LOG_BACKUP_COUNT,
            )
            fh.setFormatter(fmt)
            logger.addHandler(fh)
        except OSError as exc:
            logger.warning("Cannot open log file %s: %s", config.LOG_FILE, exc)

    return logger
```

agri_ids.service

systemd unit file for running the Raspberry Pi agent as a persistent background service.

```
# ================================================================================
#  agri_ids.service — systemd unit for the Agri-IDS hardware node
#
#  Install:
#      sudo cp agri_ids.service /etc/systemd/system/
#      sudo systemctl daemon-reload
#      sudo systemctl enable agri_ids
#      sudo systemctl start agri_ids
#
#  Logs:
#      sudo journalctl -u agri_ids -f
# ================================================================================

[Unit]
Description=Agri-IDS Agricultural Intrusion Detection System
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=root
# Point to your actual project folder
WorkingDirectory=/home/fred/Desktop/agri_ids
# Use the virtual environment Python, not the system Python
ExecStart=/home/fred/Desktop/agri_ids/venv/bin/python main.py

# Put your actual Codespace/Backend URL and Key here!
Environment="AGRI_IDS_API_URL=https://vigilant-space-telegram-9wjjr7pj6wxc6w-
8000.app.github.dev/api"
Environment="AGRI_IDS_API_KEY=REPLACE_ME"
Environment="AGRI_IDS_NODE_ID=node-001"
Environment="AGRI_IDS_LOG_LEVEL=INFO"

Restart=on-failure
RestartSec=10s
StartLimitIntervalSec=60
StartLimitBurst=3
TimeoutStopSec=30
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

requirements.txt

Python package dependencies required on the Raspberry Pi edge node.

```
# FARM-IDS Hardware Node — Python dependencies
# Install with:  pip install -r requirements.txt



# HTTP client — used for uploads AND SSE streaming (stream=True)
requests>=2.31.0

# GPIOZERO - for reading and controlling the raspberry GPIO pins
gpiozero>=2.0.0

# sqlite3 is part of the Python standard library — no install needed
```